AVYORA

Technical and Operations Manual

Architecture, features, security and day-to-day running of the Avyora skincare shop

Version 1.0 · 21 October 2026

Covers the code on branch fix/payment-backout, built from main at ba7e5a3

For the owner, the staff who run the shop, and any developer who works on it next.

Contents

About this manual 9
How the manual is organised 9
Conventions 9
1. Avyora at a glance 10
What the system does 10
Where it runs 10
The state of things today 11
2. Who uses the system 11
Customers 11
The stockroom manager 11
The owner 12
The systems that act on their own 12
3. Architecture overview 12
The big picture 12
Three kinds of server code 13
A request, end to end: placing a cash-on-delivery order 13
Design principles 14
4. Technology stack 14
Main components 14
Outside services and what happens if each is down 15
5. How the code is organised 16
Modules 16
Naming conventions worth knowing 17
6. Data and storage 17
The data, grouped 17
How an order is stored 18
Reading and writing 18
Connections 19
Migrations 19
Backups and recovery 19
7. Events and background work 20
The event log (transactional outbox) 20
Consumers 20
When events are processed 21
The job queue 21
Scheduled jobs 21
8. Caching and rendering 21
How each kind of page is served 22

The tiered cache ... 22
When the database is down ... 22
Page weight ... 23
9. Search engines, accessibility and design ... 23
Search engines ... 23
Accessibility ... 24
The design system ... 24
Photography ... 25
10. Customer journeys ... 25
Journey 1. A first visit that ends in a cash-on-delivery order ... 25
Journey 2. Paying online, then changing her mind ... 26
Journey 3. Building a routine ... 26
Journey 4. A question at midnight ... 27
Journey 5. Tracking without an account ... 27
Journey 6. A parcel that comes back ... 27
11. Catalogue and storefront ... 27
The catalogue ... 27
Catalogue mode ... 28
Shop pages ... 28
Stock labels ... 28
Honest content rules ... 29
12. Bag and checkout ... 29
The bag ... 29
Checkout ... 30
After payment is abandoned ... 30
13. Payments ... 30
Methods ... 30
Three rules that protect the money ... 31
The payment state machine ... 31
Recording provider events exactly once ... 31
Cash on delivery ... 31
Setting up Cashfree for live payments ... 32
14. Orders, tracking and notifications ... 32
What the customer sees ... 32
Who can see an order ... 32
Messages sent ... 32
15. What customers and staff are told ... 33
Order confirmation (customer email) ... 33
Shipped (customer email) ... 33
New order (owner email and WhatsApp) ... 34

Morning summary (owner) ... 34
Sign-in codes ... 34
On screen: checkout ... 34
On screen: the skin chat ... 35
On screen: staff ... 35
Changing the wording ... 36
16. Accounts and sign-in ... 36
Customer sign-in ... 36
The account area ... 36
Guests who own things ... 36
17. Routine finder and the knowledge base ... 37
The questions ... 37
How a routine is computed ... 37
Saving routines ... 38
The knowledge base ... 38
Optional face scan (switched off) ... 38
18. Ask Avyora ... 39
19. AI skin chat ... 39
Who can use it ... 39
The model and what it costs ... 39
How it behaves ... 40
Urgent symptoms ... 40
Privacy ... 40
Turning it on ... 40
20. The stockroom ... 41
Dispatch ... 41
Sending an order ... 41
Holds ... 42
Stock ... 42
Help ... 42
21. The owner console ... 42
Signing in ... 42
Screens ... 42
Prices and offers ... 43
Revenue ... 43
Orders held by the cash-on-delivery risk check ... 43
Orders that need attention ... 44
The audit log ... 44
22. Reading the numbers ... 44
What counts as a sale ... 45

The overview ... 45
Analytics (last 30 days) ... 45
How much to trust the estimate ... 46
The morning autopilot's numbers ... 46
Comparing with the payment provider ... 46
Where the figures come from ... 46
23. Deliveries with Shiprocket ... 46
What happens when the manager presses Book courier ... 46
Tracking updates ... 47
Setting it up ... 47
24. Stock autopilot ... 48
The rule ... 48
What it does ... 48
Tuning ... 48
25. Content and the CMS ... 49
Drafts, publishing and revisions ... 49
Media ... 49
Product copy and directions ... 49
26. Security ... 50
Getting products for free ... 50
Taking customer data ... 51
Staff access ... 52
Abuse and cost control ... 52
The AI skin chat ... 53
Security headers ... 53
Secrets ... 53
What is still open ... 53
27. Privacy and data protection ... 54
What is collected, and why ... 54
Permissions (consent) ... 55
How long data is kept ... 55
Who else receives data ... 55
Customer rights in practice ... 56
Safeguards built into the system ... 56
Owner decisions still needed ... 56
28. Deployment and settings ... 57
Environments ... 57
Deploying to production ... 57
Rolling back ... 58
Settings that matter most ... 58

Feature switches ... 59
Google sign-in ... 59
Commands ... 59
29. Setting up the outside services ... 60
Neon (the database) ... 60
Vercel (hosting) ... 60
Google sign-in ... 60
Email: Gmail or Resend ... 61
WhatsApp alerts (Meta Cloud API) ... 61
SMS codes (MSG91, optional) ... 61
Cashfree (online payments) ... 62
Shiprocket (couriers) ... 62
Anthropic (the AI skin chat) ... 62
Sentry (error alerts, optional) ... 62
Upstash Redis (shared cache, optional) ... 63
Nightly backups ... 63
Staff passwords ... 63
30. Scaling and performance ... 63
How the system scales ... 63
Load test (12 October 2026) ... 64
Plan limits ... 64
What to change, and when ... 64
Performance measured ... 65
31. Monitoring, incidents and runbooks ... 65
Where to look ... 65
Runbooks ... 65
32. Testing and quality ... 66
The test suite ... 66
How tests are trusted ... 67
Areas and their tests ... 67
Continuous integration ... 67
Acceptance checks ... 68
33. What is left before launch ... 68
Must do before taking real orders ... 68
Should do soon after ... 69
Waiting for approval ... 69
Not built, by design ... 69
34. Launch-day checklist ... 70
One week before ... 70
The day before ... 70

Launch morning: a real end-to-end test ... 70
The first week ... 71
35. History of the build ... 71
Foundations (7 to 8 October) ... 72
Personalisation (8 October) ... 72
Design and experience (8 to 9 October) ... 72
After the plan ... 73
Quality over time ... 73
36. Decision log ... 73
Money and orders ... 74
Payments ... 75
Data and infrastructure ... 75
Personalisation and content ... 76
Operations ... 77
37. Key flows, step by step ... 77
Online payment with Cashfree ... 77
Backing out of the payment screen ... 78
A parcel's life ... 78
A chat message ... 79
The morning sweep ... 79
38. Order and payment lifecycle ... 79
Fulfilment status ... 79
Payment status ... 80
Risk status (cash on delivery only) ... 81
Attention ... 81
What the customer sees for each state ... 81
39. API reference ... 81
Common status codes ... 82
Catalogue ... 82
Bag, wishlist and sync ... 82
Payments ... 83
Deliveries ... 83
Staff ... 83
Personal records ... 84
Customer contact ... 84
AI chat ... 84
Operations ... 85
40. Making common changes ... 85
Getting started ... 85
Add a product or a size ... 85

Change the delivery fee or free-delivery threshold ... 86
Add a setting ... 86
Change the database schema ... 86
Add an API route safely ... 86
Add an owner-only action ... 87
Add an event and a reaction to it ... 87
Change a rate limit without deploying ... 87
Change the AI chat's behaviour ... 88
Add a CMS content type ... 88
Write a test ... 88
Rebuild this manual ... 88
41. Quick-reference cards ... 88
Card 1. Stockroom: sending today's orders ... 89
Card 2. Stockroom: stock arrived or removed ... 89
Card 3. Stockroom: a parcel came back ... 89
Card 4. Owner: every morning (5 minutes) ... 89
Card 5. Owner: every week ... 90
Card 6. Owner: changing a price or running an offer ... 90
Card 7. Something is wrong ... 90
Card 8. Who to contact ... 90
42. Owner questions and answers ... 91
Orders and money ... 91
Stock ... 91
Deliveries ... 91
Staff and access ... 92
The AI chat ... 92
The site ... 92
43. Glossary ... 93
Appendix A. Database tables ... 94
Status values ... 94
Appendix B. Pages and API addresses ... 112
Pages ... 112
API addresses ... 113
Appendix C. Settings (environment variables) ... 114
Appendix D. Database migrations ... 118
Appendix E. Automated test files ... 119

If the contents list is empty, right-click it in Word and choose Update Field.

About this manual

This manual describes the Avyora skincare shop as it exists on 21 October 2026: what it does, how it is built, how it keeps money and customer data safe, and how to run it day to day. It was written from the code, not from plans. Where a feature is built but switched off, or designed but not yet built, the manual says so plainly.

It is written for three readers:

- **The owner.** Parts I, IV, VI, VII and IX explain what the system does, what needs a decision, and what to do when something goes wrong. No programming knowledge is needed.
- **The staff.** Chapter 20 (the stockroom), the cards in Chapter 41 and the in-app Help page cover everything the person who packs parcels needs.
- **A developer** joining later. Parts II, III, V and VIII explain the architecture, the reasoning behind the important decisions, and where each piece lives in the code. The appendices list every table, page, API address and setting, generated directly from the code.

How the manual is organised

Part	What it covers
I. The shop	What Avyora sells, who uses the system, and the shop at a glance
II. Architecture	How the system is put together: hosting, layers, data, events, caching, search and design
III. Customer features	Customer journeys, then everything a shopper touches, from the home page to the AI skin chat
IV. Running the shop	The owner console, the stockroom, Shiprocket deliveries, stock autopilot, the CMS
V. Security and privacy	How orders, money and personal data are protected, what data is held and for how long
VI. Operating the system	Deploying, setting up each outside service, scaling, monitoring and incidents, testing
VII. History and next steps	What is left before launch, a launch-day checklist, how it was built, and why
VIII. Developer reference	The main flows, the API, and recipes for common changes
IX. Quick reference	Printable cards, owner questions and answers, and a glossary
Appendices	Generated reference: tables, pages, APIs, settings, migrations, tests

Conventions

- **Money** is written in rupees in the text (₹1,199) and stored as whole paise in the system (119,900 paise). Paise avoid rounding errors.
- **File paths** such as `src/lib/orders.ts` point to where something lives in the code.
- **Notes** in shaded boxes are things worth stopping for: a decision, a limit, or a common mistake.
- **"Switched off"** means the code is built and tested but a setting keeps it hidden until the owner turns it on.

> **Where to start.** If you only read one chapter, read Chapter 1 (Avyora at a glance). If something is broken right now, go to Chapter 31 (incidents and runbooks).

PART I

The shop

What Avyora sells, who uses each part of the system, and where things stand today.

1. Avyora at a glance

Avyora is an online skincare shop for customers in India. It sells a range of 27 products in 30 sizes: cleansers, exfoliants, toners, essences, serums, masks, moisturisers, sunscreens and body care. What sets it apart from a plain shop is that it helps people build a simple routine: a questionnaire produces a seven-day morning and evening plan from products that fit their skin, their budget and what they already own, and an AI guide answers everyday skin questions.

What the system does

Area	In one line
Shop	Browse, filter and compare products, with prices and stock that match what checkout charges
Checkout	Pay online through Cashfree (UPI, cards, net banking) or cash on delivery
Orders	Confirmation email, a private order page, tracking by number and email
Deliveries	Shiprocket books the courier and prints the label; tracking updates move the order by themselves
Stock	Reserved the moment an order is placed, returned if payment is abandoned or a parcel comes back
Autopilot	Every morning, works out what is running low, raises restock requests, and messages the owner
Routine finder	A questionnaire that produces a safe weekly routine within a budget
Ask Avyora	Instant answers about ingredients and products, computed on the customer's device, no AI
AI skin chat	A conversation with an AI skincare guide for signed-in customers, capped at ₹0.50 per person per day
Owner console	Orders, prices and offers, stock, analytics, restock approvals, content, reviews, system health
Stockroom	The dispatch queue, stock counts, restock requests, and a help page
Content (CMS)	Product copy, journal articles and media, with drafts, revisions and publish checks

Where it runs

- **Website and servers:** Vercel, in its Singapore region (`sin1`), at https://avyora-beauty.vercel.app.
- **Database:** Neon Postgres (version 18), in AWS Singapore. This is the single source of truth for orders, payments, stock and accounts.
- **Payments:** Cashfree (currently in test mode). Razorpay code remains only to settle any older Razorpay orders.
- **Couriers:** Shiprocket, once its login is added.
- **Email:** Gmail or Resend for order emails and sign-in codes. **WhatsApp:** Meta's WhatsApp Cloud API for the owner's order alerts.
- **AI:** Anthropic's Claude Haiku 5.5, for the skin chat only.

The state of things today

The system is complete enough to take real orders, but a full public launch is waiting on business inputs rather than code. The main ones, covered in Chapter 33:

- **Sample catalogue.** The live catalogue is in `sample` mode: product records are placeholders, and production refuses real orders until verified product records are loaded.
- **Payments in test mode.** Cashfree runs against its sandbox; live keys need business verification (KYC).
- **Plan limits.** Vercel's free Hobby plan does not allow commercial use. It must move to Pro (about $20 a month) before selling.
- **Business details.** Policy pages still carry `[TO CONFIRM]` markers for the registered business name, address, GSTIN and grievance officer.
- **Photography and product information.** Approved product photos, full ingredient lists and reviewed usage directions are not in the system yet.

> **Note.** None of these are bugs. The system deliberately refuses to sell placeholder products or show unreviewed medical-sounding claims until the real information exists.

2. Who uses the system

Three kinds of people use Avyora, and each sees a different part of it.

Who	Where they go	What they can do
Customer	The shop itself	Browse, buy, track orders, save routines, chat with the AI guide
Stockroom manager	`/manager`	Pack and send orders, book couriers, count stock, ask for more
Owner	`/admin`	Everything the manager can, plus prices, offers, analytics, approvals, content and system health

Both staff roles sign in at the same page, `/admin-login`, and land in different places. The manager cannot open the owner's screens. The sign-in link being public is fine: it is a locked door, not a hidden one. Every owner-only action is checked again on the server, so even a manager who guessed an address could not change a price.

Customers

Customers can shop as guests or sign in with Google, an email code, a password, or a phone code. Signing in brings their bag and wishlist with them across devices, keeps their addresses and order history, and lets them save routines. The AI skin chat is for signed-in customers only, because its daily spending limit is per account.

The stockroom manager

The manager's world is a queue: what has to leave the building today. The dispatch screen lists orders oldest first, each with everything needed to pack it: what goes in, who it is for, where it goes, and a phone number for the courier. The manager presses a button for each step (packed, book courier), and from then on the

courier's own updates move the order. The manager can also adjust stock counts and request more stock, but cannot cancel orders or change prices: both involve money, which is the owner's decision.

The owner

The owner sees everything the manager sees, plus the money: revenue, sales by product, days of stock cover, prices and offers. The owner approves restock requests, decides on orders held by the cash-on-delivery risk check, publishes content and knowledge, moderates reviews, and watches system health. Owner sign-in needs the password and a six-digit code emailed to the owner's address.

The systems that act on their own

Some work happens without anyone pressing a button:

What	When	Effect
Payment webhooks	When Cashfree reports a payment	Marks the order paid, sends the confirmation, alerts the owner
Courier webhooks	When Shiprocket reports a parcel's progress	Moves the order to shipped, out for delivery, delivered or returned
Event drain	Straight after each order or payment, plus daily	Sends emails, runs the risk check, refreshes cached pages
Morning sweep	Daily, 08:45 India time	Releases stock from abandoned payments, raises restock requests, sends the morning summary, deletes expired private data
Event backstop	Daily, 09:15 India time	Retries any email or event that failed the first time

PART II

Architecture

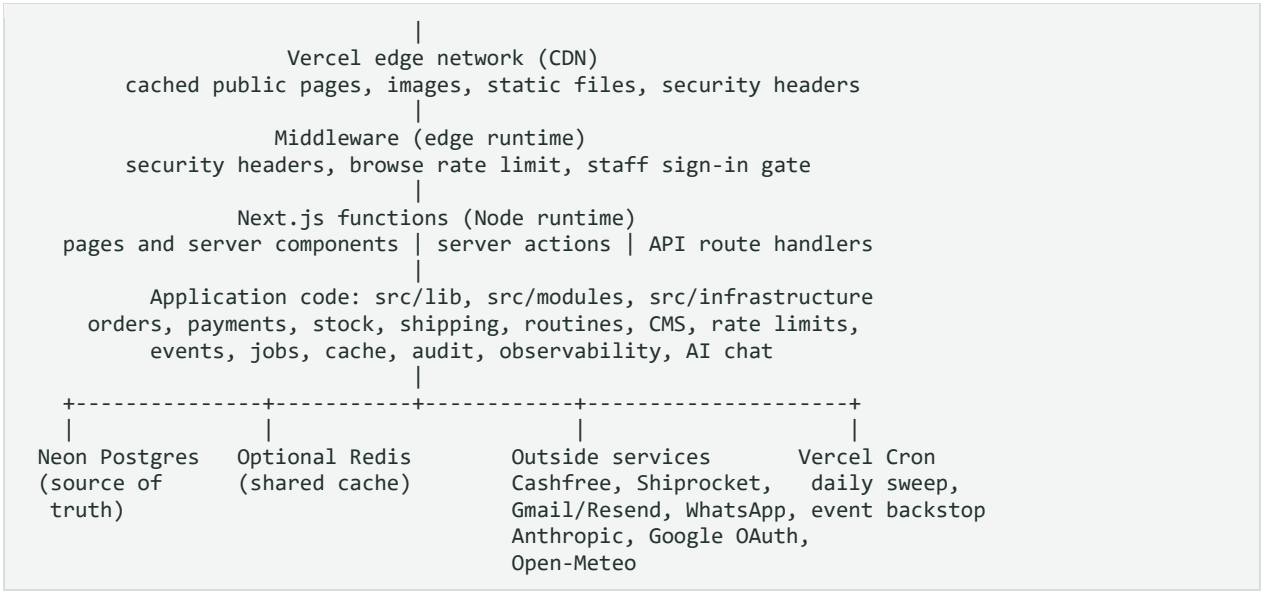
How the system is put together: where it runs, the layers a request passes through, how data is stored, how background work happens, and how pages stay fast.

3. Architecture overview

Avyora is one Next.js application. The same codebase serves the public shop, the customer account area, the owner console, the stockroom and every API. It runs on Vercel as serverless functions, keeps all durable state in one Postgres database, and talks to a handful of outside services for payments, couriers, email, WhatsApp and AI.

The big picture

```
                Customers, staff, search engines
```



Three kinds of server code

Every request that is not a cached file ends up in one of three places:

Kind	Used for	Examples
Server-rendered pages	Pages whose HTML is built on the server, cached where possible	Home, product pages, owner console, dispatch queue
Server actions	Form submissions from the site's own pages	Place order, save price, mark packed, book courier
API route handlers	Anything called by the browser's scripts or by outside services	Payment creation and webhooks, courier webhook, cart sync, AI chat, cron jobs

Server actions carry Next.js's own origin check. API route handlers that change data check the request's `Origin` header against the site's own address (`isSameOrigin` in `src/lib/security.ts`), so another website cannot submit to them on a visitor's behalf. Webhooks from Cashfree and Shiprocket are the exception: they come from outside, so they are verified by a signature or a shared token instead.

A request, end to end: placing a cash-on-delivery order

1. The customer presses **Place order** on `/checkout`. The browser sends only product ids, sizes, quantities, the address and a random idempotency key. It never sends a price.
2. The `placeOrder` server action checks the checkout rate limit (10 per 10 minutes per person and per address).
3. `createOrder` in `src/lib/orders.ts` validates every field, looks up every price on the server (including owner offers), and refuses with "price changed" if the price the customer saw is no longer current.
4. In **one database transaction**, it reserves the stock (an update that only succeeds if enough units remain), saves the address, the order and its lines, and writes an `order.placed` event to the event log.
5. The transaction commits. If two identical requests raced, the unique index on the idempotency key lets only one order exist, and the loser is answered with the winner's order.

6. After the response has gone, the event drain runs: the cash-on-delivery risk check scores the order, the confirmation email and owner alert are sent, and cached pages showing that product's stock are refreshed.
7. The customer lands on `/orders/AVY-XXXXXX`, a private page reachable only by the signed-in owner of the order or with a signed link bound to that order number.

The same pattern (validate on the server, change state in one transaction, record an event, do slow work afterwards) runs through the whole system.

Design principles

These decisions shape almost everything in the codebase.

Principle	What it means in practice
Postgres is the only source of truth	Orders, money, stock, accounts, content, events, jobs and rate limits all live in Postgres. Caches and copies can be lost without losing anything.
Never trust the browser with money	Prices, totals and payment status are always computed or confirmed on the server.
One transaction per state change	Stock, order and event are written together or not at all.
Retries must be harmless	Idempotency keys on orders and routines; payment and courier events recorded once by their provider id; every status change is a compare-and-set.
Fail closed for anything costly or sensitive	Sign-in, checkout, payment, AI chat and staff actions refuse when their safety checks cannot run. Cheap cached reads stay open.
Honest by default	No offer, claim, review, score or direction is shown unless it really exists and was approved.
Portable	Standard Postgres, self-hosted auth, no platform-only SDKs. Moving to AWS later is a change of settings and a Dockerfile, not a rewrite.

4. Technology stack

Main components

Layer	Technology	Version	Why
Framework	Next.js (App Router)	15.5	Server rendering, static generation, server actions and API routes in one codebase
UI	React, Tailwind CSS, Radix UI	React 18.3, Tailwind 3.4	Accessible components, a measured design system (`nv-` tokens)
Language	TypeScript	strict	Types shared by browser and server
Database	PostgreSQL on Neon	18	Managed, branchable (copies for rehearsals), pooled connections for serverless
Database access	Drizzle ORM over postgres-js	Drizzle 0.45	Typed queries; every value sent as a bound parameter
Validation	Zod	3.24	Strict schemas at every trust boundary

Layer	Technology	Version	Why
Customer sign-in	Auth.js (NextAuth v5) with the Drizzle adapter	5.0 beta	Google sign-in, database sessions that can be revoked
Staff sign-in	Own implementation	—	PBKDF2 password hashes, signed httpOnly cookie, emailed second step for the owner
Payments	Cashfree PG (Razorpay kept for older orders)	API 2025-01-01	Indian payment methods, UPI
Couriers	Shiprocket API	v1 external	Courier selection, AWB, pickup, labels, tracking webhooks
AI	Anthropic Claude Haiku 5.5 via `@anthropic-ai/sdk`	SDK 0.133	The cheapest current Claude model, good enough for a skincare guide
Error reporting	Structured JSON logs; Sentry when configured	Sentry 10	Works with no vendor; redacts personal data first
Tests	Vitest with PGlite (Postgres compiled to WebAssembly)	Vitest 4.1	Integration tests run real SQL from the real migrations
Images	sharp, Next.js image optimiser	—	Re-encodes uploads (strips location data), serves AVIF and WebP

Outside services and what happens if each is down

Service	Used for	If it is unavailable
Neon Postgres	Everything durable	Public pages still render from cache, showing list prices and "out of stock"; checkout, sign-in and staff actions refuse with a clear message
Cashfree	Online payments	Online payment is refused; cash on delivery still works; reconciliation settles any payment whose confirmation was missed
Shiprocket	Courier booking and tracking	"Book courier" shows the error and the order stays packed; staff can move orders by hand; tracking catches up when updates resume
Gmail or Resend	Order emails and sign-in codes	Email-code sign-in is hidden; failed order emails are retried up to five times; owner two-step sign-in can be switched off
Meta WhatsApp	Owner order alerts and morning summary	Alerts are skipped and logged; email still goes
Anthropic	AI skin chat	The chat says it is unavailable; nothing else is affected
Google OAuth	Google sign-in	Email, password and phone sign-in still work
Open-Meteo	UV and air-quality notes on the account page	The note is simply not shown
Upstash Redis (optional)	Shared cache	Reads go to Postgres: slower, still correct

5. How the code is organised

The code is in `src/`. A map produced by the graphify tool on 12 October 2026 counted 453 files (about 268,000 words), grouped into 105 clusters of closely related code. The main folders:

Folder	What lives there
`src/app/`	Every page and API route. The folder path is the web address: `src/app/admin/pricing/page.tsx` is `/admin/pricing`.
`src/components/`	Shared interface pieces: the design system (`nv/`), the bag drawer, admin widgets, the assistant panel
`src/lib/`	Core business logic: orders, money, stock, payments, notifications, rate limits, sign-in, observability
`src/modules/`	Larger self-contained areas, each with its own tests: catalogue, payments, shipping, inventory, knowledge, personalisation, CMS, scans, skin chat, reviews, support, newsletter, audit
`src/infrastructure/`	Plumbing: tiered cache, idempotency, job queue, object storage, event streaming, request ids
`src/db/`	The database schema (`schema.ts`) and connection set-up, including the read-replica router
`src/data/`	The catalogue file and other reviewed data: products, directions, formulations, knowledge records
`drizzle/`	Database migrations, applied in order
`scripts/`	Command-line tools: migrations, stock seeding, environment checks, load test, acceptance runs
`docs/`	Design notes, audits, handoffs and runbooks

Modules

Module	Responsibility
`catalog`	Product and SKU identities, display prices and stock for the storefront, catalogue integrity checks
`payments`	The payment state machine, provider event recording, webhook policy
`shipping`	Shiprocket booking, tracking updates, cash-on-delivery collection
`inventory`	Stock commands (set, adjust) and the morning autopilot
`knowledge`	Reviewed skincare knowledge: records, compiler, immutable releases, rollback
`ingredients`	Ingredient dictionary, alias resolution, formulations, interaction rules
`personalization`	Bayesian concern inference, product selection, the weekly planner, routine contracts
`personal`	Saved routines, consent records, guest ownership
`assistant`	Ask Avyora, the on-device question answerer
`skin-chat`	The AI skin chat: prompt, limits, urgent-symptom detection, cost

Module	Responsibility
`cms`	Content documents, revisions, publish checks, media
`scans`	Optional face-photo sessions (switched off)
`ai`	The skin image analysis lifecycle (switched off, no model)
`reviews`, `support`, `newsletter`, `wishlist`, `search`, `recommendations`, `analytics`, `audit`, `operations`	As named

Naming conventions worth knowing

- **Paise everywhere.** Any variable ending `Paise` is whole paise. Rupees appear only at the edges: forms and outside APIs.
- **`skuKey`** is `productId::size`, the legacy way to name a SKU. **`variantId`** is the stable SKU id such as `retinol-90ml`.
- **`.integration.test.ts`** files run against a real Postgres engine built from the migrations.
- **`ponytail:`** comments mark deliberate simplifications with a known limit and the upgrade path.

6. Data and storage

PostgreSQL is the system of record. Everything else (the optional Redis cache, the event stream, the analytics export, uploaded media) is derived from it or sits in front of it, and can be lost without losing a customer's order.

Store	Holds	Authoritative?	If it disappears
PostgreSQL (Neon, Singapore, version 18)	Orders, payments, stock, prices, carts, accounts, content, events, jobs, audit, rate limits	Yes	Restore from backup (below)
Redis (Upstash, optional)	Cached copies of display data, carts and content	No	Reads go to Postgres: slower, still correct
Object storage, media (optional)	Images uploaded through the CMS	Yes, for the image bytes	Articles lose hero images; the catalogue is unaffected
Object storage, analytics (optional)	A copy of the event log	No	Re-export from the event log
Kafka (optional)	A stream copy of events	No	Replay from the event log

The full list of 51 tables and 428 columns is in Appendix A.

The data, grouped

Group	Tables	What they hold
Customers and sign-in	users, accounts, sessions, verification_tokens, otp_codes, addresses	Accounts, sign-in sessions and codes, delivery addresses
Catalogue and pricing	catalog_products, catalog_variants, product_pricing	Product and SKU identities; owner prices and offers, versioned

Group	Tables	What they hold
Stock	inventory, restock_requests	Units on hand per SKU; requests for more
Shopping	carts, cart_items, wishlist_items	Server copies of bags and wishlists
Orders and money	orders, order_items, payment_events, idempotency_keys	Every order, its lines, every payment event, safe retries
Personal records	consent_records, skin_profiles, routine_results, routine_schedule_slots, routine_feedback, scan_sessions, scan_admissions, scan_attempts	Saved routines and photos, only with permission and always with an expiry
Knowledge	ingredients, ingredient_aliases, ingredient_interactions, formulations, formulation_ingredients, usage_profiles, evidence_sources, kb_releases, kb_active_release	Reviewed skincare knowledge and its immutable releases
Content	cms_documents, cms_revisions, media_assets	Site copy, articles, media, every revision
Operations	domain_events, event_deliveries, consumer_registrations, jobs, audit_logs, rate_limits, export_watermarks, activity_events	The event log, job queue, audit trail, rate-limit counters, analytics
Customer contact	reviews, support_requests, newsletter_subscribers	Moderated reviews, contact messages, newsletter sign-ups
Other	environmental_cache, routine_items	Weather readings cache; legacy routine lines

How an order is stored

An order is one row in `orders` and one row per line in `order_items`.

- **Totals** (`subtotal`, `discount`, `shipping`, `tax`, `total`) are whole paise. GST at 18% is already included in catalogue prices; `tax` records the GST portion of the total.
- **The delivery address is frozen** onto the order as JSON (`shipping_address`), so a customer editing their address book later cannot change where a past order went.
- **Each line freezes the product name, size and unit price** at the time of sale, plus a `pricing_snapshot` explaining why it cost that (catalogue price, owner override, or offer, with the pricing version).
- **Two statuses** move independently: `status` (pending, paid, fulfilled, shipped, out_for_delivery, delivered, cancelled, refunded, returned) and `payment_status` (unpaid, pending, authorized, paid, failed, refunded).
- **Safety fields:** `idempotency_key` (unique, makes a retried checkout return the same order), `stock_restored_at` (stock can be returned exactly once), `fraud_status` and `fraud_score` (the cash-on-delivery risk check), and `attention_reason` (set when the owner must decide something).
- **Courier fields:** `awb` (the tracking number, unique) and `shipment` (courier name, label link, latest courier status), added on 21 October 2026 by migration 0022.

Reading and writing

Every read declares how fresh it must be. The primary database serves all writes and every read that must be current: checkout, stock, payments, carts, editors, and every cache refill. Sales reports may read from an optional read replica when one exists and is less than 30 seconds behind. The read router (`src/db/read-router.ts`) checks the replica's lag at most every 10 seconds, falls back to the primary if it is lagging or

unreachable, and never retries a failed SQL statement on the primary, since that could hide a write sent to a read-only copy.

> **Note.** No read replica exists today. Every read goes to the primary. Adding one means creating a read-only compute on Neon and setting `DATABASE_READ_URL`.

Connections

Each server instance keeps a small connection pool (5 connections in production) through Neon's pooler, which multiplexes thousands of short-lived serverless clients onto a few database connections. Migrations and backups use a direct connection, because they need session-level behaviour the pooler does not keep. The practical ceiling is the database's compute size (0.25 compute units on the free plan), not the number of connections.

Migrations

The schema is defined in `src/db/schema.ts` and created by numbered SQL migrations in `drizzle/` (0000 to 0022, listed in Appendix D). Rules the project follows:

- **Additive first.** New tables and columns are added without breaking the code already running; destructive changes ship with a data check.
- **Tests use the real migrations.** Test databases are built by running the same migration files, so a test cannot pass against a schema production does not have.
- **Production is migrated by hand, after approval:** `npm run db:migrate` with the production connection string, ideally rehearsed on a Neon branch (a copy of production) first.

Backups and recovery

Setting	Value	Meaning
Neon point-in-time history	6 hours (free plan)	Any moment in the last 6 hours can be restored to a new branch
Nightly logical backup	Written, not yet running	`.github/workflows/backup.yml`: `pg_dump`, a readability check, AES-256 encryption, upload to private storage. It waits for six `BACKUP_*` secrets in the repository.

Scenario	What you get back	How long	How
Mistake noticed within 6 hours	Everything up to seconds before the mistake	About 15 minutes	Neon point-in-time restore to a new branch, check it, then copy rows back or switch to it
Mistake noticed later	The last nightly dump (once backups run)	About 1 hour	Restore the dump into a new branch (Chapter 31)
Neon project lost	The last nightly dump	A few hours	Restore into any Postgres 18, repoint `DATABASE_URL`
Redis or analytics copies lost	Nothing lost	None	They refill from Postgres

> **Decision needed.** Until the nightly backup runs, the only safety net is 6 hours of history. Setting the backup secrets (Chapter 33) is one of the highest-value tasks before launch.

7. Events and background work

Slow or unreliable work, such as sending an email, calling WhatsApp, refreshing cached pages or scoring a cash-on-delivery order, never runs inside the customer's request. It is recorded as an event and done afterwards. This keeps checkout fast, and means a failing email provider cannot fail an order.

The event log (transactional outbox)

When something important happens, a row is written to `domain_events` in the **same transaction** as the change itself. If the order commits, its event exists; if it rolls back, the event never existed. Events carry only an id and an order number; consumers re-read the current state when they act.

Event	Written when
`order.placed`	An order is created (any payment method)
`order.paid`	A payment is confirmed (online, or cash collected on delivery)
`order.payment_failed`	A payment attempt fails
`order.refunded`	A refund is recorded
`order.cancelled`	An order is cancelled
`order.shipped`	The courier picks the parcel up
`order.needs_attention`	The owner must decide something (late payment, lost parcel, return needing a refund)
`inventory.stock_out`, `inventory.changed`	Stock reaches zero or changes
`pricing.changed`	An owner edits a price or offer
`content.published`, `content.unpublished`	CMS content changes

Consumers

Each consumer reads the log independently and records its own progress in `event_deliveries`, so one slow consumer cannot hold up another.

Consumer	Acts on	What it does
`notifications`	`order.placed` (cash on delivery), `order.paid` (online), `order.shipped`	Customer confirmation email, owner email and WhatsApp alert; the "on its way" email with tracking link
`cod-risk`	`order.placed` for cash on delivery	Scores the order and holds risky ones for the owner (Chapter 21)
`revalidate`	Stock, price and content events	Refreshes the cached pages that show the changed data
`stream-relay` (optional)	Everything	Copies events to Kafka when one is configured

> **Why online orders are announced only when paid.** Until 12 October 2026, an online order was announced the moment the customer pressed Pay. A customer who backed out of the payment screen still received "order confirmed", and the owner got an alert for a sale that never happened. Now online orders are announced on `order.paid`, and cash-on-delivery orders on `order.placed`.

When events are processed

- **Straight away.** After checkout, a payment confirmation or a courier update, the drain is scheduled with Next.js `after()`, which runs once the response has been sent.
- **Daily backstop.** `/api/cron/events` runs every day at 09:15 India time and processes anything still pending.
- **Retries.** A failed consumer retries with growing waits (30 seconds, 1, 2, 4 then 8 minutes), up to 5 attempts. After that the delivery is marked failed with its error and shown in the owner console under System, where it can be replayed.

The job queue

Work that is not a reaction to an event (for example a future photo-analysis job) uses the `jobs` table, a Postgres queue with leases. Workers claim jobs with `FOR UPDATE SKIP LOCKED`, so two workers never take the same job; each claim has a 60-second lease; completion is fenced so a worker whose lease expired cannot overwrite a newer result; failures back off and finally become dead letters that the owner can replay.

Scheduled jobs

Vercel Cron calls two protected addresses. Both refuse any caller without the `CRON_SECRET` bearer token.

Job	Schedule (UTC / India)	What it does
`/api/cron/sweep`	03:15 / 08:45 daily	Cancels online orders abandoned at payment (asking Cashfree first) and returns their stock; prunes old idempotency keys and rate-limit counters; deletes expired routines, saved answers, scan photos and unconfirmed newsletter sign-ups; runs the stock autopilot and sends the morning summary
`/api/cron/events`	03:45 / 09:15 daily	The event drain backstop

> **Plan limit.** Vercel's free plan allows only daily cron jobs. On the Pro plan, change them in `vercel.json` to every 15 minutes (`*/15 * * * *`) for the sweep and every 6 hours for the events backstop, so abandoned stock is released sooner.

8. Caching and rendering

Most visitors only read. The system is built so that reading costs almost nothing: public pages are pre-built HTML served from Vercel's edge, and the database is touched only when something changes.

How each kind of page is served

Pages	How	Refresh
Home, collections, 27 product pages	Pre-built (incremental static regeneration)	Every 60 seconds, and immediately when stock, price or copy changes
Journal and articles	Pre-built	Every 5 minutes, and on publish
Policies, contact, routine finder, wishlist	Fully static	On deploy
Sitemap	Pre-built	Hourly, and on article publish
Account, checkout, orders, sign-in, staff consoles, private APIs	Built per request	Never cached: `private, no-store`, varied by cookie

The middleware stamps `private, no-store` on every personal page and API regardless of how it was rendered, so a configuration mistake cannot make one customer's page cacheable for another. `scripts/measure-pages.mjs` checks this and fails if any private address becomes cacheable.

The tiered cache

Display data (prices and stock on product cards, CMS content, ingredient data, recommendations) goes through a two-level cache:

Level	Where	Purpose
L1	Memory of each server instance	Seconds-long copy; removes repeated reads within one instance
L2	Redis (optional)	Shared between instances; minutes to hours
Source	Postgres	Always correct

Data	L1	L2	Notes
Display prices	5 s	60 s	Checkout never uses these; it reads Postgres fresh
Stock for display	5 s	30 s	A sold-out badge a few seconds stale is tolerable; overselling is impossible because reservation happens in the database
CMS content	60 s	1 hour	Publishing invalidates it at once
Ingredients	5 min	1 day	Changes only with a knowledge release
Carts and wishlists	none	30 min	
Recommendations	5 min	1 hour	
Weather conditions	5 min	30 min	

Every change that affects a cached value invalidates it explicitly; the times above are only a backstop.

When the database is down

Since 21 October 2026, the storefront survives a database outage. If the stock or price read fails, product pages still render with the catalogue's list prices and every item marked "Out of stock" (nothing can be

ordered during an outage anyway). The fallback is reported to the error log and **not cached**, so real figures return as soon as the database answers. Checkout deliberately does not fall back: it fails loudly rather than charge a price that might be wrong.

Page weight

Bundle	Size (first-load JavaScript)
Shared by every page	103 KB
Home	139 KB
Product page	142 KB
Routine finder	167 KB (includes the routine engine, so routines compute on the device)

Sentry's error-reporting code is loaded only when a reporting address is configured, and then after the page, which cut the shared bundle from 195 KB to 103 KB.

9. Search engines, accessibility and design

Search engines

The shop is built so search engines see real content, fast:

Measure	Detail
Pre-built HTML	Home, collections, all 27 product pages and the journal are pre-built, so crawlers get full content without running scripts. The collections page's HTML contains the whole product grid
Sitemap	`/sitemap.xml`, rebuilt hourly and whenever an article is published
Robots rules	`/robots.txt` allows everything public and disallows `/admin`, `/api`, `/checkout`, `/orders`, `/login` and `/signup`, and points to the sitemap
Canonical addresses	Each public page declares its own canonical URL. (An earlier version declared the home page as canonical for every page, which told search engines to ignore the rest; fixed in the October re-audit)
Product structured data	Each product page carries schema.org Product data: name, description, image, brand, offers with the current price and availability. A rating appears only when published, verified-purchase reviews exist
Private pages	Never indexed: account, checkout, orders, staff consoles and newsletter pages are marked no-index, and never cached
Speed	Shared JavaScript 103 KB; layout shift 0; HTML 7 to 20 KB compressed
Site address	Everything is built from `NEXT_PUBLIC_SITE_URL`, so moving to the shop's own domain is one setting (plus the redirects in Chapter 29)

Bots are handled with care for search: known search engines are checked **before** any blocking rule, so no change to bot handling can accidentally de-index the shop. Scrapers with obvious tool names (`python-requests`, `scrapy`, `wget`, `sqlmap` and similar) are refused, and page views are limited to 120 a minute per address at the edge.

Accessibility

Accessibility was checked with automated passes at 1280, 1440 and 1920 pixels wide and by keyboard. The last pass found no issues.

Area	What is in place
Structure	One main heading per page; headings in order (the collections and product pages gained hidden section headings to avoid skipping levels); page language set
Keyboard	A "skip to content" link; every control reachable in a sensible order; a visible 2-pixel focus outline for keyboard users only; the menu traps focus while open and returns it to the menu button on Escape
Forms	Every field has a label; errors are linked to their field (`aria-invalid`, `aria-describedby`) and announced (`role="alert"`)
Images	Every image has alternative text; a failed image keeps its space and shows the text
Motion	Animations are skipped under "reduce motion"; content never stays hidden waiting for an animation
Dialogs	The bag drawer and dialogs trap focus, close on Escape, and hide the background from screen readers
Live updates	The routine finder marks recalculation as busy; the chat reads replies as they arrive (`aria-live`)
Zoom	At 200% zoom the header controls fit and stay reachable; some desktop grids need horizontal scrolling until the narrow layouts are built

> **Not yet done.** Mobile and tablet layouts were deferred by the owner until the desktop design is approved. At narrow widths the site works but is not yet designed for them.

The design system

The current design (switched on with `NEXT_PUBLIC_REDESIGN=1`) was measured from a reference site at three widths and rebuilt to within a few pixels: section heights at 1440 pixels match the reference to 0 to 4 pixels over an 11,400-pixel page.

Token	Value
Page background	#FAFAFA
Text	#1A1A1A
Muted text	#696666
Faint text	#ADADAD
Cards	#FFFFFF
Lines	#E8E8E8
Layout	1240-pixel centred column; 40-pixel side gutters; 8-pixel grid gaps
Corners	18 px cards, 12 px inner panels, fully rounded buttons
Header	76 px
Type	Inter (400 and 500) for text and headings; Instrument Serif for the wordmark. Both self-hosted through `next/font`, with metric-matched fallbacks so text does not jump when the font loads
Type scale	Hero 100/100 px, section headings 80/88, card titles 32, body 16 to 20

The building blocks live in `src/components/nv/`: containers, sections, grids, headings, buttons (dark, light, outline, ghost), cards, status messages, spinners, skeletons, form fields, an accordion, a dialog and an image component that never collapses. A review page at `/design-system` (not linked, not indexed) shows each one in every state.

Photography

No approved photography exists yet. The product photos are stock images, and one of them (used for the Centella Cleansing Balm, the Ceramide Cream and the Water-Gel Sorbet Moisturizer) shows another brand's label. It is kept off the home page but still appears on those product pages. Replacing it is a launch blocker. The image slots and their sizes are listed in `docs/product-onboarding.md`.

PART III

Customer features

Everything a shopper touches: the catalogue, the bag and checkout, payments, orders and tracking, accounts, the routine finder, Ask Avyora and the AI skin chat.

10. Customer journeys

Before the feature-by-feature chapters, this chapter follows real customers through the shop, so the pieces can be seen working together. Each journey says what the customer sees and what the system does behind it.

Journey 1. A first visit that ends in a cash-on-delivery order

Who. Meera, 27, oily skin with occasional breakouts. She has never bought from Avyora and does not want to create an account.

1. **She lands on the home page** from a search result. The page arrives as pre-built HTML from Vercel's edge in a few tens of milliseconds; no server or database work happens for her visit. She sees the hero ("Skincare built around a simple routine"), four best sellers with current prices, and the three essentials at their lowest prices.
2. **She opens Collections** and filters by "Acne Control". Filtering happens in her browser; the product grid was already in the page.
3. **She opens the LHA sebum-control product.** She picks a size and quantity 2. The quantity selector stops at the number in stock, or 10, whichever is lower. Directions show "being reviewed", because none are approved yet; nothing generic is invented.
4. **She adds it to the bag.** The bag asks the server for a current price quote for that one SKU and shows the line priced for that size, times two. Below ₹1,199 it shows "Add ₹… for free delivery".
5. **She goes to checkout.** The page reads prices and stock fresh from the database. She enters her email and address. Because SMS is configured, she is sent a six-digit code to prove her mobile number. She chooses **Cash on delivery**.

6. **She presses Place order.** In one transaction the server prices her bag, reserves two units, saves her address, order and lines, and writes an `order.placed` event. Her page moves straight to the order confirmation.
7. **Straight after,** the risk check scores her order (no warning signs: approved), she receives a confirmation email with a private link to her order, and the owner gets an email and a WhatsApp line: "New order AVY-7KQ2MX — ₹798 (COD). 2 items to Pune."
8. **In the stockroom,** her order appears at the bottom of the dispatch queue (oldest first). The manager packs it, presses **Mark packed**, then **Book courier**; Shiprocket assigns Delhivery and a tracking number. The label is printed.
9. **When the courier picks it up,** Shiprocket's update moves her order to "On its way", and she receives an email with the tracking link and "Cash on delivery: please keep ₹798 ready."
10. **On delivery,** the courier's update marks the order delivered and records the ₹798 as paid. It now counts in the owner's revenue.

Journey 2. Paying online, then changing her mind

Who. Ritu, a returning customer, signed in with Google.

1. Her bag, saved on the server, follows her from her phone to her laptop when she signs in.
2. At checkout she chooses **Pay online** and presses Pay. The server creates the order (reserving stock) and a Cashfree payment session, and the browser remembers the attempt.
3. On Cashfree's page she has second thoughts and presses Back.
4. Back on checkout, the page finds the remembered attempt and asks the server to cancel it. The server asks Cashfree to terminate the payment session; Cashfree confirms it is closed; the order is cancelled and its stock returned.
5. Ritu sees: "Nothing was ordered and you have not been charged." The Pay button works again, with a fresh attempt.
6. **Nobody was told about an order that did not happen:** no confirmation email, no owner alert. Online orders are announced only when paid.
7. Later she pays for real. Cashfree's signed webhook confirms the payment; the amount matches the order total exactly; the order becomes paid; now the confirmation and the owner alert go out.

Journey 3. Building a routine

Who. Arjun, 34, sensitive skin, a budget of ₹1,500, already owns a cleanser he likes.

1. He opens the **routine finder** and answers the questions. On pregnancy he can skip; on reactivity he chooses "very reactive"; under owned products he adds his cleanser.
2. The plan is computed **on his device**, instantly, by the same engine the server uses.
3. Because he is very reactive, he gets gentle essentials only: his own cleanser, a moisturiser and a sunscreen. No actives, no exfoliation. "Not included yet" lists the treatments left out, each with its reason.
4. The week is shown as a real seven-day table, morning and evening. The total of what he would need to buy is within ₹1,500.

5. He chooses **Allow saving and save**. That records his permission (routine saving, policy version `routine-saving-v1`) and saves the routine to his account for 180 days. The server recomputes it from his answers; the browser's result is never trusted.
6. A week later, his account page asks "How is week 1 going?" with three short choices.

Journey 4. A question at midnight

Who. Sana, signed in, worried about a new rash.

1. She opens **Talk about your skin** and reads the notice: an AI, not a doctor; messages go to Anthropic to get a reply; nothing is saved.
2. She writes that her lips are swollen after a new serum. Before the AI answers, the server's own check spots "swollen … lips" and sends: "Please see a doctor or dermatologist soon about swelling of the face, lips, eyes or throat. If it is severe or getting worse quickly, or you have trouble breathing, go to emergency care now."
3. The AI's reply follows, without product suggestions, as its instructions require for urgent signs.
4. The next day she asks about oily T-zone and dry cheeks. The AI suggests the matching Avyora gel moisturiser and explains which concern it is meant for. Each reply costs a few paise from her ₹0.50 daily budget.

Journey 5. Tracking without an account

Who. A guest who lost the confirmation email.

1. She opens **Track order** and enters her order number and email.
2. If both match, she sees the status in plain words and a coarse location. If either is wrong, the answer is the same "we could not find that order", so the page cannot be used to discover orders.
3. Twenty tries a minute is the most anyone gets.

Journey 6. A parcel that comes back

1. A COD parcel is refused at the door. The courier marks it undelivered: the order keeps "On its way" and the owner is alerted: "The courier could not deliver this parcel. Call the customer before it goes back."
2. The owner calls; no answer. The parcel returns.
3. When Shiprocket reports it delivered back to the seller, the order becomes **returned** and the stock goes back on the shelf, exactly once.
4. The next time this customer orders cash on delivery, the risk check sees an earlier COD order that came back, and weighs it.

11. Catalogue and storefront

The catalogue

The range is 27 products in 30 sizes (SKUs), defined in `src/data/mock-data.ts` and mirrored in the database tables `catalog_products` and `catalog_variants`. Every size has a stable id built from the product and size, for example `retinol-90ml`, so "30ml", "30 ml" and "30 ML" can never become three different SKUs.

What lives where:

Information	Source of truth	Edited by
Product identity, name, category, sizes, list price	The catalogue file	A developer, through a reviewed change
Prices and offers	`product_pricing` (overrides the list price)	The owner, in Pricing
Stock	`inventory`	The manager (and the owner, with a double confirmation)
Product copy and directions	CMS documents, then approved directions	The owner, in Content
Ingredients, formulations, cautions	Knowledge records and releases	Reviewed data, published by the owner

Catalogue mode

`NEXT_PUBLIC_CATALOGUE_MODE` decides what the shop is allowed to sell:

Mode	Behaviour
`sample` (default)	Products are labelled samples, a notice shows on shopping pages, and a production build refuses orders with "This is a preview shop: the products shown are samples and cannot be ordered yet."
`verified`	Only onboarded products with verified records are shown and sold

`ALLOW_SAMPLE_ORDERS=1` lets a staging deployment take sample orders for sandbox testing. It must never be set on the real shop.

Shop pages

- **Home** (`/`): the hero ("Skincare built around a simple routine"), best sellers with size choice and quick add, what Avyora does, the three essentials at their lowest current prices, an FAQ and a support form.
- **Collections** (`/collections`): every product with filters by category and concern, sort by price, and search (`?q=`). The server-rendered page contains the full grid so search engines and first paint see real products; filters apply in the browser.
- **Product pages** (`/products/[slug]`): sizes with their own prices and stock, quantity (capped at what is available and at 10 per SKU), ingredients, directions (only approved ones; otherwise "being reviewed"), verified-purchase reviews, related products that pass the ingredient-conflict check, and product structured data for search engines.
- **Compare** (`/compare`): side by side, with unit prices and which fields are verified.
- **Wishlist** (`/wishlist`): saved in the browser for guests, in the database for signed-in customers.

Stock labels

Stock	Label
Above the SKU's low-stock level	In stock
At or below it	Only a few left

Stock	Label
Zero, or never counted	Out of stock (cannot be added)
Backorder allowed	Available

A SKU nobody has counted cannot be sold. "We have never counted this" and "we have none" are the same thing from the customer's side, so a fresh deployment sells nothing until stock is entered (`npm run db:seed-inventory <n>` or the stockroom screen).

Honest content rules

`src/lib/content-claims.ts` holds rules that run in the test suite and when CMS content is published. They block:

- offers the checkout does not apply (for example "Buy 2, Get 3rd Free", which was removed for that reason);
- absolute claims such as "zero irritation", "cures", "100% safe", "anti-wrinkle" or "shrinks pores";
- placeholder text and dummy phone numbers;
- delivery figures written anywhere except `src/lib/money.ts`.

`npm run check:launch` lists every remaining launch blocker (unconfirmed business details, missing directions) and exits with an error until there are none.

12. Bag and checkout

The bag

A bag line is `{productId, size, quantity}` and never a copy of the product or its price. The bag asks the server for current prices and stock (`GET /api/catalog/availability`, up to 50 SKUs at once, briefly cached) and shows "Checking current prices…" until they arrive. A price the server has not confirmed is marked as such.

Rule	Value
Most per SKU	10 (and no more than counted stock)
Most lines	50
Free delivery	At ₹1,199 or more
Delivery charge	₹79 below ₹1,199
GST	Included in every price (18%)

For signed-in customers the bag and wishlist are mirrored to the server. On sign-in a guest bag is merged with the account's: per SKU the larger quantity wins (never the sum, so repeated sign-ins cannot inflate it), capped at the purchase limit and stock, with any change explained in the bag. On sign-out the account's bag is cleared from the browser. A bag save for a different account than the one signed in is refused, so one person's bag can never be written into another's.

Checkout

`/checkout` reads prices and stock fresh from the database (never from a cache), because this is the moment the customer agrees to a total.

1. **Details.** Email, delivery address (with an Indian PIN code and mobile number check) and payment method: Cashfree (online) or cash on delivery.
2. **Phone check for guests.** When an SMS provider is configured, a guest proves the delivery mobile number with a six-digit code. The proof is signed, lasts 8 hours and is bound to that number; the server refuses a guest order without it. Signed-in customers are not asked.
3. **Price check.** Each line carries the price the customer was shown. If an offer ended or the owner changed a price in the meantime, the order is refused with "The price of … changed from ₹949 to ₹999" before anything is reserved, and the page shows the new total.
4. **Stock check.** Reservation happens in the database with an update that only succeeds if enough units remain. Twenty simultaneous attempts on ten units grant exactly ten.
5. **Idempotency.** Every checkout carries a random key. Pressing Place order twice, a network retry or a reload all return the same order. Since 21 October 2026 a replay is only answered for the **same email**, so a key alone can never reveal another customer's order.

After payment is abandoned

Online orders reserve stock when the customer presses Pay. If they back out:

- On returning to the shop (Back button, reload, or the return page), checkout asks Cashfree to terminate the payment session. Only when Cashfree confirms it is closed (`TERMINATED` or `EXPIRED`) is the order cancelled and its stock returned. If a payment succeeded in the meantime, the payment is applied instead.
- The page says "nothing was ordered and you have not been charged", resets the Pay button and uses a new idempotency key, so the next attempt is a fresh order.
- Anything not closed this way is caught by the morning sweep, which also asks the provider before cancelling.

13. Payments

Methods

Method	Provider	Status
UPI, cards, net banking, wallets	Cashfree Payment Gateway	Built and tested in Cashfree's sandbox; live keys need KYC
Cash on delivery	—	Live; subject to the risk check
Razorpay	Razorpay	Replaced by Cashfree on 9 October 2026; code kept only to settle older Razorpay orders

Three rules that protect the money

1. **The amount is computed on our server.** The browser sends product ids, sizes and quantities only. Cashfree is asked to collect exactly the order total the server calculated.
2. **The browser's word is never evidence.** When the customer returns from Cashfree, `/api/payments/cashfree/verify` asks Cashfree for the order's payments rather than believing the browser.
3. **The webhook is authoritative.** Cashfree also posts each payment result to `/api/webhooks/cashfree`, signed with the secret key over the exact raw bytes. A closed tab cannot lose a payment, because the webhook still arrives; and the daily reconciliation asks Cashfree about any order still in doubt.

The payment state machine

All three paths (return page, webhook, reconciliation) feed the same decision function, `decide()` in `src/modules/payments/state-machine.ts`, applied with the order row locked (`SELECT … FOR UPDATE`), so a webhook and the browser arriving within milliseconds are applied one after the other.

Current state	Signal	Result
unpaid, pending, failed	Captured, amount equals the order total	paid (stock re-reserved if it had been released)
any	Captured, amount differs from the total	Rejected and recorded; order unchanged
paid	Captured again	Ignored as a duplicate
refunded	Captured	Rejected
paid or refunded	Failed	Ignored: a failed retry after a success changes nothing
unpaid, pending	Failed	failed, stock released

Two cases need a person, and set `attention_reason` on the order with an `order.needs_attention` event:

- **Paid after the stock was sold to someone else.** The money is real and must be honoured, but the goods are gone: refund or fulfil from a backorder.
- **Paid after the order was cancelled.** The stock is re-reserved; confirm with the customer before sending.

Recording provider events exactly once

Every webhook delivery is written to `payment_events` keyed by the provider's own event id, in the same transaction as its effect. A duplicate delivery conflicts on that key and changes nothing. An event for an order that does not exist yet (the webhook outran our own checkout commit) is answered with a retryable 503 and not recorded, so Cashfree's retry is processed properly.

Cash on delivery

A cash-on-delivery order is real when placed, so it is announced straight away and stays `unpaid` until delivered. Since 21 October 2026, when it is delivered (by the courier's update or the manager's button) the payment is recorded as **paid** through the same state machine. Before this, COD sales never appeared in revenue.

Every COD order passes the risk check (Chapter 21) before it can be packed.

Setting up Cashfree for live payments

1. Complete Cashfree KYC and activation. Cashfree reviews the policy pages, which still have `[TO CONFIRM]` markers to fill in first.
2. Set `CASHFREE_APP_ID`, `CASHFREE_SECRET_KEY` and `CASHFREE_ENV=production` in Vercel.
3. Register the webhook `https://YOUR-DOMAIN/api/webhooks/cashfree` in the Cashfree dashboard.
4. Place one small real order and refund it, to check the whole path.

> **Note.** In the sandbox, Cashfree cannot reach `localhost`, so local tests rely on the return-page verify plus reconciliation. Test the real webhook on a staging address in a normal browser.

14. Orders, tracking and notifications

What the customer sees

The order page shows five steps, in words customers use rather than internal names:

Internal status	Customer sees
pending, paid	Order placed
fulfilled	Being prepared
shipped	On its way
out_for_delivery	Out for delivery
delivered	Delivered

Once a courier is booked the page also shows the courier name and the tracking number, linked to Shiprocket's public tracking page.

Who can see an order

`/orders/AVY-XXXXXX` shows a name, address and phone number, so access is strict: the signed-in account that placed it, or a signed token bound to that one order number (carried in the link in the confirmation email). A token for one order cannot be replayed against another. `/track-order` needs both the order number and the email it was placed with, gives the same answer whether the order is missing or the email is wrong, and is rate limited (20 a minute) so it cannot be used to guess.

Messages sent

When	To	Channel	Content
COD order placed, or online order paid	Customer	Email	Items, total, payment, address, private order link
Same	Owner	Email (`OWNER_EMAIL`)	Order summary and payment state
Same	Owner	WhatsApp	One line: order number, total, paid or COD, item count, city

When	To	Channel	Content
Parcel picked up by the courier	Customer	Email	Courier, tracking number and link, amount to keep ready if COD
Every morning	Owner	WhatsApp and email	Last 24 hours of orders and sales, what is low, restock requests raised
Owner signs in	Owner	Email	Six-digit sign-in code

A customer email the provider refused is retried (up to 5 times); a failed owner copy is only logged.

> **Known issue.** On 12 October 2026 Meta rejected the WhatsApp alert with "API access blocked". The WhatsApp access token or app permission needs renewing in Meta's developer console; email alerts are unaffected.

15. What customers and staff are told

The exact wording the system uses, taken from the code, so the owner knows what customers receive and can ask for changes. Text in angle brackets is filled in per order.

Order confirmation (customer email)

Sent when a cash-on-delivery order is placed, or an online order is paid. Subject: **Your Avyora order <order number>**.

```
Thank you, <name>.

Order <order number>
  2 x <product> (<size>)

Total: ₹<total>
<Paid online. | Cash on delivery — please keep the amount ready. | Payment pending.>

Delivering to:
<address line>
<city state PIN>
<phone>

Track your order here:
<private order link>

We will email again when it is on its way.
```

The private link carries a signed token for that one order, so a guest who closes the tab can still find it.

Shipped (customer email)

Sent when the courier picks the parcel up. Subject: **Your Avyora order <order number> has shipped**.

```
Good news, <name>: your order <order number> is on its way.

Courier: <courier>
```

```
Tracking number: <AWB>
Follow it here: https://shiprocket.co/tracking/<AWB>

<Cash on delivery: please keep ₹<total> ready. | Already paid. Nothing to pay at the door.>

Your order: <private order link>
```

New order (owner email and WhatsApp)

Email subject: **New order <order number> — ₹<total>**. The body lists the items, total, payment (paid, cash on delivery, or NOT PAID), the customer's name and email, and the delivery address.

WhatsApp, kept to one line for reading on a phone:

```
New order AVY-7KQ2MX — ₹798 (COD). 2 items to Pune.
```

Morning summary (owner)

WhatsApp:

```
Avyora, last 24 hours: 3 orders, ₹2,847. Running low: 2 items. 2 new restock requests raised for
you to approve.
```

Email (subject **Avyora morning summary**) adds one line per low item, for example "HA Toner (150ml): 10 left, about 5 days", and ends: "Approve or decline requests in the owner console under Requests." On a quiet stock day it says "Stock: everything is above its reorder level."

Sign-in codes

Customer email code. Subject: **<code> is your Avyora sign-in code**.

```
Your Avyora sign-in code is <code>.

It expires in 10 minutes and can be used once.
If you did not ask to sign in, you can ignore this email — nobody can
use this code without it.
```

Owner's second step. Subject: **Your Avyora sign-in code**.

```
Your owner sign-in code is <code>. It expires in 10 minutes.

If you did not just try to sign in, someone has your password: change it now.
```

On screen: checkout

Situation	Message
Price changed since shown	"The price of <product> (<size>) changed from ₹949 to ₹999. Review your updated total, then place your order again."
Sold out while ordering	"<product> (<size>) has just sold out."

Situation	Message
Not enough left	"Only <n> left of <product> (<size>). Please reduce the quantity."
Size withdrawn	"<product> is no longer sold in <size>. Please choose a size again."
Guest phone not verified	"Verify your mobile number with the code we send before placing the order."
Backed out of payment	"Your payment was not completed, so nothing was ordered and you have not been charged. Your bag is still here: try again, or choose cash on delivery."
Sample catalogue on the live shop	"This is a preview shop: the products shown are samples and cannot be ordered yet."
Too many attempts	"Too many attempts. Please wait <n> seconds and try again."
Safety checks unavailable	"This is temporarily unavailable. Please try again in a moment."

On screen: the skin chat

Situation	Message
Not signed in	"Please sign in to use the skin chat." (the page itself redirects to sign-in)
Chat not configured	"The skin chat is not switched on yet."
Daily budget or rate reached	"You have sent a lot of messages. Please wait a little and try again."
Urgent symptom	"Please see a doctor or dermatologist soon about <reason>. If it is severe or getting worse quickly, or you have trouble breathing, go to emergency care now. Skincare products are not the right fix for this."
Off-topic request the model declines	"I can't help with that one. I'm happy to talk about skin concerns and everyday skincare: tell me what you're noticing."
Provider failure	"Sorry, the skin chat is unavailable right now. Please try again in a little while."

On screen: staff

Where	Message
Dispatch, unpaid online order	"Not paid. Do not send; check with the owner."
Dispatch, COD	"Cash on delivery — collect payment at the door."
Dispatch, held by risk check	"On hold — this cash-on-delivery order failed an automatic check. Confirm with the customer before packing."
Dispatch, attention	"On hold — the owner needs to decide what happens to this order before it is packed."
Book courier, not packed	"Mark the order packed before booking a courier."
Book courier, Shiprocket not set up	"Shiprocket is not connected yet. Ask the owner to add its login."
Book courier, no courier available	"No courier could take this parcel yet: <reason>. Check the PIN code, or retry from the Shiprocket panel."
Pricing, two editors	Refused with a "changed since you opened it" message
CMS publish with a banned claim	"Not published: "<phrase>" (<reason>). Edit the draft and try again."

Changing the wording

Customer emails live in `src/lib/order-notifications.ts` and `src/lib/notify.ts`; staff and checkout messages in the files of the screens that show them. Changing them is a small, safe edit, but the claim rules (Chapter 11) apply to customer-facing text, and delivery figures must come from `src/lib/money.ts`.

16. Accounts and sign-in

Customer sign-in

The sign-in page asks for an email first, then offers only the methods that account can actually use:

Method	Notes
Google	Through Auth.js; the account is created on first sign-in
Email code	Six digits, stored hashed, valid 10 minutes, five wrong guesses burn it. Offered only when email delivery works
Password	PBKDF2-hashed; sign-up and reset by email code
Phone code	Offered only when an SMS provider is configured (DLT registration is required in India)

Customer sessions are stored in the database, so they can be revoked from the server. All sign-in steps are rate limited (10 attempts per 10 minutes per identity, 30 per network address; codes 3 per 15 minutes, at most one a minute).

The account area

Page	Purpose
`/account`	Hub: orders, addresses, routines, data and permissions
`/account/orders`	Order history with links to each order and invoice
`/account/addresses`	Address book: add, edit, delete, choose a default. Each address is checked against its owner on every change
`/account/routines`	Saved routines and their state (current, outdated, expired, permission withdrawn), weekly feedback, delete, withdraw permission
`/account/data`	What is stored, and the permissions granted

A signed-in customer with a saved address may also see a short note when conditions where they live are unusual: very high UV, very dry air or poor air quality (from Open-Meteo, cached per region). It shows nothing on an ordinary day.

Guests who own things

A guest who saves a routine is given a random secret in an `HttpOnly` cookie; only its SHA-256 hash is stored. When they later sign in, everything the guest owned is moved to the account in one transaction.

17. Routine finder and the knowledge base

The routine finder (`/routine-finder`) is Avyora's main point of difference: a short questionnaire that produces a real seven-day plan, morning and evening, from products that suit the customer's skin, budget and what they already own. It is built so that it can never recommend something unsafe because of a scoring quirk.

The questions

- Up to three priorities (for example breakouts, dryness, dark spots).
- Skin type and how reactive it is; whether it is irritated right now.
- Age band, pregnancy and breastfeeding, allergies (with ingredient names), prescribed treatments. Each has "not sure" or "prefer not to say", and each can be skipped.
- Experience with skincare, how regularly they would follow a routine, routine size (3, 4 or 5 steps), budget (₹0 to ₹10,000), and products they already own.

Every unanswered safety question stays **unknown**. Unknown is never treated as "no".

How a routine is computed

The same pure function, `computeRoutine`, runs in the browser (so results appear instantly) and on the server (when a routine is saved). It reads everything clinical from one knowledge release.

1. **Inference.** Ranks the customer's concerns. With no validated statistical parameters (today), it simply uses the customer's own order and shows no percentages.
2. **Safety rules.** Decide the mode (recovery if irritated, gentle if very reactive) and which product classes are excluded.
3. **Selection.** For each product: hard exclusions first (allergy that cannot be ruled out, ingredient conflicts with owned products, retinoids in pregnancy or if unsure, under-18s, prescribed treatments, any unknown safety answer for elective treatments). An excluded product never reaches ranking, so no weighting can bring it back. Then availability (counted stock, a real price), owned products first, ranking, and filling within budget and the step limit.
4. **Planning.** Builds the seven days: essentials every day in their sessions (cleanse morning and evening, moisturise both, sunscreen in the morning); treatments only with approved directions and a stated weekly maximum, spread across the week and never two irritating actives in one session; at most one optional extra per evening.
5. **Honest result.** Complete, partial or no match, with every excluded product and the reason, every unfilled step, and "what we are unsure about".

Situation	What the customer gets
Currently irritated or very reactive	Gentle cleanser, moisturiser, sunscreen only: no actives, no exfoliation
Pregnant, breastfeeding, or prefers not to say	No retinoids or strong actives
Beginner	Essentials only, at most 3 steps a session
Allergy named	Only products whose complete ingredient list proves the allergen absent

Situation	What the customer gets
Tiny budget plus owned products	A routine from what they own; nothing to buy

> **Today's honest limit.** No product has approved usage directions or a complete reviewed ingredient list yet, so every routine is essentials plus clearly optional additions, and treatments appear under "Not included yet: its usage directions are still being reviewed". When reviewed records are added and published, treatments become eligible with no code change.

Saving routines

Saving needs explicit permission ("routine saving" consent), recorded in `consent_records` with the policy version. A saved routine stores the inputs, the result, the engine version and the knowledge release that produced it. Guest routines expire after 30 days, account routines after 180. If the knowledge release later changes, the routine is shown as outdated and can be recalculated from its saved answers; if a release is revoked, the old result is withheld. Withdrawing permission hides saved routines immediately.

The private routine API (`/api/routines`) recomputes on the server: the browser sends inputs only, never a result or a price. Each save needs an idempotency key, so a retry returns the same routine.

The knowledge base

Anything with clinical or safety meaning (decision rules, ingredient cautions, interactions, directions, formulations, explanation templates, education answers) is a **record** that is either a draft or approved, with a reviewer, a date and evidence sources.

- **Compiler.** `npm run kb -- build` compiles the approved records into six JSON artifacts with SHA-256 checksums. The release id is derived from the content, so identical input always gives an identical release. It refuses to compile drafts, approvals without sources, contradictory or unreachable rules, unknown ingredients, or any formulation problem.
- **Releases** (`kb_releases`) are immutable. One is active (`kb_active_release`), with the previous one kept for rollback. The owner publishes, rolls back (with a reason) or revokes (with a reason; never the active one) from `/admin/knowledge`.
- **Today.** The release holds the catalogue and 17 ingredient identities. 42 items await qualified review: 6 safety rules, 6 explanation templates, 5 interaction rules, 17 ingredient cautions and 8 treatment directions.

Optional face scan (switched off)

`/scan` lets a customer take or upload a photo, checks it in the browser (format, size, brightness, face count) and, if hosted scans are enabled, uploads it privately. It is switched off by default (`NEXT_PUBLIC_FACE_SCAN`, `FACE_SCAN_HOSTED`), and **no analysis model exists**, so every path ends in "Analysis unavailable" and the questionnaire. No score is ever invented.

The safeguards are built for when a model is added: separate photo consent; images re-encoded to strip location and camera data; photos deleted after at most 24 hours (a database rule); at most 3 scans a day and 10 a month per person, 100 a day overall; at most 2 paid model calls per scan; withdrawing consent deletes the photo at once.

18. Ask Avyora

Ask Avyora (`/assistant`, and under each routine) answers questions about ingredients, products, a customer's own routine, prices and orders. It uses **no AI model**: answers are looked up from the published knowledge release on the customer's own device, in under a tenth of a millisecond.

Topic	Answer comes from
What is this ingredient?	The release's ingredient records: name, INCI, aliases, class. Cautions only when reviewed
How do I use this product?	Approved directions only; otherwise "not published yet, check the pack"
My routine on Tuesday evening	The customer's own saved schedule
Why was this left out?	The routine's own decision trace
Price or stock	The live price quote
Where is my order?	The existing order lookup (order number plus email)

- **Privacy.** Questions never leave the device: nothing is logged or sent, except the public release, the public price quote and the order lookup.
- **Clarifies rather than guesses.** "Vitamin C" or "toner" get a question back, since several things match.
- **Refuses medical questions.** Diagnoses, medicines and doses are declined with a pointer to a dermatologist.
- **Bounded.** Questions are capped at 200 characters and 24 words.

19. AI skin chat

The skin chat (`/skin-chat`, linked from the home page as "Talk about your skin") is a conversation with an AI skincare guide, added on 12 October 2026. Customers describe what they are noticing (breakouts, dryness, sensitivity) and get plain, practical guidance built around Avyora's own products.

Who can use it

Signed-in customers only. The page sends signed-out visitors to sign in and back; the server refuses any message without a session before the AI is called, so a refused message costs nothing.

The model and what it costs

Item	Value
Model	Claude Haiku 5.5 (`claude-haiku-5-5`), Anthropic's cheapest current model
List price	$0.10 per million input tokens, $0.50 per million output tokens
Typical message	About ₹0.03 to ₹0.07
Per-person budget	**₹0.50 a day**, charged from each reply's real token usage
Per-person rate	8 messages a minute
Site-wide cap	2,000 messages a day

Item	Value
Reply length	At most 1,200 tokens
History sent	The latest 12 messages only

The daily budget works like a meter. Before each message the server checks the person's spend so far today (`chatSpendDay`, stored in paise in the rate-limit table) without adding to it. After the reply, it calculates the real cost from the token counts Anthropic reports (`replyCostPaise` in `src/modules/skin-chat/chat.ts`, using ₹95 to the dollar on purpose so a weaker rupee never pushes spend over) and adds it. A person can go over by at most one reply, a few paise. At about ₹0.03 to ₹0.04 a message, ₹0.50 is roughly 12 to 15 messages a day.

How it behaves

The instructions to the model (`SYSTEM_PROMPT`) are fixed text, identical on every request so the provider can cache them. They set out:

- **Avyora products first.** When everyday care can help, it recommends the one or two matching products from the catalogue by name, and says which listed concern each is meant for. It never names other brands or shops. If nothing fits, it says so rather than stretching a product.
- **Not a doctor.** It never diagnoses, names a disease as what someone has, or suggests prescription medicines or stopping a treatment.
- **Safety exclusions.** No product recommendations for urgent signs. Nothing with retinal, retinol or retinoid in its name for anyone pregnant, breastfeeding or under 18. No "cures", guarantees, percentages or study claims.
- **Shop in preview.** No prices, offers, delivery times or stock, and no invented ingredients or directions.
- **Stays on topic** and ignores attempts to change its rules or reveal them.
- **Does not ask for personal details** such as names, phone numbers, addresses or photos.

Urgent symptoms

Urgent symptoms are detected **by the server**, not left to the model: swelling of the face, lips, eyes or throat; trouble breathing or swallowing; a mole that is changing, growing or bleeding; pus, fever or spreading redness; widespread blistering; a chemical burn; eye pain or vision change. When one is mentioned, a "see a doctor promptly, or emergency care if it is severe" notice is sent first, whatever the model then says. A test proves it.

Privacy

The conversation exists only in the open browser tab and disappears when it is closed. The server stores nothing but token counts and cost, for billing. Before the first message, a notice explains that it is an AI and not a doctor, that messages go to Anthropic to produce a reply, and that nothing is saved.

Turning it on

1. Create an API key at console.anthropic.com and set a monthly spend limit there as a second safety net.
2. Add it in Vercel as `ANTHROPIC_API_KEY`. Without it the chat says "The skin chat is not switched on yet."

3. Adjust limits without a code change through `RATE_LIMIT_OVERRIDES`, for example `{"chatSpendDay":{"limit":100}}` for ₹1 a day.

PART IV

Running the shop

The screens staff use every day, and the automation that does the rest: deliveries through Shiprocket, stock that reorders itself, and content that cannot publish a false claim.

20. The stockroom

The stockroom console (`/manager`) is for the person who packs parcels. It is open to both staff roles, so the owner can pack when the manager is away. It has four screens: **Dispatch**, **Stock**, **Requests** and **Help**.

Dispatch

Orders are listed **oldest first**, because the person who has waited longest should be served first. Delivered, cancelled, refunded and returned orders leave the queue. Each card shows:

- the order number and date, and what the customer currently sees ("Customer sees: Being prepared");
- what to pack: quantity, product and size for every line;
- who it goes to: name, city, state, PIN code and phone;
- the payment line: "Cash on delivery: collect payment at the door", or in red, "Not paid. Do not send; check with the owner";
- any hold, in red, with the reason;
- once booked: the courier, the tracking number (linked to Shiprocket's tracking page), the latest courier status, and **Print the shipping label**.

Sending an order

1. Pick the items under "Pack" and box them.
2. Press **Mark packed**. The customer now sees "Being prepared".
3. Press **Book courier**. Shiprocket chooses the courier, assigns the tracking number, schedules the pickup and produces the label. Any problem (for example a PIN code no courier reaches) is shown on the card, and the order stays packed for another try.
4. **Print the label**, stick it on the box, and hand it over at pickup.
5. From here the courier's own updates move the order: on its way, out for delivery, delivered. Nothing more to press.

If a parcel goes another way (hand delivery, or Shiprocket not set up), the manual buttons remain: **Hand to courier**, **Out for delivery**, **Mark delivered**. Marking a cash-on-delivery order delivered records the cash as collected.

Holds

Card says	Meaning	What to do
On hold: the owner needs to decide	A payment or courier problem needs the owner	Do not pack. It moves on its own once cleared
On hold: failed an automatic check	A risky cash-on-delivery order	Do not pack. The owner confirms with the customer
Risk check still running	Scored a moment after placing	Wait and refresh
Not paid. Do not send	Online payment never completed	Leave it. The morning sweep cancels it if never paid

The manager cannot cancel an order. Cancelling involves refunds and telling the customer, which is the owner's decision.

Stock

The Stock screen lists every SKU, counted or not, with the count on the shelf.

- **Adding or removing.** Type the change, not the new total: `+24` when putting 24 on the shelf, `-2` when taking two off (damaged, samples), then **Apply**. The database does the arithmetic, so two people counting at once cannot overwrite each other.
- **Asking for more.** Type a quantity under "Need more" and press **Request**. One open request per item; the line then says "Already requested".

Anything showing "Out of stock" cannot be bought by customers, including items nobody has counted.

Help

The Help screen (`/manager/help`, added 21 October 2026) answers the everyday jobs step by step: sending an order, holds, unpaid orders, new stock, low stock, returned parcels and customer calls. It is written for the days the owner is not there to ask.

21. The owner console

The owner console (`/admin`) is for the owner only. Every action on it is checked again on the server, so a manager who guessed an address would still be refused.

Signing in

At `/admin-login`, the owner enters the username and password, then a six-digit code emailed to `OWNER_EMAIL`. The code expires in 10 minutes and allows five guesses. Sessions last 8 hours. Setting `OWNER_TWO_STEP=off` removes the second step, for use only if email is broken.

Screens

Screen	What it is for
Overview	Money taken, orders awaiting payment, paid orders not yet packed, low and out-of-stock warnings

Screen	What it is for
Orders	Every order; open one for its lines, payment history, risk panel, attention panel and status controls
Inventory	Stock levels and backorder switches. Changing a count asks twice, because the stockroom's physical count should win
Pricing	Price and offer per SKU
Analytics	Units and revenue per product over 30 days, days of stock cover, a 7-day estimate
Requests	Restock requests (Ordered or Decline), support messages (Mark answered), reviews to moderate (Publish or Reject)
Content	The CMS (Chapter 25)
Knowledge	Validate, publish, roll back and revoke knowledge releases
Catalogue	Preview catalogue records before onboarding
System	Job queue depth, failed and dead jobs and event deliveries with Replay, pending and failed scans, photos past their deletion time

Prices and offers

In **Pricing**, find the product and size and enter rupees:

- **Price**: the normal price.
- **Offer**: the discounted price, which must be below the normal price.
- **Label**: what customers see on the badge, for example "Festive 20% off".
- **Ends**: leave blank to run until stopped. To end an offer, clear the Offer box and save.

Each price row has a version. If two people edit the same price, the second save is refused with "changed since you opened it", rather than silently overwriting. The storefront, the bag and checkout all use the same pricing rule, so the price shown is the price charged.

Revenue

Revenue counts **paid orders only**. Online orders count when paid; cash-on-delivery orders count when delivered (since 21 October 2026). Cancelled, refunded and returned orders are excluded.

Orders held by the cash-on-delivery risk check

A fake COD order is expensive: you pay to pick, pack, send and get it back, and only find out at the end. Every COD order is scored a moment after it is placed (`src/lib/cod-risk.ts`). Each signal has a weight:

Signal	Weight
Address with no house or flat number	35
Name or address looks like filler text	50
3 or more COD orders from this customer (email or phone) in 24 hours	30, plus 15 for each order beyond 3, up to 70
Earlier COD orders came back undelivered	Up to 80, by the share returned
Large first order with no purchase history	25

The score is **the strongest signal plus 40% of the others**, capped at 100. It is not a plain sum: extra signals count for less than the strongest one. For example, filler text (50) plus a missing house number (35) scores 50 + 14 = 64, a hold rather than a refusal.

Score	Result
Below 40	Approved: goes straight to dispatch
40 or more	Held for the owner's review
70 or more, with at least two signals agreeing	Refused automatically: the order is cancelled and its stock returned

A single signal, however strong, can hold an order but never refuse it. A wrong hold costs one phone call and a few hours; a wrong refusal loses a real customer, so the rules treat the two mistakes differently.

Open the order in Orders. The panel shows the score and the reasons, with two choices:

- **Release for dispatch.** The hold clears and the order appears normally for the stockroom.
- **Cancel and restock.** The order is cancelled and the stock goes back on the shelf.

> **Phone the customer before cancelling.** None of the reasons are proof. An address without a house number is common in older localities and villages; a large first order is what a generous customer looks like. If you find yourself releasing nearly everything, the thresholds (`REVIEW_AT` 40 and `REJECT_AT` 70 in `src/lib/cod-risk.ts`) are too strict and should be tuned against real orders.

Orders that need attention

An order with an attention reason is held from dispatch until the owner clears it with a note saying what was done. The reasons are:

- paid after its stock was sold to someone else;
- paid after it was cancelled;
- lost, damaged, destroyed or untraceable according to the courier;
- undelivered;
- returned to us while prepaid, so a refund is due.

The note is recorded in the audit log.

The audit log

Every sensitive change writes a row to `audit_logs` with who did it, their role, the before and after values, and the request id. That covers prices, stock, order status, risk decisions, attention notes, courier bookings, courier status changes (recorded as "courier"), publishing, knowledge releases, support answers and review moderation. "I did not touch it" can always be checked.

22. Reading the numbers

What each figure in the owner console means, how it is calculated, and how far to trust it.

What counts as a sale

Every sales figure uses one rule, shared by the analytics screen and the morning stock autopilot (`realSale` in `src/lib/analytics.ts`):

Order	Counts?
Paid online	Yes
Cash on delivery, placed and not cancelled	Yes (as demand), even before delivery
Online, abandoned at the payment screen	No
Cancelled (by the owner, the risk check or the sweep)	No
Refunded	No
Returned to us by the courier	No

> **Fixed on 21 October 2026.** The analytics screen used to count returned parcels and online orders abandoned at payment as sales, which inflated units, revenue and the reorder estimate. Both now use the same rule.

The overview

Figure	Meaning
Money taken	Total of orders whose payment status is **paid**: online payments, plus cash-on-delivery orders once delivered
Awaiting payment	Orders still unpaid: open online attempts and undelivered cash-on-delivery orders
Paid, not packed	Paid orders the stockroom has not marked packed yet: the backlog to watch
Low or out of stock	SKUs at or below their low-stock level, and those at zero

Analytics (last 30 days)

Column	How it is calculated
Units sold	Sum of quantities on real sales of that SKU in the window
Revenue	Sum of the line totals (after offers) on those sales, in rupees
Daily rate	Units sold ÷ 30
Cover	Stock on hand ÷ daily rate, in days. Blank when the product is not selling (it would otherwise say "forever") or has no count
Next 7 days	Average daily revenue over the more recent half of the window × 7
Change	Recent half's daily average against the earlier half's, as a percentage. Blank when the earlier half sold nothing
Sample	How many days the estimate is based on

Rows are ordered by units sold, not revenue, because the question the screen answers is "what do I need to reorder", which depends on how many left the shelf.

How much to trust the estimate

The forecast is deliberately simple: a moving average, not a prediction model. With only a few weeks of history, anything cleverer draws a confident line through noise, and stock bought against a made-up trend is real money lost. It assumes the next week looks like the last, which is wrong during a festival, a launch, a viral post or after a bad review. The sample size is printed next to it as its own warning. Treat it as a prompt to look, not an instruction to buy.

The morning autopilot's numbers

The autopilot uses a 28-day window rather than 30 and plans for a 7-day supplier lead time plus 30 days of cover (Chapter 24). Its "days left" is stock ÷ daily rate, rounded down.

Comparing with the payment provider

Revenue in the owner console should match Cashfree's settled payments (online) plus cash collected through Shiprocket's COD remittances, less refunds. A small difference at the edges of a period is normal (a payment made at 23:59 may settle the next day). A larger one usually means a refund made in Cashfree without the order being cancelled here, or a parcel delivered without the courier's update arriving: both show up as individual orders to check.

Where the figures come from

These are the heaviest reads in the system (every order in the window), and none needs the last few seconds of data, so they read with "eventual" consistency: from a read replica when one is configured, otherwise from the main database.

23. Deliveries with Shiprocket

Shiprocket is a courier aggregator: it chooses among Delhivery, Blue Dart, Xpressbees and others by the rules set in the Shiprocket dashboard, books the pickup and produces the shipping label. Avyora's integration was added on 21 October 2026, built from Shiprocket's published API (apidocs.shiprocket.in).

What happens when the manager presses Book courier

The four calls are made in `src/modules/shipping/shiprocket.ts`:

Step	Shiprocket call	Result
1. Sign in	`POST /auth/login`	A token, kept for 9 of its 10 days
2. Create the order	`POST /orders/create/adhoc`	Shiprocket order id and shipment id
3. Assign the courier	`POST /courier/assign/awb`	The tracking number (AWB) and courier name
4. Schedule pickup	`POST /courier/generate/pickup`	Best effort; can be redone in the Shiprocket panel
5. Get the label	`POST /courier/generate/label`	A label PDF link; best effort, also downloadable from the panel

Steps 2 and 3 must succeed; if either fails, the order stays packed and can be booked again. A booking is "claimed" in the database before it starts, so a double click cannot book a parcel twice. The parcel is sent as one standard size (0.5 kg, 15 × 12 × 8 cm by default, adjustable in settings), prepaid or COD, with the order total in rupees, the customer's 10-digit mobile, and the order date in India time.

Only packed orders that are not on hold can be booked: paid online, or cash on delivery that passed the risk check.

Tracking updates

Shiprocket posts every tracking event to `https://YOUR-DOMAIN/api/webhooks/courier`. (Shiprocket refuses webhook addresses containing "shiprocket", "sr" or "kr", hence "courier".) Each request must carry the security token in the `x-api-key` header; it is compared in constant time, and with no token configured the address refuses everything, because an open one would let anyone mark a parcel delivered.

Shiprocket status	Order becomes	Also
Picked up, shipped, in transit, reached destination, delayed, at warehouse, handed to courier	Shipped ("On its way")	The customer gets the "on its way" email with tracking
Out for delivery	Out for delivery	
Delivered	Delivered	Cash on delivery is recorded as paid
Returned to origin and delivered back to us	Returned	Stock is put back on the shelf; a prepaid order is flagged for a refund
Lost, damaged, destroyed, undelivered, untraceable, return initiated	Unchanged	The owner is alerted with the reason
Label generated, manifest, pickup queued and others	Unchanged	Shown on the dispatch card

Updates only move an order **forward**. A late "in transit" cannot pull a delivered order back, two updates racing cannot both move it, and a repeated "returned" cannot restock twice. Shiprocket expects a 200 answer for every delivery, so a processing failure is logged and answered 200; the next update for the same parcel carries the newer status anyway.

Setting it up

1. Create a Shiprocket account and add the pickup address. Note its **nickname** exactly as saved.
2. In Shiprocket, go to Settings → API → **Add New API User**. Use an email different from the main login, and copy the password immediately; it is shown once.
3. In Vercel, set `SHIPROCKET_EMAIL`, `SHIPROCKET_PASSWORD` and `SHIPROCKET_PICKUP_LOCATION`. Optionally set the parcel size: `SHIPROCKET_PARCEL_WEIGHT_KG`, `_LENGTH_CM`, `_BREADTH_CM`, `_HEIGHT_CM`.
4. Generate a long random token (`openssl rand -hex 32`) and set it as `SHIPROCKET_WEBHOOK_TOKEN`.
5. In Shiprocket, go to Settings → API → **Webhooks**. Add `https://YOUR-DOMAIN/api/webhooks/courier`, paste the same token as the security token, and enable it.
6. Apply database migration 0022 (adds `awb` and `shipment` to orders) to production, with approval.
7. Book one real parcel and watch it move through the statuses.

> **Note.** Booking costs money per shipment through your Shiprocket wallet. The integration was tested against a fake Shiprocket that follows the documented API (16 tests), not against a live account.

24. Stock autopilot

Every morning at 08:45 India time, as part of the daily sweep, `runStockAutopilot` in `src/modules/inventory/autopilot.ts` looks after stock so the owner does not have to.

The rule

For each SKU, from real sales over the last **28 days**:

Quantity	Calculation
Daily rate	Units sold in 28 days ÷ 28
Reorder point	The larger of the SKU's low-stock level and (daily rate × 7 days of supplier lead time)
Days left	Stock on hand ÷ daily rate
Quantity to request	Enough for the 7-day lead time plus 30 days of sales, and at least twice the low-stock level, minus what is on hand

"Real sales" means orders paid online, or cash on delivery, that were not cancelled, refunded or returned. Abandoned online orders are not demand. Backorder SKUs are skipped.

Example. A toner that sold 56 units in four weeks sells 2 a day. Its reorder point is 14. With 12 left (6 days), the autopilot asks for 74 − 12 = 62.

What it does

1. Raises a restock request for every SKU at or below its reorder point that does not already have an open one, signed "autopilot", with the reason in words: "Raised automatically. Sells about 2.0 a day, so about 6 days left. Enough for 7 days' delivery plus 30 days of sales."
2. Sends the owner one morning message: WhatsApp (short) and email (the full list). For example: "Avyora, last 24 hours: 3 orders, ₹2,847. Running low: 2 items. 2 new restock requests raised for you to approve."

The owner still decides what is actually bought: closing a request as **Ordered** means "I have ordered it from the supplier". The system's job is to make sure nothing runs dry unnoticed.

Tuning

`LEAD_DAYS` (7), `COVER_DAYS` (30) and `SALES_WINDOW_DAYS` (28) are constants at the top of the file. Each SKU's low-stock level (default 5) is the floor. A festival, a launch or a bad review breaks the "next month looks like last month" assumption, so treat requests as prompts to look, not instructions to buy.

> **Not built, on purpose.** Automatic approval and supplier ordering. Approving means money and a supplier relationship the system does not know about; it needs supplier details and a spending rule from the owner first.

25. Content and the CMS

The CMS (`/admin/content`) manages the words on the site that are not part of the catalogue file.

Content type	Fields
Product copy	Tagline (up to 160 characters), description (up to 2,000), how to use (up to 1,500), up to 6 highlights
Article (journal)	Title, excerpt, body (up to 20,000 characters), hero image

Drafts, publishing and revisions

- **Save draft** stores a revision; nothing changes on the site.
- **Publish** makes it live. It first runs the content rules (Chapter 11) and refuses a draft containing an unsupported offer, an absolute claim or a placeholder: "Not published: … Edit the draft and try again." Publishing invalidates the cache, so the page updates at once.
- **Unpublish** takes it down; **Restore** brings back any earlier revision as a new draft.
- Two editors on one document: the second save is refused with a "changed since you opened it" message instead of overwriting.
- Publishing is rate limited (5 a minute per staff member) and audited.

Media

Images are uploaded to S3-compatible object storage when one is configured (the `STORAGE_S3_*` settings and `STORAGE_PUBLIC_BASE_URL`). Without it, uploads are switched off and articles use no hero image. The CSP allows images from the configured media domain only.

Product copy and directions

Product pages show approved directions first, then the CMS "how to use" text, otherwise "Directions specific to this product are being reviewed". Generic advice such as "use twice daily" is never invented.

PART V

Security and privacy

How the shop stops people getting products for free, taking customer data, or misusing staff access; what personal data it holds, who receives it and how long it is kept.

26. Security

This chapter is organised as a threat model: for each thing an attacker might try, what stops it and where. It reflects a review of the payment, order, account and staff code carried out on 21 October 2026, during which one weakness was found and fixed (the order replay key, below), and the dependency audit reported no known vulnerabilities in the libraries the site runs (`npm audit --omit=dev`: 0).

Getting products for free

Attempt	What stops it	Where
Edit the price in the browser or the request	Prices are never accepted from the client; every line is priced on the server from the catalogue and owner overrides. The shown price is only compared, and a mismatch refuses the order	`src/lib/orders.ts`
Order a negative or huge quantity	Strict schema: whole numbers 1 to 20 per line on the server (the shop caps at 10), at most 50 lines; unknown sizes refused, never substituted	`checkoutSchema`
Order more than is in stock	Reservation is a conditional database update inside the order transaction; it cannot go below zero	`reserveStock`
Pay ₹1 for a ₹1,000 order	Cashfree is asked to collect the server's total; a captured amount different from the order total is rejected by the payment state machine	`state-machine.ts`
Pretend to have paid (fake return page)	The return page's claim is ignored; the server asks Cashfree for the order's payments	`cashfree/verify`
Send a fake payment webhook	The webhook is verified with Cashfree's signature over the exact raw bytes and timestamp, compared in constant time; unsigned or altered requests get 401	`api/webhooks/cashfree`
Replay a real webhook	Each provider event is recorded once by its id; duplicates change nothing	`recordProviderEvent`
Mark an order paid by hand	No screen or API sets payment status; only provider events and COD delivery do, through the state machine	`admin/actions.ts`
Fake a courier "delivered" to trigger COD payment	The courier webhook needs a secret token in `x-api-key`, compared in constant time; with no token configured it refuses everything	`api/webhooks/courier`
Use a discount code	There are no coupons, bundles or loyalty points; no code path applies a discount. The content rules block advertising offers checkout does not apply	`content-claims.ts`

Attempt	What stops it	Where
Place fake cash-on-delivery orders	Risk scoring holds or refuses suspicious ones; guests prove their mobile number by SMS when SMS is configured; checkout is rate limited	`cod-risk.ts`, `checkout-phone.ts`
Hold stock hostage with abandoned payments	Abandoned online orders are cancelled (after asking Cashfree) and their stock returned, on back-out and by the morning sweep	`reservation-sweep.ts`
Order a sample product on the live shop	Sample catalogue orders are refused in production	`catalogue-mode.ts`
Cancel someone else's payment	The cancel endpoint needs that order's signed access token	`cashfree/cancel`

Taking customer data

Attempt	What stops it
Open another customer's order page	Needs the signed-in owner of the order, or a signed token bound to that order number. A token for one order is useless for another
Guess orders on the tracking page	Needs order number **and** email; same answer for "no such order" and "wrong email"; 20 lookups a minute per identity and per address
Reuse another customer's checkout replay key	**Fixed 21 October 2026.** A retried checkout used to return the existing order (and a link to it) for anyone holding its random idempotency key. It now does so only for the same email; anyone else gets a generic error. A test covers it
Read or change another customer's address, routine, scan or wishlist	Every query is scoped to the owner (account id, or the guest cookie's hash); a foreign id answers "not found". Tests check each
Write one person's bag into another's	Bag saves carry an account key; a mismatch is refused
Enumerate accounts or newsletter subscribers	Sign-in and newsletter answers do not reveal whether an address exists
SQL injection	Every query goes through Drizzle with bound parameters; there is no string-built SQL. A test runs classic payloads against a real Postgres
Cross-site scripting	React escapes everything rendered; the only raw HTML is the product's structured data, built from our own catalogue with < escaped. Ask Avyora renders plain text only
Cross-site request forgery	Server actions carry Next.js's origin check; every state-changing API compares `Origin` with the site's host
Read data from logs	All logs and error reports pass through `redact()`: credentials, emails, phones, addresses, card fields, questionnaire answers, photos, IP addresses, tokens and signed URLs are masked, and free text is scrubbed of emails and mobile numbers
Intercept data in transit	HTTPS everywhere, with `Strict-Transport-Security` (2 years, preload)

Staff access

Control	Detail
Separate roles	Owner and manager, with the role sealed inside the signed session
Every action re-checked on the server	`isAdmin()` (owner only) at the top of every owner action; pricing, content, system and knowledge also check the role themselves. Middleware alone is never trusted
Passwords	PBKDF2-SHA256, 210,000 iterations, per-password salt, constant-time comparison; stored only as hashes in settings
Owner second step	A six-digit code emailed to `OWNER_EMAIL` (added 21 October 2026). A leaked password alone cannot open the owner console
Sessions	HMAC-signed, `httpOnly`, `SameSite=Lax` cookie; 8 hours
Brute force	8 sign-in attempts per 15 minutes per username and per address; codes allow 5 guesses; same timing whether the username or the password was wrong
Fail closed	No credentials, no session secret, or no rate limiter means no staff sign-in
Audit	Every sensitive change is logged with actor, role, before and after

Abuse and cost control

All limits are durable counters in Postgres shared by every server instance, keyed by hashed identifiers so no raw email or IP address is stored. Each can be changed without a code change through `RATE_LIMIT_OVERRIDES`.

Policy	Per identity	Per address	Window	If the counter is down
Customer sign-in	10	30	10 minutes	Refuse
Send a code	3	5	15 minutes	Refuse
Resend a code	1	—	60 seconds	Refuse
Staff sign-in	8	8	15 minutes	Refuse
Order lookup	20	20	1 minute	Refuse
Checkout	10	10	10 minutes	Refuse
Payment creation	15	15	10 minutes	Refuse
Price quote	60	60	1 minute	Allow (cheap, cached)
Support form	3	10	1 hour	Refuse
Reviews	5	20	1 hour	Refuse
AI chat messages	8	8	1 minute	Refuse
AI chat spend	₹0.50	₹0.50	1 day	Refuse
AI chat, whole site	2,000	—	1 day	Refuse
Page views (edge)	—	120	1 minute	Best effort

Request bodies are read with hard size limits (for example 4 KB for staff sign-in, 48 KB for a chat, 64 KB for a payment webhook, 256 KB for a courier webhook) and refused with 413 once over, without buffering the whole body.

The AI skin chat

Risk	Control
Running up the bill	Sign-in required; ₹0.50 per person per day metered from real usage; 2,000 messages a day site-wide; reply and history length caps; a spend limit set on the Anthropic account
Harmful advice	Fixed safety instructions; urgent symptoms detected by the server and the doctor notice sent first regardless of the model
Prompt injection ("ignore your rules")	Only user and assistant turns are accepted (the request schema rejects a "system" role or extra fields); the rules sit in the fixed system prompt, which tells the model to ignore such requests
Leaking personal data	Nothing is stored; the customer is told not to share personal details

Security headers

Set on every response by the middleware and `next.config.ts`:

- **Content-Security-Policy:** scripts only from this site and Cashfree's SDK (`sdk.cashfree.com`); network calls only to this site, Cashfree and Sentry; frames only from Cashfree; `object-src 'none'`, `base-uri 'self'`, `form-action 'self'`, `frame-ancestors 'none'`.
- **Strict-Transport-Security**, **X-Content-Type-Options**, **X-Frame-Options**, **Referrer-Policy**, **Cross-Origin-Opener-Policy**.
- **Permissions-Policy:** camera, microphone and location denied everywhere, except camera on `/scan` only.

> **Known trade-off.** The CSP allows inline scripts, because almost every page is pre-built and a per-request nonce would block the scripts Next.js emits. React's escaping is the defence against inline injection. If the site moves to mostly per-request rendering, switch to a nonce.

Secrets

- All secrets live in Vercel's environment settings, never in the code or the browser bundle. Only variables starting `NEXT_PUBLIC_` reach the browser, and none of those is secret.
- `AUTH_SECRET` (customer sessions) and `SESSION_SECRET` (staff sessions) must be different. `npm run check:env -- --production` fails if they match, and never prints a secret value.
- Generated local test values live in `.env.acceptance.local`, which is git-ignored.

What is still open

Gap	Risk	Action
No web application firewall	A determined scraper or a flood of traffic is limited only by the application's rate limits	Turn on Vercel Firewall (Bot Protection; Attack Challenge Mode when under attack), or put the domain behind Cloudflare with its managed rules
Nightly backup not running	Only 6 hours of history to recover from	Set the six `BACKUP_*` secrets (Chapter 33)

Gap	Risk	Action
Neon database password	It has appeared in tool output during development	Rotate it in Neon and update `DATABASE_URL` in Vercel
WhatsApp token blocked	Owner alerts fail (email still works)	Renew the token in Meta's developer console
Inline scripts allowed by CSP	See above	Move to nonces if rendering becomes dynamic
No alerting channel	Problems are visible in the owner console and logs, but nobody is paged	Configure Sentry (`SENTRY_DSN`) and an alert rule

27. Privacy and data protection

This chapter lists what personal data the shop holds, why, who else receives it, and how long it is kept. It describes the system; it is not legal advice. India's Digital Personal Data Protection Act 2023 and the IT Rules apply to an online shop, and the policy wording should be reviewed by a lawyer before launch.

What is collected, and why

Data	When	Why	Where
Name, email, phone, delivery address	Checkout	To deliver the order and contact the customer about it	`orders`, `addresses`
Order contents and amounts	Checkout	The record of the sale, tax and accounts	`orders`, `order_items`
Google account email and name	Google sign-in	To create and identify the account	`users`, `accounts`
Password (hashed only)	Password sign-up	Sign-in	`users`
Sign-in codes (hashed only)	Code sign-in	Sign-in; deleted after use or expiry	`otp_codes`
Bag and wishlist	Signed-in shopping	To keep them across devices	`carts`, `cart_items`, `wishlist_items`
Quiz answers and saved routines	Only if the customer allows saving	To show the routine again	`skin_profiles`, `routine_results`
Face photo	Only if the scan feature is on and the customer consents	Analysis (not available)	Private storage, at most 24 hours
Reviews, support messages	When submitted	To publish the review; to answer the message	`reviews`, `support_requests`
Chat messages	AI skin chat	To produce a reply	**Not stored** by Avyora; sent to Anthropic for the reply
Anonymous usage events	Browsing	Analytics: aggregate skin type and concern counts, page use	`activity_events`
Network address	Every request	Rate limits	Stored only as a keyed hash, never the raw address

Permissions (consent)

Anything beyond what an order needs is saved only with a recorded permission:

Permission	Covers	Withdrawn by
`routine_saving`	Saving quiz answers and routines	Account → Routines → withdraw; saved routines stop being shown at once
`photo_processing`	Uploading a face photo (feature switched off)	Account → withdraw; the photo is deleted immediately
`model_research`	Use in future research (not used)	—

Each grant is stored with the policy version the customer agreed to, in an append-only table: a permission is never edited, only withdrawn and, if given again, granted anew. The database itself refuses to save a routine or scan without a live permission from the same owner.

How long data is kept

Data	Kept for	How it is removed
Guest routine	30 days	Daily sweep
Account routine	180 days	Daily sweep
Face photo	At most 24 hours (a database rule)	Deleted on completion, failure or withdrawal; sweep retries failed deletes
Scan observations	At most 7 days	Daily sweep
Unconfirmed newsletter sign-up	7 days	Daily sweep
Sign-in codes	Usable for 10 minutes; the row is deleted a day after expiry	Daily sweep (scheduled from 21 October 2026; the clean-up existed but was never run before)
Rate-limit counters	One day after their window	Daily sweep
Orders and invoices	No automatic deletion	Needed for tax and accounts; the period is an owner decision (see below)
Support requests	Proposed 12 months after closing	**Not implemented**: needs the owner to confirm the period
Consent records	Kept as the audit of permissions	Period for owner and legal review
Chat conversations	Not stored	Exist only in the open browser tab

Who else receives data

Service	Receives	Purpose
Cashfree	Order number, amount, customer name, email, phone	Taking payment
Shiprocket and its couriers	Name, address, phone, email, items, amount (for COD)	Delivery
Gmail or Resend	Email address and message content	Sending order emails and codes
Meta (WhatsApp)	Order number, total, item count, city (owner alerts only; no customer name)	Telling the owner about orders
Anthropic	Chat messages	Producing the AI reply

Service	Receives	Purpose
Google	Sign-in only	Google sign-in
Neon, Vercel	Everything stored or processed	Hosting
Open-Meteo	A region derived from the PIN code, never the address	Weather notes
Sentry (if enabled)	Error reports, after redaction	Error monitoring

Customer rights in practice

Request	How it is handled today
See my data	Account → Data shows what is held and the permissions granted
Delete my saved routines	Account → Routines → delete, or withdraw the permission
Delete my account	**No self-service button yet.** The database is built for it: deleting a user removes their permissions, saved answers, routines, scans and feedback in one cascade, and keeps their orders (needed for accounts) detached from the account. Until a button is built, the owner handles requests by email and a developer runs the deletion
Correct my details	Addresses are editable in the account; order addresses are frozen as the record of delivery
Complain	The grievance officer named on the policy pages

Safeguards built into the system

- Passwords, codes, guest secrets and network addresses are stored only as hashes.
- Logs and error reports are redacted before anything is written (Chapter 26).
- Private pages are never cached by the CDN or a shared browser cache.
- Face photos are re-encoded on upload, which strips location and camera data.
- Staff screens show only what each role needs: the manager sees delivery details, not revenue; review moderation shows the text, not the reviewer's contact details.

Owner decisions still needed

From `docs/legal-facts-needed.md`:

1. **Grievance officer:** a named person, with email and address, published on the policy pages. The law expects complaints acknowledged within 24 hours and resolved within 15 days.
2. **Registered entity details:** name, address and GSTIN for the terms, privacy and contact pages.
3. **Retention periods** for order records and support messages.
4. **A self-service "delete my account" button**, if wanted (a small piece of work, since the database already cascades).
5. **Return window for cosmetics:** most Indian direct-to-consumer brands accept returns only on unopened items for hygiene reasons; whatever is chosen must be stated plainly.
6. **Who pays return shipping.**

7. **Labelling compliance:** cosmetics sold in India need a manufacturing licence under the Cosmetics Rules 2020, and Legal Metrology rules require net quantity, MRP, batch number and expiry on the label. A labelling failure means a recall.

PART VI

Operating the system

Deploying and configuring, how far the system scales, how to watch it and recover from problems, and how it is tested.

28. Deployment and settings

Environments

Environment	Address	Database	Used for
Local development	http://localhost:9002 (`npm run dev`)	Whatever `.env.local` points at	Day-to-day development
Local acceptance	http://localhost:9006 (`acceptance` launch entry)	A disposable Postgres in Docker (`avyora-acceptance-db`, port 5435)	Full checks with a real database and test data; needs Docker Desktop running
Redesign review	http://localhost:9003 / 9004	None (deliberately empty)	Reviewing the design without touching data
Preview	Vercel preview URLs, one per branch	A Neon branch (never production)	Checking a change before it merges
Production	https://avyora-beauty.vercel.app	Neon `main` branch	The live shop

The local launch entries are defined in `.claude/launch.json`: `dev` (9002), `prod-verify` (3000), `dev-redesign` (9003), `start-redesign` (9004), `acceptance` (9006) and `acceptance-start` (9007).

> **Common mistake.** `.env.local` points at `localhost:5432`. If another project's database container is using that port (as happened on 21 October 2026), local builds and the 9002 server talk to the wrong database and time out. Stop the other container, or point `.env.local` at your own database.

Deploying to production

1. **Check settings:** `npm run check:env -- --production`. It reports what is missing or malformed and never prints a secret.
2. **Database:** use Neon's **pooled** connection string (the host contains `-pooler`). Apply new migrations, after approval: `DATABASE_URL='<production-url>' npm run db:migrate`, ideally on a Neon branch first.

3. **Stock:** a fresh database sells nothing until stock is counted: `npm run db:seed-inventory 25`, or count in the stockroom.
4. **Merge to `main`.** Vercel builds and deploys automatically. A production build refuses to run without a database, so fallback pages are never published as real.
5. **Verify in order:** the home page renders; `/api/health` answers; `/api/auth/session` returns 200 (a 500 means `AUTH_SECRET` is missing or the database is unreachable); sign in with Google; save an address; place a test order.

Rolling back

- **Code:** in Vercel, promote the previous deployment (instant). Migrations are additive, so older code runs against the newer schema.
- **A migration:** migrations are additive, so rolling back the code is usually enough. Migration 0014 also has a rollback script (`docs/runbooks/0014_catalog_variants.rollback.sql`); rehearse any schema rollback on a Neon branch first and run it in one transaction.
- **A knowledge release:** roll back from `/admin/knowledge`.

Settings that matter most

The full list of 67 settings is in Appendix C. The essential ones:

Group	Settings
Database	`DATABASE_URL` (pooled), optional `DATABASE_READ_URL`
Customer sign-in	`AUTH_SECRET`, `AUTH_GOOGLE_ID`, `AUTH_GOOGLE_SECRET`
Staff sign-in	`ADMIN_USERNAME`, `ADMIN_PASSWORD_HASH`, `MANAGER_USERNAME`, `MANAGER_PASSWORD_HASH`, `SESSION_SECRET` (32+ characters, different from `AUTH_SECRET`), `OWNER_TWO_STEP`
Email	`GMAIL_USER` and `GMAIL_APP_PASSWORD`, or `RESEND_API_KEY` and `EMAIL_FROM`; `OWNER_EMAIL`
WhatsApp	`WHATSAPP_TOKEN`, `WHATSAPP_PHONE_NUMBER_ID`, `WHATSAPP_TO`
Payments	`CASHFREE_APP_ID`, `CASHFREE_SECRET_KEY`, `CASHFREE_ENV`
Couriers	`SHIPROCKET_EMAIL`, `SHIPROCKET_PASSWORD`, `SHIPROCKET_PICKUP_LOCATION`, `SHIPROCKET_WEBHOOK_TOKEN`, parcel size
AI chat	`ANTHROPIC_API_KEY`
Jobs	`CRON_SECRET`
Shop mode	`NEXT_PUBLIC_CATALOGUE_MODE`, `NEXT_PUBLIC_SITE_URL`
Tuning	`RATE_LIMIT_OVERRIDES` (JSON)

Staff password hashes are generated with `node scripts/hash-password.mjs 'the-password'` (at least 12 characters). The hash is colon-separated, not `$`-separated, because `$` is mangled when environment files are loaded.

Feature switches

Setting	Effect when set
NEXT_PUBLIC_REDESIGN=1	The current design (Inter type, measured layout); unset shows the earlier design
NEXT_PUBLIC_CATALOGUE_MODE=verified	Sell only onboarded, verified products
ALLOW_SAMPLE_ORDERS=1	Staging only: allow sample orders
NEXT_PUBLIC_FACE_SCAN=1	Show the optional face-photo page
FACE_SCAN_HOSTED=1	Allow photo uploads (needs PRIVATE_STORAGE)
NEXT_PUBLIC_NEWSLETTER=1, NEWSLETTER_ENABLED=1	Show and accept newsletter sign-ups
OWNER_TWO_STEP=off	Owner signs in with password only (emergency use)

Google sign-in

The production redirect address must be registered on the Google OAuth client exactly: `https://avyora-beauty.vercel.app/api/auth/callback/google`. A trailing slash or `http` instead of `https` causes `redirect_uri_mismatch`, the most common launch-day failure. While the Google app is in **Testing**, only listed test users can sign in; publish it for real customers.

Commands

Command	What it does
npm run dev	Development server on port 9002
npm run build / npm run build:offline	Production build; offline builds without a database (for CI)
npm run typecheck, npm run lint, npm run format:check	Code checks
npm test, npm run test:coverage	The test suite, with coverage thresholds
npm run test:stress	Concurrency tests against a scratch database
npm run db:migrate	Apply migrations
npm run db:seed-inventory <n>	Give every SKU a stock count
npm run db:check-catalog	Check catalogue file and database agree
npm run kb -- build / status / publish -- confirm / rollback / revoke	Knowledge releases
npm run check:env -- --production	Check settings without printing secrets
npm run check:launch	List launch blockers
npm run catalogue:validate	Validate product onboarding records
npm run load-test	Load test a running build

29. Setting up the outside services

Each service below is set up once, by the owner, in that service's own website. The values it produces go into Vercel under Project → Settings → Environment Variables (scope: Production), and locally into `.env.local`. **Never paste these values into chat, email or the code.** After changing any of them, redeploy (Vercel → Deployments → the latest → Redeploy) so the site picks them up, then run `npm run check:env -- --production` to confirm.

Neon (the database)

1. Sign in at console.neon.tech and open the Avyora project.
2. **Connection string:** Dashboard → Connect. Choose the **pooled** connection (the host contains `-pooler`). This is `DATABASE_URL`.
3. **Branches for rehearsals:** Branches → Create branch from `main`. A branch is a full copy of production at that moment, cheap and quick. Use one to rehearse a migration or test against real data, and delete it afterwards.
4. **Point-in-time restore:** Branches → Create branch → choose "Past data" and a timestamp. On the free plan the window is 6 hours.
5. **Rotate the password** (recommended now): Roles → the application role → Reset password. Copy the new connection string into `DATABASE_URL` in Vercel straight away, then redeploy; the old password stops working immediately.
6. **When traffic grows:** upgrade the plan for more compute (autoscaling), more storage and a longer restore window. Optionally add a read-only compute and set `DATABASE_READ_URL`.

Vercel (hosting)

1. **Plan:** Settings → Billing → upgrade to **Pro** before taking real orders. The free Hobby plan forbids commercial use.
2. **Settings:** Settings → Environment Variables. Set each value for Production; preview deployments should use their own Neon branch, never production.
3. **Region:** functions run in `sin1` (Singapore), set in `vercel.json`, next to the Neon database.
4. **Cron:** the schedules are in `vercel.json`. On Pro, change the sweep to `*/15 * * * *`.
5. **Firewall:** Firewall → enable Bot Protection. Under active abuse, turn on Attack Challenge Mode.
6. **Domain:** Settings → Domains → add the shop's own domain when bought. Then update `NEXT_PUBLIC_SITE_URL`, the Google sign-in redirect, and the Cashfree and Shiprocket webhook addresses.

Google sign-in

1. Go to console.cloud.google.com → APIs & Services → Credentials → Create credentials → OAuth client ID (type: Web application).
2. **Authorised redirect URIs**, exactly:
   - `http://localhost:9002/api/auth/callback/google` (local)

   - `https://avyora-beauty.vercel.app/api/auth/callback/google` (production, and later the shop's own domain)
3. Copy the client id and secret into `AUTH_GOOGLE_ID` and `AUTH_GOOGLE_SECRET`.
4. OAuth consent screen → **Publish app**. While it is in Testing, only listed test users can sign in. Basic profile and email scopes need no Google review.
5. Generate `AUTH_SECRET` (for example `openssl rand -base64 32`). It must differ from `SESSION_SECRET`.

Email: Gmail or Resend

The site sends order confirmations, shipping emails, sign-in codes, the owner's two-step codes and the morning summary. Either provider works; if both are set, Gmail is used.

Gmail (simplest to start):

1. On the Google account that will send mail, turn on 2-Step Verification (Google Account → Security).
2. Security → 2-Step Verification → **App passwords** → create one named "Avyora".
3. Set `GMAIL_USER` (the address) and `GMAIL_APP_PASSWORD` (the 16-character app password).
4. Limit: about 500 messages a day. Move to Resend before real volume.

Resend (for volume):

1. Create an account at resend.com and **verify a sending domain** (DNS records at the domain registrar). Until a domain is verified, Resend only delivers to the account owner's own address.
2. Create an API key; set `RESEND_API_KEY` and `EMAIL_FROM` (an address on the verified domain).

Owner address: set `OWNER_EMAIL` to where order alerts, the morning summary and the owner's sign-in codes should go.

WhatsApp alerts (Meta Cloud API)

1. At developers.facebook.com, create an app of type Business and add the WhatsApp product.
2. Add and verify the sending phone number. Note its **phone number id** (not the number itself): `WHATSAPP_PHONE_NUMBER_ID`.
3. Create a **permanent** access token through a system user in Meta Business Settings, with WhatsApp messaging permission: `WHATSAPP_TOKEN`. Temporary tokens expire within a day, which is the usual cause of "API access blocked".
4. `WHATSAPP_TO`: the owner's number with country code and no plus sign, for example `91XXXXXXXXXX`.
5. **Template:** a business-initiated message outside a 24-hour conversation window must use an approved template, or Meta silently drops it. Create a template for order alerts, wait for approval, and set `WHATSAPP_TEMPLATE` (and `WHATSAPP_TEMPLATE_LANG`, default `en`).

SMS codes (MSG91, optional)

Indian transactional SMS requires **DLT registration** with TRAI: the business registered with an operator, and each message template approved in advance. Messages from unregistered senders are dropped by the carriers. This takes days to weeks, so treat it as a procurement task.

Once registered, set `MSG91_AUTH_KEY` and `MSG91_OTP_TEMPLATE_ID`. Phone sign-in and the guest checkout phone check then appear automatically; without SMS they stay hidden rather than offering something that cannot send.

Cashfree (online payments)

1. Sign in to the Cashfree merchant dashboard. Complete **KYC** and payment gateway activation. Cashfree reviews the site's policy pages, so fill in their `[TO CONFIRM]` details first.
2. Developers → API keys: copy the **App ID** and **Secret key** into `CASHFREE_APP_ID` and `CASHFREE_SECRET_KEY`. Set `CASHFREE_ENV=production` (or `sandbox` for testing).
3. Developers → Webhooks → add `https://YOUR-DOMAIN/api/webhooks/cashfree` for payment events. The site verifies each one with the secret key.
4. Place one small real order and refund it from the dashboard, to prove the whole path.

Shiprocket (couriers)

The full steps are in Chapter 23. In short:

1. Add the pickup address in Shiprocket; note its nickname: `SHIPROCKET_PICKUP_LOCATION`.
2. Settings → API → Add New API User (with a different email from the main login): `SHIPROCKET_EMAIL`, `SHIPROCKET_PASSWORD`.
3. Generate a long random token: `SHIPROCKET_WEBHOOK_TOKEN`.
4. Settings → API → Webhooks: `https://YOUR-DOMAIN/api/webhooks/courier`, the same token, enabled.
5. Courier priority (cheapest, fastest, or recommended) is set in Shiprocket's own settings; the site uses whichever courier Shiprocket assigns.
6. Keep the Shiprocket wallet topped up: bookings fail when it is empty.

Anthropic (the AI skin chat)

1. At console.anthropic.com, create an organisation and add billing.
2. Settings → Limits: set a **monthly spend limit** (for example $5). This is a second safety net behind the site's own ₹0.50 per person per day.
3. API Keys → Create key: `ANTHROPIC_API_KEY`.
4. Usage → watch daily spend in the first weeks.

Sentry (error alerts, optional)

1. Create a project at sentry.io (platform: Next.js).
2. Copy the DSN into `SENTRY_DSN` and `NEXT_PUBLIC_SENTRY_DSN`.
3. Create an alert rule (for example, email when a new issue appears). Without Sentry, errors are still logged as structured JSON in Vercel; they just do not page anyone.

Upstash Redis (shared cache, optional)

Only worth adding when traffic grows. Create a Redis database at upstash.com in a Singapore region, and copy its REST URL and token into `REDIS_REST_URL` and `REDIS_REST_TOKEN`. Losing Redis only costs speed, never correctness.

Nightly backups

1. Create a **private** S3-compatible bucket (Cloudflare R2, AWS S3 or similar), separate from any media bucket, with a lifecycle rule that deletes objects after 30 days.
2. Create an access key limited to that bucket.
3. In GitHub → the repository → Settings → Secrets and variables → Actions, add:

Secret	Value
`BACKUP_DATABASE_URL`	A **direct** (not pooled) Neon connection string
`BACKUP_PASSPHRASE`	A long random passphrase. Store it somewhere safe: without it the backups cannot be read
`BACKUP_S3_ENDPOINT`	The bucket's endpoint, for example `https://<account>.r2.cloudflarestorage.com`
`BACKUP_S3_BUCKET`	The bucket name
`BACKUP_S3_ACCESS_KEY_ID`	The access key id
`BACKUP_S3_SECRET_ACCESS_KEY`	The secret key

1. Run the workflow once by hand (Actions → Backup → Run workflow) and check a file appears in the bucket. From then on it runs daily at 02:00 India time.
2. Once a month, restore last night's dump into a scratch Neon branch (Chapter 31) and delete the branch afterwards.

Staff passwords

1. On a computer with the project: `node scripts/hash-password.mjs 'a-long-new-password'` (at least 12 characters). It prints the hash only; the password itself is stored nowhere.
2. Set `ADMIN_PASSWORD_HASH` (owner) or `MANAGER_PASSWORD_HASH` (manager), and the matching usernames.
3. `SESSION_SECRET`: at least 32 random characters, different from `AUTH_SECRET`. Changing it signs every staff member out.

30. Scaling and performance

How the system scales

- **Servers** scale automatically on Vercel: each request can run on a new instance. There is no server to resize.
- **Public pages** are pre-built and served from Vercel's edge, so readers barely touch the servers and never touch the database.

- **The database** is the first thing to fill up, because checkout, sign-in, carts and rate limits all write to it. Neon can be given more compute with a setting change, and a read replica can take the reporting load.
- **Costly actions are capped** (payments, sign-in, AI chat), so a spike in traffic cannot become a spike in the bill.

Load test (12 October 2026)

A local production build against a copy of the production database (Neon branch, same 0.25 compute units), 20 seconds per stage:

Simultaneous visitors	Requests a second	Median	95th percentile	Errors
10	147	29 ms	283 ms	0
50	133	92 ms	1.9 s	0
100	179	266 ms	3.3 s	0
250	134	876 ms	6.3 s	0
500	149	2.0 s	10.5 s	876 (the laptop's connection limit)

One local process is not Vercel's capacity. What carries over is the shape: pages are cheap, and the database-bound APIs (cart, sign-in, checkout, rate limiter) are the bottleneck.

Plan limits

Platform	Free tier limit	What happens
Vercel Hobby	Non-commercial use only; 1,000,000 function calls, 4 active-CPU hours and 100 GB transfer a month; scaling by 1,000 instances per 10 seconds	Exceeding a monthly limit pauses that feature until the month resets; a sudden spike gets `503 FUNCTION_THROTTLED`
Neon free	0.25 compute units, 512 MB per branch, 6 hours of history	Slower queries under load; limited recovery window
Vercel Cron (Hobby)	Daily jobs only	Abandoned stock waits up to a day

What to change, and when

When	What runs out	Fix
Before taking real orders	Vercel Hobby is not allowed for commercial use	**Move to Vercel Pro (about $20 a month).** Then set the sweep to every 15 minutes
A few thousand customers a day	Neon free compute and storage	Paid Neon plan with autoscaling; longer history
Tens of thousands a day	Rate-limit writes on the database	Move rate limits to Redis (one file, `rate-limit.ts`); set the Redis cache settings
Hundreds of thousands a day	Database reads	Add a Neon read replica (`DATABASE_READ_URL`)
Millions of users	Everything	A dedicated team and a re-architecture; not worth planning for now

Performance measured

Public pages: 0 database queries for 14 page requests served from cache; HTML 7 to 20 KB compressed; layout shift 0 on every page at 1280, 1440 and 1920 pixels wide; no accessibility findings in the automated acceptance pass.

31. Monitoring, incidents and runbooks

Where to look

Question	Where
Is the site up?	`/api/health` (checks the database too)
Errors in the last hours	Vercel → Project → Logs (Hobby keeps raw logs for 1 hour) or Observability → Errors
Stuck emails or jobs	Owner console → System: failed and dead jobs and event deliveries, with Replay
Orders needing a decision	Owner console → Orders (held and attention panels)
Payment state of an order	The order page in the owner console (payment events listed) and the Cashfree dashboard
Parcel state	The dispatch card, and Shiprocket's panel
AI chat spend	Anthropic console usage; `chat reply` log lines carry tokens and paise

Every log line is structured JSON with a request id. An error page shows a reference number (the request's correlation id), which finds the matching log line.

Runbooks

The site shows "Something went wrong"

1. Note the reference number on the page.
2. Check `/api/health`. If the database is down, public pages still show products as out of stock; check Neon's status page and the project's compute.
3. Search the Vercel logs for the reference number.
4. If the latest deployment caused it, promote the previous deployment in Vercel.

A customer paid but the order shows unpaid

1. Open the order; check its payment events.
2. Check the Cashfree dashboard for the order number.
3. If Cashfree shows success, wait for the webhook or the next sweep, which reconciles. Never mark it paid by hand: there is no such button on purpose.
4. If the webhook is failing, check `CASHFREE_SECRET_KEY` and the webhook address in Cashfree.

A paid order shows "needs attention"

Read the reason. If the stock was sold to someone else: refund in Cashfree, or restock and send. If paid after cancelling: confirm with the customer, then send or refund. Clear the hold with a note saying what was done.

Order emails are not arriving

1. Owner console → System: look for failed deliveries of the `notifications` consumer and their error.
2. Gmail: check the app password is still valid. Resend: the sending domain must be verified.
3. Fix, then press Replay. Failed sends retry five times on their own first.

WhatsApp alerts fail with "API access blocked"

The Meta token or app permission has expired or been blocked. Create a new permanent token in Meta's developer console and update `WHATSAPP_TOKEN` in Vercel.

"Book courier" fails

Read the message on the card. "Pincode not serviceable" means no courier reaches that address: call the customer. Sign-in failures mean the Shiprocket API user's password changed: update `SHIPROCKET_PASSWORD`. The order stays packed; book again when fixed, or send another way and use the manual buttons.

Parcels are not moving to "On its way"

Check Shiprocket → Settings → API → Webhooks: the address, the toggle, and that the token equals `SHIPROCKET_WEBHOOK_TOKEN`. A 401 in the logs for `/api/webhooks/courier` means the token is wrong; a 503 means it is not set.

The owner cannot sign in (no code arrives)

Check email delivery works (the same settings as order emails). In an emergency, set `OWNER_TWO_STEP=off` in Vercel, redeploy, sign in, fix email, then remove the setting.

Restoring the database

Within 6 hours: in Neon, create a branch from `main` at a time just before the incident, check it, then copy the damaged rows back or make it the default and update `DATABASE_URL`.

From a nightly dump (once backups run):

```
aws s3 cp s3://$BUCKET/db/<stamp>.dump.enc . --endpoint-url $ENDPOINT
openssl enc -d -aes-256-cbc -pbkdf2 -iter 200000 -in <stamp>.dump.enc -out avyora.dump -pass
env:PASSPHRASE
pg_restore --no-owner --no-privileges --dbname "$RESTORE_TARGET_URL" avyora.dump
npm run db:migrate
```

Then compare row counts for `orders`, `order_items`, `payment_events` and `inventory` against Cashfree's totals before switching traffic. Rehearse this once a month on a scratch branch.

32. Testing and quality

The test suite

Measure	Value (21 October 2026)
Test files	100 (listed in Appendix E)
Tests	1,131 passing, 3 opt-in skipped

Measure	Value (21 October 2026)
Integration tests	Run against PGlite, a real Postgres engine compiled to WebAssembly, built from the same migrations as production
Run time	About 4 minutes on the development machine

Integration tests are the backbone. A mocked database would test the mock; these run the real SQL, constraints and triggers that the guarantees rest on. Outside services (Cashfree, Shiprocket, Anthropic, email, WhatsApp) are replaced with fakes that follow their documented APIs.

How tests are trusted

Important tests are checked by **mutation**: the protection they guard is deliberately broken, and the test must fail. Examples from this project:

- Removing the stock guard (`quantity >= n`) fails four inventory tests, including "twenty attempts against ten units grant exactly ten".
- Letting the courier move an order backwards fails two shipping tests.
- Treating "prefer not to say" on pregnancy as "no" fails three routine tests.
- Alerting the owner when an online order is created (rather than paid) fails a payment test.
- Disabling the urgent-symptom notice fails a chat test.

Areas and their tests

Area	Covered by
Pricing and checkout	Cart, checkout quote, display-matches-charge, idempotency (including the replay-key fix), price-change refusal
Stock	Reservation, restoration exactly once, sweep, autopilot
Payments	State machine, webhook signatures and dedupe, reconciliation, Cashfree back-out
Deliveries	Shiprocket booking (fake API), tracking mapping, forward-only rule, webhook token, returns and COD collection
Security	SQL injection payloads, rate limits (including concurrency on real Postgres), body limits, trusted IP, ownership on every personal record, owner two-step sign-in, CSP and camera policy
Personalisation	560-profile routine matrix, selection, planner, inference, knowledge compiler, assistant
Content	Claim rules over all customer-facing text, CMS publish checks
Outages	Storefront renders during a database outage; checkout still refuses

Continuous integration

Every push and pull request runs, on GitHub Actions: install, type check, lint, formatting, the full test suite with coverage thresholds, a runtime dependency audit (fails on high), a build-tools audit (fails on critical), and an offline production build.

Acceptance checks

`scripts/acceptance-pass.mjs` loads ten key pages at three widths in headless Edge, saves screenshots, and records load timings, layout shift, page weight, horizontal overflow and basic accessibility checks (language, one main heading, heading order, image alternative text, named buttons and links, labelled fields). The last full pass found no accessibility issues, no overflow and zero layout shift.

PART VII

History and next steps

What is left before a full launch, how the system came to be what it is, and why it was built this way.

33. What is left before launch

Code is not what stands between Avyora and a full launch. These are the remaining items, most of which only the owner can supply.

Must do before taking real orders

#	Item	Why	How
1	Move Vercel to Pro	Hobby forbids commercial use and pauses features at its limits	Vercel → Settings → Billing (about $20 a month); then make the sweep run every 15 minutes
2	Business details	Policy pages carry `[TO CONFIRM]`; Cashfree reviews them	Registered name, address, GSTIN, grievance officer, support reply time, return and delivery windows. `npm run check:launch` lists each one
3	Cashfree live keys	Payments are in test mode	Complete KYC, set live keys, register the webhook
4	Verified products	The catalogue is in sample mode and refuses real orders	Product records with full ingredient lists and evidence (`npm run catalogue:validate`), then `NEXT_PUBLIC_CATALOGUE_MODE=verified`
5	Approved photography	Placeholder images, one showing another brand's label	Photos for the slots in `docs/product-onboarding.md`
6	Count the stock	Uncounted SKUs cannot be sold	Stockroom, or `npm run db:seed-inventory`
7	Nightly backups	Only 6 hours of history today	Create a private S3-compatible bucket and set the six `BACKUP_*` repository secrets
8	Rotate the Neon password	It appeared in tool output during development	Neon → Roles → reset, then update `DATABASE_URL` in Vercel

Should do soon after

Item	How
Shiprocket	Create the API user and webhook (Chapter 23), apply migration 0022
AI skin chat	Create an Anthropic key with a spend limit, set `ANTHROPIC_API_KEY`
WhatsApp alerts	Renew the Meta token
Email domain	Verify a sending domain (Resend) or keep Gmail with an app password
Error alerts	Set up Sentry (`SENTRY_DSN`) with an alert rule
Web application firewall	Vercel Firewall Bot Protection, or Cloudflare
Paid Neon plan	When traffic grows: more compute, longer history
Reviewed skincare knowledge	Directions, cautions, rules and interactions approved by a qualified reviewer, then published, so routines can include treatments
Delete leftover Neon branches	`loadtest-2026-10-12`, `cart-verify-prompt2`, `catalog-0014-rehearsal`, `rehearsal-0020-0021` (with approval)

Waiting for approval

Item	Status
Migration 0022 (courier columns)	Applied to the local acceptance database only. Needs approval for production
Branch `fix/payment-backout`	Holds the payment back-out fix, the AI skin chat, Shiprocket, the stock autopilot, owner two-step sign-in, the replay-key fix, the outage fallback and the stockroom Help page. Not yet committed or merged

Not built, by design

Feature	Why not	What it would need
Coupons, bundles, free gifts, loyalty points	No business rules exist; advertising them without checkout support would be misleading	Eligibility, stacking, caps, expiry and refund rules; a points ledger
Automatic supplier ordering	Approving spend needs supplier details and a budget rule	Suppliers, prices, lead times, an approval limit
Face-photo analysis	No evaluated, licensed model	A model with a commercial licence, evaluated per skin tone, with calibrated outputs
Customer self-service cancellation and refunds	No approved rule	A cancellation window and refund policy
Newsletter sending	No sending tool	A mailing service that honours unsubscribes
Mobile and tablet layouts	Desktop first, by the owner's choice	Approval of the desktop design

34. Launch-day checklist

A step-by-step order for going live, once the items in Chapter 33 are in hand. Tick each line; do not skip ahead, because each stage depends on the one before.

One week before

Done	Task
☐	Vercel on the Pro plan; sweep schedule changed to every 15 minutes in `vercel.json`
☐	Neon password rotated; `DATABASE_URL` updated; site redeployed and `/api/health` answers "ok"
☐	Nightly backup secrets set; one manual backup run; the file seen in the bucket
☐	One restore rehearsed into a scratch Neon branch, then the branch deleted
☐	Business details filled in; `npm run check:launch` reports nothing outstanding
☐	Policy pages reviewed by a lawyer; grievance officer named
☐	Verified product records validated; photography uploaded; the wrong-brand stock photo gone
☐	Cashfree KYC complete; live keys set; webhook registered
☐	Shiprocket API user, pickup address and webhook set; wallet topped up; migration 0022 applied with approval
☐	Email: a verified domain on Resend, or Gmail within its daily limit; `OWNER_EMAIL` set
☐	WhatsApp: permanent token and an approved template
☐	Anthropic key with a monthly spend limit (if the chat is part of launch)
☐	Sentry alerts, Vercel Firewall Bot Protection
☐	Staff logins set; the owner has signed in with the emailed code; the manager has signed in and read the Help page

The day before

Done	Task
☐	Build with `NEXT_PUBLIC_CATALOGUE_MODE=verified`; `ALLOW_SAMPLE_ORDERS` **not** set
☐	`npm run check:env -- --production` passes
☐	Stock counted for every product in the stockroom
☐	Prices and any launch offer set in the owner console, with an end date
☐	Google sign-in app published; production redirect address registered
☐	Leftover test orders and Neon test branches cleaned up (with approval)

Launch morning: a real end-to-end test

Use your own card or UPI and real address. Every step should be checked before telling anyone the shop is open.

1. **Browse.** Home, collections, a product page: prices and "In stock" correct.
2. **Online order.** Buy one low-priced item with Pay online. Check:

   - the Cashfree page shows the right amount;
   - after paying, the confirmation page shows the order;
   - the confirmation email arrives with a working order link;
   - the owner email and WhatsApp alert arrive;
   - the order appears in the owner console as paid and in the stockroom dispatch queue.
3. **Back-out.** Start another order, press Back on the Cashfree page. Check "nothing was ordered" appears and no email or alert was sent.
4. **Cash on delivery.** Place a COD order. Check the risk check approved it and it appears for dispatch.
5. **Dispatch.** In the stockroom: Mark packed, Book courier, print the label. Check the tracking number on the order page.
6. **Tracking.** When the courier collects, check the order moves to "On its way" and the shipping email arrives.
7. **Refund.** Refund the online order from the Cashfree dashboard.
8. **Chat.** Sign in as a customer and send one message to the skin chat.
9. **Logs.** Vercel → Logs: no errors from the test. Owner console → System: nothing failed.

The first week

Daily	What to watch
Morning summary	Arrives every day around 08:45; orders and stock look right
Owner console → Orders	Held COD orders decided the same day; nothing left needing attention
Owner console → System	No failed or dead deliveries
Vercel → Usage	Function calls, CPU and bandwidth well under the plan
Neon → Monitoring	Database CPU not pinned; no connection errors
Anthropic → Usage	Chat spend in line with ₹0.50 per active person
Shiprocket	Pickups happening; no stuck parcels
Cashfree	Settlements match the owner console's revenue

If anything looks wrong, Chapter 31 has the steps; the quickest safe move after a bad deployment is promoting the previous one in Vercel.

35. History of the build

The system was built in a 30-step plan between 7 and 10 October 2026 (`docs/avyora-30-implementation-prompts.md`), against a written specification and an audit of the earlier site, then extended from 12 to 21 October. The full record, with every test count and verification, is `docs/implementation-progress.md`.

Foundations (7 to 8 October)

Step	What changed
1. Build baseline	A portable build; a rule that a deployment fails rather than bake an outage into cached pages; stable tests
2. Bag prices	The bag priced every line from a frozen copy of the product; now each SKU is priced with the same rule as checkout, quantities and sizes are honoured, and a changed price refuses the order
3. Routine safety	Irritated and very reactive skin get no actives; "prefer not to say" on pregnancy is never treated as "no"; beginners get three steps; no invented directions
4. Honest claims	Removed offers the checkout did not apply ("Buy 2, Get 3rd Free", cashback, free gifts), absolute claims and a dummy WhatsApp number; added automated claim checks
5. SKU identities	Stable ids for every size; catalogue tables; migration rehearsed and reversible
6. Ingredients	A canonical dictionary with ambiguity handling ("vitamin C" never silently means one thing); formulations and directions as reviewable records
8. Consent	Nothing personal is saved without permission; everything saved expires
9. Knowledge releases	Compiled, checksummed, immutable releases with rollback and revocation
10. Durable limits	Rate limits moved from server memory to Postgres; trusted client addresses; bounded request bodies
11. Caching	Pre-built public pages; private pages never cached; the shared bundle cut from 195 KB to 103 KB
12. Account sync	Bag and wishlist merge on sign-in without inflating quantities

Step 7 (shared typed recommendation contracts) was skipped in the order the steps were given, and its role was absorbed by steps 16 and 17.

Personalisation (8 October)

Step	What changed
13. Inference	A Bayesian concern ranker that refuses to show percentages without validated parameters
14. Selection	Safety exclusions before ranking; budget in whole paise; owned products first
15. Weekly planner	A real seven-day schedule that validates every session
16. Routine API	Server recomputation; idempotent saves; owner scoping; guest-to-account transfer
17. Routine finder	The new questionnaire and results, computed on the device with the same engine as the server
18. Ask Avyora	On-device answers with no AI model and no data leaving the device

Design and experience (8 to 9 October)

Step	What changed
19. Reference	The target design measured at three widths
20 to 22. Design system, hero, landing	Rebuilt to the measured reference (section heights within a few pixels); real products and prices, no invented reviews
23. Shop and checkout	The design applied across shopping pages without changing their logic
24. Accounts	Saved routines page; address ownership tests

Step	What changed
25. Staff	Knowledge operations; support messages
26 to 27. Face scan	Built and switched off; no model, no invented scores
28. Privacy lifecycle	Weekly feedback, consent withdrawal, expiry sweeps, broader log redaction
29. Reviews and newsletter	Verified-purchase reviews with moderation; double opt-in newsletter (switched off)
30. Acceptance	Three-width acceptance pass; handoff document

After the plan

Date	Work
9 October	Cashfree replaced Razorpay for online payments
10 October	Re-audit: 22 findings fixed; product onboarding validator; sample and verified catalogue modes
12 October	Backing out of payment no longer orders anything; online orders announced only when paid; load test to 500 simultaneous visitors; production error review; architecture map with graphify; testing strategy
12 October	AI skin chat with Claude Haiku 5.5, server-side urgent-symptom detection, signed-in only, ₹0.50 per person per day
21 October	Security review (replay-key fix); Shiprocket booking and tracking; COD sales counted as revenue once delivered; returned parcels restocked; "on its way" emails; stock autopilot and morning summary; owner two-step sign-in; stockroom Help page; storefront survives a database outage; returned orders leave the dispatch queue and show "Returned" to the customer; analytics count only real sales; expired sign-in codes cleaned up daily; dead "Forgot?" link fixed

Quality over time

Date	Tests passing
7 October (start)	403
8 October	852
9 October	911
10 October	987
12 October	1,086
21 October	1,131

36. Decision log

The important choices, why they were made, and when to revisit them. A future developer should read this before changing any of them.

Money and orders

Store money in whole paise

Decision. Every amount is an integer number of paise. Rupees appear only in forms and outside APIs.

Why. Decimal rupees in floating point drift (0.1 + 0.2 is not 0.3). A total that is one paisa off fails the payment amount check and leaves a paid order unconfirmed. Integers cannot drift.

Revisit. Never.

Never accept a price from the browser

Decision. The browser sends what was chosen; the server prices it. The price the customer saw is sent only so a change can be detected and refused.

Why. Trusting a client price lets anyone buy a serum for ₹1 by editing a request.

Revisit. Never.

One transaction for stock, order and event

Decision. Reserving stock, writing the order and its lines, and recording the event happen in one database transaction.

Why. If any part fails, nothing exists. An order can never exist for goods that were not available, and an event can never announce an order that rolled back.

Revisit. Only if orders move to a separate service, which would need a saga pattern instead.

Refuse a changed price rather than charge it

Decision. If a price changed between showing and ordering, the order is refused before reserving anything, and the new price is shown.

Why. Charging a price the customer never agreed to is a complaint and possibly a chargeback; refusing costs one click.

Announce online orders only when paid

Decision (12 October 2026). Cash-on-delivery orders are announced when placed; online orders when paid.

Why. Customers who backed out of the payment screen were receiving "order confirmed", and the owner was alerted about sales that never happened.

Count COD as paid on delivery

Decision (21 October 2026). A delivered cash-on-delivery order records a captured payment through the normal state machine.

Why. Revenue counts paid orders only, so COD sales were invisible in the owner's figures. Recording through the state machine keeps one path for all money.

Revisit. If COD remittance reports from Shiprocket are later imported, they could confirm the amount actually received.

Payments

Cashfree instead of Razorpay

Decision (9 October 2026). Online payments moved to Cashfree. Razorpay code remains only to settle older orders.

Why. The owner's choice of provider. The payment state machine, event recording and reconciliation were already provider-neutral, so only the provider client and routes changed.

The webhook is authoritative, the browser is not

Decision. The return page asks Cashfree; the signed webhook confirms; reconciliation catches anything missed.

Why. A closed tab or dropped connection must never decide whether money was taken.

"Unknown" is its own answer

Decision. Reconciliation answers paid, not paid, no session, or unknown. Only "not paid" and "no session" allow stock to be released.

Why. An early version cancelled orders after 30 minutes without asking the provider, while the provider was still retrying a successful payment's webhook. A timeout is not a failure.

Data and infrastructure

Postgres for everything durable, including rate limits and the job queue

Decision. No separate queue service, no Redis required. Rate limits, jobs, events and caches of record all live in Postgres.

Why. One system to run, back up and reason about. At this shop's volume a table is not the bottleneck. Serverless memory is neither shared nor durable, so an in-memory rate limit is no limit at all.

Revisit. When rate-limit writes measurably slow the database (tens of thousands of customers a day): move hot policies to Redis. The change is confined to `src/lib/rate-limit.ts`.

Fail closed for costly or sensitive actions

Decision. Sign-in, checkout, payments, the AI chat and staff actions refuse when their rate limiter cannot be reached. Cheap cached reads stay open.

Why. A limiter that silently switches off during a database hiccup removes protection exactly when something is wrong.

Display may fall back; checkout may not

Decision (21 October 2026). If the database is unreachable, shop pages show list prices and "out of stock"; checkout fails.

Why. A whole storefront going down because one stock read failed turned a short outage into a site outage. But a fallback price must never become a charge.

Pre-built pages and a CSP without nonces

Decision. Public pages are pre-built, so the CSP allows inline scripts instead of per-request nonces.

Why. Nonces must be stamped into each response, which pre-built pages cannot do; trying it blanked the site. Pre-building is what makes the shop fast and cheap to serve.

Revisit. If most pages become per-request.

Additive migrations, applied by hand

Decision. Schema changes add rather than alter where possible, are rehearsed on a Neon branch, and are applied to production only with the owner's approval.

Why. Code can then be rolled back without a schema rollback, and no surprise reaches the live database.

Personalisation and content

Unknown is never "no"

Decision. Any unanswered safety question (pregnancy, allergies, age) is unknown, and unknown blocks elective treatments.

Why. The original quiz mapped "prefer not to say" on pregnancy to "no", which made retinoids eligible.

Safety before ranking

Decision. Excluded products never reach the scoring step.

Why. A weighted score can always be overpowered by a large enough weight. An exclusion cannot.

No invented knowledge

Decision. No direction, caution, percentage, review or score is shown unless it exists and was approved.

Why. Skincare advice that sounds authoritative but was never reviewed is a safety and legal risk. The system says "being reviewed" instead.

Ask Avyora without an AI model

Decision. The on-device assistant uses lookups over the approved knowledge release, not a language model.

Why. Free to run, instant, private (questions never leave the device), and it cannot make anything up.

The AI skin chat on Claude Haiku 5.5, metered per person

Decision (12 and 21 October 2026). The chat uses the cheapest current Claude model, requires sign-in, and charges each reply's real cost against ₹0.50 per person per day.

Why. The owner's budget was about ₹1 for 10 to 15 messages. Self-hosting an open model on a cloud GPU costs around ₹80 to ₹90 an hour whether used or not, which is far more at this volume. Metering real usage (rather than counting messages) matches the bill exactly; sign-in makes the budget per person rather than per network address.

Revisit. If volume grows to thousands of chats a day, compare against batch pricing or a self-hosted model.

Urgent symptoms detected by code, not the model

Decision. A fixed list of urgent signs is matched on the server, and the doctor notice is sent before the model's reply.

Why. It works even if the model misreads a message, and a test can prove it.

Operations

Shiprocket as the courier aggregator

Decision (21 October 2026). Deliveries are booked through Shiprocket; its tracking webhooks move orders.

Why. The owner's choice. One integration reaches many couriers, prints labels and reports tracking, which removes a button press per leg for the stockroom.

Courier updates only move forward

Decision. A tracking update can advance an order but never move it back, and a return is accepted only after the parcel left.

Why. Webhooks arrive late and out of order. A delayed "in transit" must not undo "delivered".

The autopilot raises requests, it does not buy

Decision. The morning autopilot creates restock requests and a summary; the owner approves.

Why. Buying commits money to a supplier the system knows nothing about. The useful part, never running out unnoticed, needs no money decision.

A second step for the owner only

Decision (21 October 2026). The owner signs in with a password and an emailed code; the manager with a password.

Why. The owner console controls prices and shows every customer; the stockroom does not. An emergency switch (`OWNER_TWO_STEP=off`) avoids a lock-out if email fails.

PART VIII

Developer reference

The main flows traced step by step through the code, and the request and response formats of the API.

37. Key flows, step by step

Each flow names the file that does each step, so a developer can follow it in the code.

Online payment with Cashfree

```
Browser (checkout)          Avyora server                          Cashfree
------------------          -------------                          --------
Pay  --------------------> POST /api/payments/cashfree/create
                            same-origin, rate limit, createOrder()
                            (prices, stock reserved, order.placed)
                            createCashfreeOrder() ------------------> order + session
                            order.payment_reference set
                            payment state: session_opened
                            reconciliation job scheduled
       <------------------- { paymentSessionId, orderNumber, accessToken }
```

```
rememberAttempt()
Cashfree checkout opens ------------------------------------------> customer pays
                                                                          |
                            POST /api/webhooks/cashfree  <------------+ signed event
                            verify signature (raw body)
                            recordProviderEvent(): once per event id
                            decide(): amount == total -> paid
                            order.paid event -> drain after response
Return page -------------->  POST /api/payments/cashfree/verify
                            asks Cashfree for payments (not the browser)
       <------------------- { status: "paid" }
Confirmation shown; bag cleared; email and owner alert sent
```

Step	File
Create the order and payment session	`src/app/api/payments/cashfree/create/route.ts`, `src/lib/orders.ts`, `src/lib/cashfree.ts`
Remember the attempt in the browser	`src/lib/payment-attempt.ts`, `src/app/checkout/checkout-client.tsx`
Webhook	`src/app/api/webhooks/cashfree/route.ts`
Record and apply events	`src/modules/payments/payment-service.ts`, `state-machine.ts`
Return page	`src/app/checkout/return/page.tsx`, `src/app/api/payments/cashfree/verify/route.ts`
Announce	`src/lib/event-consumers.ts`, `src/lib/order-notifications.ts`

Backing out of the payment screen

1. The customer presses Back or closes Cashfree's page. The checkout page fires `pageshow` (or the return page loads) and finds the remembered attempt.
2. The browser calls `POST /api/payments/cashfree/cancel` with the order number and its access token.
3. The server asks Cashfree to terminate the order (`PATCH /pg/orders/{id}` with `order_status: TERMINATED`) and re-reads its state.
4. **TERMINATED or EXPIRED:** `releaseIfUnpaid` cancels the order and returns its stock, exactly once. Answer: `cancelled`.
5. **PAID** (paid in the meantime): the payment is applied instead. Answer: `paid`.
6. **Anything else** (a payment in progress): nothing changes. Answer: `pending`; the sweep will ask again later.
7. The page resets the Pay button and makes a new idempotency key. A retry with the old key gets `409 attempt_closed`, never a reopened attempt.

A parcel's life

```
Manager: Mark packed        order.status = fulfilled
Manager: Book courier       bookCourier()  (src/modules/shipping/shipments.ts)
                              claim: orders.shipment = {booking:true} where null
                              Shiprocket: login, create adhoc, assign AWB,
                                          generate pickup, generate label
                              orders.awb, orders.shipment saved; audit row
Courier picks up            POST /api/webhooks/courier  (x-api-key checked)
                              status 42 -> shipped, order.shipped event
```

```
                              -> "on its way" email with tracking link
In transit / delayed        recorded on the shipment; order unchanged
Out for delivery (17)       -> out_for_delivery
Delivered (7)               -> delivered; COD recorded as paid
  or Undelivered (21)       attention set; owner alerted
  or RTO delivered (10)     -> returned; stock restored once; refund flagged if prepaid
```

A chat message

1. `POST /api/chat` with the conversation so far (user and assistant turns only).
2. Same-origin check; a 503 if no Anthropic key is configured; a size-bounded body; a strict schema (1 to 40 messages, each 1 to 1,000 characters, the last from the user).
3. A 401 if not signed in.
4. Rate limits checked together: 8 a minute, today's spend under ₹0.50 (checked without adding), and the site-wide 2,000 a day. If any fails: 429, and the model is never called.
5. The history is trimmed to the latest 12 turns, starting with the customer and alternating.
6. The urgent-symptom check runs on the latest message; if it matches, the doctor notice is streamed first.
7. The model is called with the fixed, cached system prompt, adaptive thinking at low effort, and a 1,200-token cap. Text streams back as it is written.
8. After the reply, `replyCostPaise(usage)` is added to the person's daily spend, and a log line records tokens and cost. If the model refused, a polite redirect is sent; if the provider failed, an apology.

The morning sweep

1. Vercel Cron calls `GET /api/cron/sweep` with the `CRON_SECRET` bearer token.
2. Abandoned online orders: each is reconciled with its provider first; only "not paid" or "no session" cancels and restocks.
3. Housekeeping: expired idempotency keys, rate-limit counters, routines, saved answers, scan photos and rows, unconfirmed newsletter sign-ups.
4. Stock autopilot: restock requests raised; the morning summary sent.
5. The response lists the count of each, so a manual run shows what happened.

38. Order and payment lifecycle

Every way an order's two statuses can change, and who or what changes them. Any transition not listed here is refused by the code.

Fulfilment status

From	To	Triggered by	Where
(new)	pending	Checkout creates the order	`createOrder`
pending	paid	A captured payment equal to the total	Payment state machine

From	To	Triggered by	Where
pending, paid	fulfilled ("packed")	Manager or owner: Mark packed. A COD order only once the risk check approved it	Stockroom; owner order page
pending	cancelled	Owner; risk check refusal; morning sweep for abandoned online payment; customer backing out of Cashfree	Owner actions; `cod-risk`; `releaseIfUnpaid`
paid, fulfilled	cancelled	Owner (refund separately in Cashfree)	Owner order page
fulfilled	shipped	Courier pick-up update, or Hand to courier	Courier webhook; stockroom
shipped	out_for_delivery	Courier update, or the stockroom button	Courier webhook; stockroom
shipped, out_for_delivery	delivered	Courier update, or Mark delivered	Courier webhook; stockroom; owner
fulfilled, shipped, out_for_delivery	returned	Courier: delivered back to us (RTO delivered)	Courier webhook
cancelled	paid	A payment arriving after cancellation (flagged for the owner)	Payment state machine

Rules that apply to every change:

- **Compare-and-set.** The update only succeeds if the order is still in the status that was read, so two people (or a person and a courier update) cannot both move it.
- **Courier updates only move forward.** A late "in transit" never undoes "delivered".
- **Stock moves exactly once.** Cancelling or returning claims `stock_restored_at` in the same statement that checks it is empty; a second attempt does nothing.
- **Every change is audited** with who did it (a staff username, or "courier").

Payment status

From	To	Signal	Stock effect
unpaid, failed	pending	A payment session opened (customer pressed Pay)	None
unpaid, pending, failed	authorized	Provider authorised the amount (card holds)	Re-reserved if it had been released
unpaid, pending, authorized, failed	paid	Captured, amount equals the order total	Re-reserved if it had been released; attention if it was sold meanwhile
unpaid, pending	failed	The attempt failed	Released
paid	refunded	Refund recorded	—
any	(unchanged)	Captured amount differs from the total	Rejected and recorded
paid, refunded	(unchanged)	A failed attempt after success	Ignored
paid	(unchanged)	A second capture	Ignored as a duplicate

Cash-on-delivery orders stay **unpaid** until delivered, then receive a captured signal for the order total through the same table.

Risk status (cash on delivery only)

From	To	Triggered by
pending	approved	Score below 40
pending	review	Score 40 or more
pending	rejected	Score 70 or more with at least two signals (order cancelled, stock returned)
review	approved	Owner: Release for dispatch
review	rejected	Owner: Cancel and restock

Online orders are approved without scoring: the money has already moved.

Attention

Set when	Cleared by
Paid after its stock was sold to someone else	The owner, with a note
Paid after the order was cancelled	The owner, with a note
The courier reports it undelivered, lost, damaged, destroyed, untraceable, or returning	The owner, with a note
Returned while paid online (refund due)	The owner, with a note

An order with an attention reason is held from dispatch until it is cleared.

What the customer sees for each state

Internal state	Customer sees
pending, paid	Order placed
fulfilled	Being prepared
shipped	On its way
out_for_delivery	Out for delivery
delivered	Delivered
cancelled, refunded	The tracker stops, with a plain explanation
returned	"Returned": the courier could not deliver and brought it back; anything paid online is refunded (fixed 21 October 2026: it used to show "Order placed")

39. API reference

All routes are under `/api`. Unless stated, a route that changes data requires a same-origin request (the browser's `Origin` must match the site), returns JSON, and on refusal returns `{ "error": "…" }` or `{ "error": { "code", "message" } }` with a matching status code. Private responses carry `Cache-Control: private, no-store`. The full list of 34 addresses is in Appendix B.

Common status codes

Code	Meaning
400	The request failed validation
401	Sign-in required, or a webhook signature or token is wrong
403	Wrong origin, or not your order
404	Not found, or not yours (the two are deliberately indistinguishable)
409	A conflict: price changed, attempt closed, wrong account, stale edit
413	The body is over its size limit
422	A valid request with no possible result (for example no routine fits)
429	Rate limited; `Retry-After` says when to try again
503	A dependency is not configured or unavailable; `limiter_unavailable` when the rate limiter cannot be reached

Catalogue

GET /api/catalog/availability

Current display prices and stock for up to 50 SKUs.

```
GET /api/catalog/availability?skus=retinol::90ml,ha-toner::200ml
200 { prices: { "retinol::90ml": { price, wasPrice, offerLabel, … } }, stock: { "retinol::90ml":
12 }, quoteVersion, validUntil }
```

An uncounted SKU reports 0 (checkout refuses it); `stock` is null only when the deployment has no database. Cached briefly at the edge (15 seconds, then stale for 30).

GET /api/catalog/release

The published knowledge release (manifest and artifacts) the browser's routine engine and Ask Avyora use. Public; no personal data.

Bag, wishlist and sync

Route	Request	Response
`GET /api/cart`	—	`{ lines: [{ productId, size, quantity }] }`
`POST /api/cart`	`{ lines, accountKey? }` (at most 50 lines, 16 KB)	`{ ok, stored }`; 409 if `accountKey` belongs to another account
`GET /api/wishlist`	—	`{ signedIn, productIds }`
`POST /api/wishlist`	`{ productIds }` (at most 100)	`{ signedIn, stored }`
`GET /api/sync`	—	`{ accountKey, cart, wishlist }` for the signed-in account, all null for a guest
`POST /api/sync/merge`	The guest bag and wishlist	The merged state, with any capped or dropped lines explained; 401 if not signed in

Payments

POST /api/payments/cashfree/create

Creates the order and a Cashfree payment session. The body is the checkout input: `email`, `address`, `items` (each `productId`, `size`, `quantity`, optional `expectedUnitPaise`), `idempotencyKey`, and for guests `phoneProof`.

```
200 { paymentSessionId, mode: "sandbox" | "production", orderNumber, accessToken }
409 { error, code: "price_changed" }       price changed since shown
409 { error, code: "attempt_closed" }      that key's attempt was cancelled; make a new key
400 { error }                              validation, stock, sample catalogue
```

POST /api/payments/cashfree/verify

`{ orderNumber }` → `{ status: "paid" | "pending" | "failed" | "not_started", orderNumber }`. Asks Cashfree; never trusts the browser.

POST /api/payments/cashfree/cancel

`{ orderNumber, token }` → `{ status: "cancelled" | "paid" | "pending", orderNumber }`. 403 without the order's access token.

POST /api/webhooks/cashfree

From Cashfree only. Headers `x-webhook-timestamp` and `x-webhook-signature` are verified against the raw body. 401 for a bad signature; 503 (retryable) when the order is not yet visible; otherwise `{ ok, duplicate? }`.

Deliveries

POST /api/webhooks/courier

From Shiprocket only. Header `x-api-key` must equal `SHIPROCKET_WEBHOOK_TOKEN`. Body (fields used): `awb`, `shipment_status_id`, `shipment_status`.

```
401 { error: "Invalid token." }
503 { error: "Courier tracking is not configured." }
200 { ok: true, result: "advanced" | "recorded" | "flagged" | "unknown_awb" }
200 { ok: true, ignored: "…" }   not a tracking update
```

Staff

POST /api/admin/login

`{ username, password, code? }` (4 KB).

```
200 { ok: true, role: "owner" | "manager" }   and the session cookie
200 { step: "code" }                          owner: a code was emailed; send again with code
401 { error }                                 wrong credentials or code
502 { error }                                 the code could not be emailed
503 { error }                                 staff sign-in not configured
```

POST /api/admin/logout

Clears the staff session.

Personal records

Route	Purpose
`POST /api/consent`	`{ purpose, policyVersion }`, where purpose is `routine_saving` (or `photo_processing` when hosted scans are on). Answers `{ consent: { purpose, policyVersion, grantedAt } }`
`DELETE /api/consent?purpose=`	Withdraws at once
`POST /api/routines`	Saves a routine. Needs the `Idempotency-Key` header and routine-saving consent; body is the profile and release id only. The server recomputes the result
`GET /api/routines`	The owner's saved routines
`GET, DELETE /api/routines/[id]`	One routine, with its validity (current, outdated, revoked); delete always answers 204
`POST /api/routines/[id]/feedback`	Weekly feedback (accounts only, once a week)
`POST /api/scans, POST /api/scans/[id]/upload, GET/DELETE /api/scans/[id]`	Optional face-photo sessions; 503 `scan_disabled` while switched off

Customer contact

Route	Request	Response
`POST /api/support`	Name, email, message, contact consent (4 KB)	`201 { ok, reference }`
`POST /api/reviews`	`{ productId, rating 1–5, body 20–2000 characters }`; signed in, with a delivered order containing the product	Stored unpublished until moderated
`POST /api/newsletter`	Email and consent (switched off)	The same answer whatever the address's state

AI chat

POST /api/chat

`{ messages: [{ role: "user" | "assistant", content }] }`, 1 to 40 messages, each up to 1,000 characters, the last from the user; no other fields (48 KB).

```
200 text/plain stream   the reply, preceded by the doctor notice when urgent
401 { error: "Please sign in to use the skin chat." }
429 { error: { code: "rate_limited", message }, retryAfterSeconds }
503 { error: "The skin chat is not switched on yet." }
```

Operations

Route	Caller	Purpose
`GET /api/health`	Uptime monitors	`200 { status: "ok", database: "ok" }` or `503 { status: "degraded", database: "down" }`
`GET /api/cron/sweep`	Vercel Cron (bearer `CRON_SECRET`)	Morning sweep; answers counts
`GET /api/cron/events`	Vercel Cron	Event drain backstop
`GET /api/cron/scan-retention`	Any scheduler with `CRON_SECRET` (not scheduled yet)	Photo-retention backstop; must run at least hourly before hosted photo analysis is ever enabled
`POST /api/activity`	The site	Anonymous usage events
`/api/auth/[...nextauth]`	Auth.js	Customer sign-in
`GET /api/account/deliver-to`	The site	The header's "Deliver to" line: first name, city and PIN, fetched after the page loads so pages stay cacheable

40. Making common changes

Recipes for the changes a developer is most likely to be asked for, with the files involved and the checks that must pass. Every change should end with `npm run typecheck`, `npm run lint`, `npm test` and `npm run build:offline` passing, as CI requires.

Getting started

1. Install Node 24 and npm, then `npm ci`.
2. Copy `.env.example` to `.env.local` and fill in at least `DATABASE_URL` (a Neon branch, never production), `AUTH_SECRET` and `SESSION_SECRET`.
3. `npm run db:migrate`, then `npm run db:seed-inventory 25` so products are sellable.
4. `npm run dev` and open http://localhost:9002.
5. For a disposable local database instead of a Neon branch, start Docker Desktop and use the `acceptance` launch entry (port 9006), which creates and migrates its own Postgres container.

Code style: Prettier with single quotes, semicolons, trailing commas and 120-character lines (`npm run format`). Comments explain **why**, not what. Deliberate simplifications carry a `ponytail:` comment naming their limit and how to lift it.

Add a product or a size

1. Add it to `src/data/mock-data.ts`: id, slug, name, category, concerns, sizes with list prices.
2. Give it a routine role in `src/data/routine-roles.ts` (cleanse, moisturise, protect, treatment, optional or none).
3. Generate the catalogue rows: `npx tsx scripts/catalog-seed-sql.ts --rows`, then create a migration for them with `npx drizzle-kit generate --custom --name catalog_<what>` and paste the rows in. The catalogue migration test fails until the database matches the file.

4. For a verified product, prepare its onboarding record (format: `docs/onboarding/example-products.synthetic.json`) and run `npm run catalogue:validate -- products.json --evidence evidence.json` until it is READY.
5. After deploying and migrating, set its stock in the stockroom and its price in the owner console if it differs from the list price.
6. `npm run db:check-catalog` confirms the file and the database agree.

Change the delivery fee or free-delivery threshold

Edit `FREE_SHIPPING_THRESHOLD_PAISE` and `STANDARD_SHIPPING_PAISE` in `src/lib/money.ts`. Every banner, the bag's progress bar, checkout, the shipping policy and the FAQ read them from there. A test (`content-claims.test.ts`) fails if a delivery figure is written anywhere else.

Add a setting

1. Read it with `process.env.NAME` in the one module that needs it. Never in a client component unless it starts with `NEXT_PUBLIC_` and is not secret.
2. Document it in `.env.example` with a comment saying what it is for and where the value comes from.
3. If a deployment would break without it, add a check to `scripts/check-env.mjs` (it reports shapes and lengths, never values).
4. Appendix C of this manual picks it up automatically on the next rebuild.

Change the database schema

1. Edit `src/db/schema.ts`. Prefer adding tables and nullable columns over changing or dropping existing ones.
2. `npx drizzle-kit generate --name <what>` writes the next numbered SQL file in `drizzle/`. Read it: it should contain only the intended change.
3. Tests build their databases from the migrations, so a schema the migrations do not create fails the tests.
4. Rehearse on a Neon branch (`DATABASE_URL=<branch> npm run db:migrate`), then ask the owner before applying to production.
5. If the change is destructive, write a data check first and a rollback script in `docs/runbooks/`.

Add an API route safely

Every route that changes data follows the same order, which is what keeps the API safe. A minimal skeleton:

```
export const runtime = 'nodejs';
export const dynamic = 'force-dynamic';

export async function POST(request: Request) {
  if (!isSameOrigin(request)) return Response.json({ error: 'Invalid request origin.' }, {
status: 403 });
  const session = await auth().catch(() => null);           // if it needs a person
  if (!session?.user?.id) return Response.json({ error: 'Sign in first.' }, { status: 401 });
  const limited = await limit([{ policy: 'myPolicy', subject: { kind: 'user', id: session.user.id
} }]);
  if (!limited.allowed) return limitResponse(limited);
  const read = await readBoundedJson(request, BODY_LIMITS.myRoute);   // size cap
```

```
  if (!read.ok) return read.response;
  const parsed = schema.strict().safeParse(read.json);                // strict zod schema
  if (!parsed.success) return Response.json({ error: 'Invalid request.' }, { status: 400 });
  // ... scope every query to session.user.id ...
  return privateJson({ ok: true });                                    // private, no-store
}
```

Then:

- Add the body limit to `BODY_LIMITS` in `src/lib/request-body.ts` and the policy to `DEFAULT_POLICIES` in `src/lib/rate-limit.ts` (with `onFailure: 'closed'` unless it is a cheap read).
- If the address is private, make sure `src/lib/cache-policy.ts` marks it so (a test checks every private path).
- Write a route-level test with `createMigratedDb()` and a foreign-owner case that must get 404.

Webhooks are the exception to the origin check: verify a signature or a shared token in constant time instead, refuse everything when the secret is not configured, and record events by the provider's id so repeats change nothing.

Add an owner-only action

In a `'use server'` file under `src/app/admin/`:

1. First line: `if (!(await isAdmin())) return;` (or check `getStaffSession()` and its role). Never rely on the page or middleware.
2. Validate the form fields; never trust a status or amount from the form. Compare-and-set on the state you read, so two staff pressing different buttons cannot both win.
3. `recordAudit({ actor, actorRole, action, entityType, entityId, before, after })`.
4. `revalidatePath(...)` for every screen that shows the changed data, and `revalidateProduct(id)` if the storefront shows it.

Add an event and a reaction to it

1. Add the name to `EventName` in `src/lib/events.ts`.
2. Emit it with `emitEvent(name, subjectId, { orderId, orderNumber }, tx)` **inside the transaction** that makes the change.
3. Handle it in a consumer in `src/lib/event-consumers.ts`: re-read the current state, act, and throw if the action failed and should be retried (up to 5 attempts, then visible in System).
4. If it should happen promptly, schedule the drain after the response (`after(...)`, as the payment service does).

Change a rate limit without deploying

Set `RATE_LIMIT_OVERRIDES` in Vercel to JSON, for example:

```
{"chatSpendDay":{"limit":100,"ipLimit":100},"checkout":{"limit":20}}
```

Invalid values are ignored and reported; unknown policy names are ignored. Redeploy to apply.

Change the AI chat's behaviour

- **What it says:** `SYSTEM_PROMPT` in `src/modules/skin-chat/chat.ts`. Keep it free of dates, random values or per-request data, so it stays identical between requests and the provider can cache it. The tests check the safety phrases are present.
- **Urgent signs:** the `URGENT` list in `src/modules/skin-chat/red-flags.ts`. Add a phrase, then a test that it is flagged and a test that a harmless sentence is not.
- **Cost:** `USD_PER_MTOK` and `INR_PER_USD` in `chat.ts` must be updated if the model or its prices change; `MAX_REPLY_TOKENS` and `MAX_HISTORY_MESSAGES` bound each message.
- **Model:** `CHAT_MODEL`. Check the new model's prices and parameters before switching.

Add a CMS content type

Add a Zod schema to `CONTENT_TYPES` in `src/modules/cms/content-types.ts` with length limits on every field, add a form in the content editor, and read it through `content-read.ts` with a cache key. Publishing already runs the claim rules and invalidates the cache.

Write a test

- **Pure logic** (pricing, risk scoring, planning): plain unit tests, no database.
- **Anything touching the database:** `const { client, db } = await createMigratedDb();` then `vi.mock('@/db', () => ({ db, isDatabaseConfigured: () => true }))`. This runs the real migrations in PGlite.
- **Outside services:** pass a fake `fetch` or client (see the Shiprocket and Cashfree tests). Never call a real provider from a test.
- **Then break the code on purpose** and check the test fails. A test that cannot fail protects nothing.

Rebuild this manual

The source is in `docs/manual/source/`. After code changes, run `node extract.mjs` (re-reads tables, routes, settings, migrations and tests) and `node build.js ../Avyora-Technical-Manual.docx`, then update the contents list in Word. Add new chapters as Markdown files; a heading written `# Title {#label}` is numbered automatically, and the label written inside double square brackets anywhere in the text becomes its chapter number.

PART IX

Quick reference

One-page cards for the stockroom and the owner, and the words used in this manual.

41. Quick-reference cards

These cards are written to be printed and kept by the packing table or the owner's desk.

Card 1. Stockroom: sending today's orders

1. Sign in at `/admin-login` with the manager login. You land on **Dispatch**.
2. Work from the **top**: oldest first.
3. Skip any card with a **red line** (on hold, or not paid). Those wait for the owner.
4. Pick the items under **Pack** and box them.
5. Press **Mark packed**.
6. Press **Book courier**. If it shows an error, tell the owner and move to the next card.
7. Press **Print the shipping label**, stick it on, and put the box with the others for pickup.
8. When the courier collects, hand over every labelled box. The orders update by themselves from here.

> Cash on delivery? The card says "Cash on delivery: collect payment at the door". The courier collects it; you do nothing extra.

Card 2. Stockroom: stock arrived or removed

1. Open **Stock**. Find the product and size.
2. Count what is in your hand.
3. Type `+` and the number (for example `+24`), or `-` and the number for damaged or samples (`-2`).
4. Press **Apply**. Check the "On shelf" number changed.

Never type the new total: type the change.

Card 3. Stockroom: a parcel came back

1. The courier marks it returned; the stock is put back automatically.
2. Open the box. Check every item.
3. Damaged items: open **Stock**, type a minus number for them, press **Apply**, and tell the owner.

Card 4. Owner: every morning (5 minutes)

1. Read the **morning summary** on WhatsApp or email: orders, sales, what is low.
2. Open `/admin` → **Requests**:
   - Restock requests: order from your supplier, then press **Ordered** (or **Decline**).
   - Support messages: reply by email, then **Mark answered**.
   - Reviews: **Publish** genuine ones, **Reject** spam.
3. Open **Orders**:
   - Held COD orders: phone the customer, then **Release for dispatch** or **Cancel and restock**.
   - Orders needing attention: read the reason, act (refund, resend, contact the customer), then clear it with a note.

Card 5. Owner: every week

Check	Where
Sales and stock cover by product	Owner console → Analytics
Nothing stuck in the system	Owner console → System (failed or dead jobs and deliveries: press Replay once fixed)
Courier problems	Shiprocket panel: undelivered and returned parcels
AI chat spend	Anthropic console → Usage
Payments match	Cashfree dashboard settlements against Analytics revenue
Stock count	Spot-check a few shelves against the Stock screen

Card 6. Owner: changing a price or running an offer

1. Owner console → **Pricing** → find the product and size.
2. **Price** is the normal price. **Offer** is the sale price (must be lower). **Label** is the badge text. **Ends** is optional.
3. Press **Save**. The shop, bag and checkout all change together.
4. To end an offer early: clear **Offer**, press **Save**.

Card 7. Something is wrong

Symptom	First step	Chapter
"Something went wrong" on the site	Note the reference number; check `/api/health`	31
Customer says they paid, order unpaid	Check the Cashfree dashboard; never mark it paid by hand	31
No order emails	Owner console → System, failed deliveries	31
WhatsApp alerts stopped	Renew the Meta token	31
Book courier fails	Read the card's message; PIN code or Shiprocket login	31
Owner sign-in code not arriving	Check email settings; emergency `OWNER_TWO_STEP=off`	31
Data deleted by mistake	Neon point-in-time restore within 6 hours	31

Card 8. Who to contact

For	Where
Hosting, deployments, logs	Vercel dashboard (vercel.com)
Database	Neon console (console.neon.tech)
Payments	Cashfree merchant dashboard
Couriers and labels	Shiprocket panel (app.shiprocket.in)
AI chat billing	Anthropic console (console.anthropic.com)
WhatsApp alerts	Meta for Developers
Code changes	The repository on GitHub and the developer who maintains it

42. Owner questions and answers

Short answers to the questions owners ask most, with where to read more.

Orders and money

A customer says they paid but I see "unpaid". Should I mark it paid? No, and there is no button for it on purpose. Check the Cashfree dashboard. If Cashfree shows the payment, the webhook or the next reconciliation will mark the order paid on its own. If Cashfree shows no payment, the customer was not charged. (Chapter 31.)

Why does revenue not include an order I can see? Revenue counts paid orders only. Online orders count when paid; cash-on-delivery orders count when delivered. Cancelled, refunded and returned orders never count.

Can I give a customer a discount code? Not yet: there are no coupons, by design, until the rules are defined (Chapter 33). You can run an offer on any product in Pricing, with an optional end date.

What happens if I change a price while someone is checking out? Their order is refused with a message showing the new price, before anything is reserved. They see the new total and can place the order again. Nobody is charged a price they did not see.

Why was a cash-on-delivery order cancelled by itself? The risk check refused it: a score of 70 or more with at least two signals agreeing (for example filler text in the address and several COD orders the same day). The stock went back on the shelf. If you think it was genuine, contact the customer; one strong signal on its own only ever holds an order for you to decide.

How do I refund someone? From the Cashfree dashboard. Then cancel the order in the owner console if it has not shipped, so the stock goes back.

Stock

The site says "Out of stock" but I have the product. Nobody has counted it in the system. The stockroom adds the count in Stock (type + and the number, press Apply).

Why did I get a restock request I did not ask for? The morning autopilot raises one when a product will run out within the supplier lead time at its recent sales rate. The note says how fast it sells and how many days are left. Approve it once you have ordered from the supplier, or decline it.

Can the system order from my supplier automatically? No. It needs supplier details and a spending rule from you first (Chapter 24).

Deliveries

Which courier will be used? Whichever Shiprocket assigns by the priority set in your Shiprocket account: cheapest, fastest or recommended.

A parcel shows "delayed". Do I need to do anything? No. Delays are recorded on the parcel; the order stays "On its way". You are alerted only for problems that need you: undelivered, lost, damaged or returning.

What happens to a returned parcel? When Shiprocket reports it back with you, the order becomes "returned" and the stock goes back on the shelf automatically. If the customer had paid online, the order is flagged so you remember to refund them.

Staff and access

The manager is away. Can I pack orders? Yes. The stockroom is open to the owner too; use "Stockroom" from the owner console.

How do I change a staff password? Generate a new hash (`node scripts/hash-password.mjs 'new-password'`), set it in Vercel, redeploy (Chapter 29). Changing `SESSION_SECRET` signs everyone out.

I am not getting the sign-in code email. Check the spam folder and that email sending works (order emails arriving?). In an emergency, set `OWNER_TWO_STEP=off` in Vercel and redeploy, sign in, fix email, then remove it.

Can the manager see revenue or change prices? No. The manager's screens show what to pack and stock levels only. Every owner action is checked on the server, so guessing an address does not help.

The AI chat

How much will the chat cost me? At most ₹0.50 per signed-in person per day, plus a site-wide cap of 2,000 messages a day. A typical message costs ₹0.03 to ₹0.07. Set a monthly spend limit in the Anthropic console as well.

Can I make it recommend specific products more? It already recommends Avyora products first, matched to the concern, and never other brands. To change its wording or priorities, a developer edits its instructions (Chapter 40).

Does it give medical advice? No. It is instructed never to diagnose or prescribe, and urgent symptoms trigger a "see a doctor" notice from the server before the AI answers.

The site

Is the site fast enough for a sale or a big campaign? Public pages are served from Vercel's cache and barely touch the servers. The limits are the plan (move to Pro) and the database's compute (upgrade Neon if traffic grows). See Chapter 30.

What happens if the database goes down? Product pages stay up and show items as out of stock; checkout, sign-in and staff screens show a "try again" message. Nothing is lost; everything resumes when the database answers.

How do I put the shop on my own domain? Add it in Vercel → Settings → Domains, then update `NEXT_PUBLIC_SITE_URL`, the Google sign-in redirect, and the Cashfree and Shiprocket webhook addresses (Chapter 29).

Where is a full list of everything the site does? Appendix B lists every page and API address; Appendix C every setting.

43. Glossary

Term	Meaning
AWB	Air waybill: the courier's tracking number for a parcel
Backorder	Selling a SKU past zero on purpose (made to order, pre-order)
Cash on delivery (COD)	The customer pays the courier at the door
CMS	Content management system: where the owner edits site text
Consent	A recorded permission, for example to save a routine
Cron	A job that runs on a schedule
CSP	Content Security Policy: a browser rule limiting where scripts and data may come from
Edge	Vercel's servers close to the visitor, which serve cached pages
Event (domain event)	A record that something happened, such as an order being paid, used to trigger follow-up work
Idempotency key	A random key that makes repeating the same request harmless
INCI	The standard international names for cosmetic ingredients
ISR	Incremental static regeneration: pre-built pages refreshed in the background
Knowledge release	An immutable, checksummed bundle of approved skincare knowledge
Migration	A numbered change to the database structure
Neon branch	A copy of the database, used for rehearsals and tests
Outbox	Writing events in the same transaction as the change, so neither can exist without the other
Paise	One hundredth of a rupee; all money is stored in paise
PGlite	Postgres compiled to run inside the test process
Rate limit	A cap on how often something can be done
Reconciliation	Asking the payment provider what really happened to an order
RTO	Return to origin: the courier brings an undelivered parcel back
Server action	A form submission handled on the server
SKU	Stock-keeping unit: one product in one size
Sweep	The daily job that releases abandoned stock and tidies expired data
Token (AI)	A unit of text the AI model reads or writes; billing is per token
Webhook	A message a provider sends to the site when something happens

PART X

Reference appendices

Generated from the code itself on 21 October 2026, so they list exactly what exists: every database table and column, every page and API address, every setting, every database migration and every automated test file.

Appendix A. Database tables

The database has **51 tables** with **428 columns**, defined in `src/db/schema.ts` and created by the migrations in `drizzle/`. Money is always stored as whole **paise** (₹1 = 100 paise) in integer columns, so no rounding error can enter a total. Times are stored with time zone.

Status values

Type	Allowed values
order_status	pending, paid, fulfilled, shipped, out_for_delivery, delivered, cancelled, refunded, returned
payment_status	unpaid, pending, authorized, paid, failed, refunded
content_status	draft, published, archived
job_status	queued, running, succeeded, dead

users

Customer accounts (Google, email or phone sign-in).

Column	Type	Required	Notes
id	text	no	—
name	text	no	—
email	text	yes	—
email_verified	timestamp	no	—
image	text	no	—
phone	text	no	—
phone_verified	timestamp	no	—
password_hash	text	no	—
created_at	timestamp	yes	—

otp_codes

One-time sign-in and verification codes, stored hashed, five guesses each, ten minutes.

Column	Type	Required	Notes
id	text	no	—

Column	Type	Required	Notes
identifier	text	yes	—
channel	text	yes	—
code_hash	text	yes	—
expires_at	timestamp	yes	—
attempts	integer	yes	—
consumed_at	timestamp	no	—
created_at	timestamp	yes	—

accounts

Links a customer to an outside sign-in provider such as Google (Auth.js).

Column	Type	Required	Notes
user_id	text	no	—
type	text	yes	—
provider	text	yes	—
provider_account_id	text	yes	—
refresh_token	text	no	—
access_token	text	no	—
expires_at	integer	no	—
token_type	text	no	—
scope	text	no	—
id_token	text	no	—
session_state	text	no	—

sessions

Customer sign-in sessions, stored in the database so they can be revoked.

Column	Type	Required	Notes
session_token	text	yes	primary key
user_id	text	no	—
expires	timestamp	yes	—

verification_tokens

Auth.js email verification tokens.

Column	Type	Required	Notes
identifier	text	yes	—
token	text	yes	—
expires	timestamp	yes	—

addresses

Saved delivery addresses, and the address frozen onto each order.

Column	Type	Required	Notes
id	text	no	—
user_id	text	no	links to another table
full_name	text	yes	—
line1	text	yes	—
line2	text	no	—
landmark	text	no	—
city	text	yes	—
state	text	yes	—
postal_code	text	yes	—
country	text	yes	—
phone	text	yes	—
is_default	boolean	yes	—
created_at	timestamp	yes	—

catalog_products

One row per product: stable id, slug, name, category.

Column	Type	Required	Notes
id	text	yes	primary key
slug	text	yes	—
name	text	yes	—
category	text	yes	—
published	boolean	yes	—
created_at	timestamp	yes	—
updated_at	timestamp	yes	—

catalog_variants

One row per sellable size (SKU) with a stable id such as retinol-90ml.

Column	Type	Required	Notes
id	text	yes	primary key
product_id	text	no	—
legacy_stock_key	text	yes	—
size_label	text	yes	—
volume_ml	numeric	no	—

Column	Type	Required	Notes
active	boolean	yes	—

carts

A shopping bag kept on the server, for guests and accounts.

Column	Type	Required	Notes
id	text	no	—
user_id	text	no	links to another table
anonymous_id	text	no	—
created_at	timestamp	yes	—
updated_at	timestamp	yes	—

cart_items

Lines in a server-side bag: SKU and quantity, never a price.

Column	Type	Required	Notes
id	serial	yes	primary key
cart_id	text	no	—
product_id	text	yes	—
size	text	yes	—
variant_id	text	no	links to another table
quantity	integer	yes	—
added_at	timestamp	yes	—

orders

Every order: customer, totals in paise, payment and fulfilment status, risk check, courier shipment.

Column	Type	Required	Notes
id	text	no	—
order_number	text	yes	—
user_id	text	no	links to another table
email	text	yes	—
status	orderStatus	yes	—
payment_status	paymentStatus	yes	—
subtotal	integer	yes	—
discount	integer	yes	—
shipping	integer	yes	—
tax	integer	yes	—
total	integer	yes	—

Column	Type	Required	Notes
currency	text	yes	—
shipping_address_id	text	no	links to another table
shipping_address	jsonb	no	—
payment_provider	text	no	—
payment_reference	text	no	—
notes	text	no	—
idempotency_key	text	no	—
stock_restored_at	timestamp	no	—
fraud_status	text	yes	—
fraud_score	integer	no	—
fraud_reasons	jsonb	no	—
attention_reason	text	no	—
awb	text	no	—
shipment	jsonb	no	—
created_at	timestamp	yes	—
updated_at	timestamp	yes	—

order_items

The lines of an order, with name, size and price frozen at the time of sale.

Column	Type	Required	Notes
id	serial	yes	primary key
order_id	text	no	—
product_id	text	yes	—
variant_id	text	no	links to another table
product_name	text	yes	—
size	text	yes	—
unit_price	integer	yes	—
quantity	integer	yes	—
line_total	integer	yes	—
pricing_snapshot	jsonb	no	—

reviews

Product reviews from verified buyers, unpublished until staff approve.

Column	Type	Required	Notes
id	text	no	—
product_id	text	yes	—

Column	Type	Required	Notes
user_id	text	no	links to another table
order_id	text	no	links to another table
rating	integer	yes	—
title	text	no	—
body	text	no	—
published	boolean	yes	—
created_at	timestamp	yes	—

consent_records

Append-only record of each permission granted or withdrawn (saving routines, photos).

Column	Type	Required	Notes
id	uuid	yes	primary key
user_id	text	no	links to another table
anonymous_owner_hash	text	no	—
purpose	text	yes	—
policy_version	text	yes	—
granted_at	timestamp	yes	—
withdrawn_at	timestamp	no	—

skin_profiles

Quiz answers a customer chose to save, with expiry.

Column	Type	Required	Notes
id	uuid	yes	primary key
user_id	text	no	links to another table
anonymous_owner_hash	text	no	—
consent_id	uuid	yes	—
consent_purpose	text	yes	—
schema_version	integer	yes	—
answers	jsonb	yes	—
created_at	timestamp	yes	—
updated_at	timestamp	yes	—
expires_at	timestamp	yes	—

routine_results

Saved routines with the engine, knowledge release and versions that produced them.

Column	Type	Required	Notes
id	text	no	—
user_id	text	no	links to another table
anonymous_id	text	no	—
anonymous_owner_hash	text	no	—
profile_id	uuid	no	links to another table
consent_id	uuid	no	—
consent_purpose	text	no	—
schema_version	integer	yes	—
input_hash	text	no	—
kb_release	text	no	—
engine_version	text	no	—
model_version	text	no	—
inference_version	text	no	—
idempotency_key	text	no	—
request_hash	text	no	—
answers	jsonb	yes	—
result	jsonb	yes	—
created_at	timestamp	yes	—
expires_at	timestamp	no	—

routine_schedule_slots

One row per step of a saved weekly routine (day, morning or evening, position).

Column	Type	Required	Notes
routine_id	text	no	—
day	smallint	yes	—
session	text	yes	—
position	smallint	yes	—
role	text	yes	—
optional	boolean	yes	—
product_id	text	no	—
sku_id	text	no	—
owned_item_id	text	no	—

scan_sessions

Optional face-photo sessions (feature switched off); photo kept at most 24 hours.

Column	Type	Required	Notes
id	uuid	yes	primary key
user_id	text	no	links to another table
anonymous_owner_hash	text	no	—
consent_id	uuid	yes	—
consent_purpose	text	yes	—
status	text	yes	—
mode	text	yes	—
model_version	text	no	—
quality	jsonb	no	—
result	jsonb	no	—
object_key	text	no	—
object_expires_at	timestamp	no	—
created_at	timestamp	yes	—
expires_at	timestamp	yes	—

routine_feedback

Weekly "how is it going" answers on a saved routine.

Column	Type	Required	Notes
id	uuid	yes	primary key
routine_id	text	no	—
user_id	text	no	—
week	smallint	yes	—
adherence	text	yes	—
tolerability	text	yes	—
reported_change	text	yes	—
created_at	timestamp	yes	—

support_requests

Messages sent through the contact form, answered from the owner console.

Column	Type	Required	Notes
id	uuid	yes	primary key
name	text	yes	—
email	text	yes	—
message	text	yes	—
status	text	yes	—
resolution	text	no	—

Column	Type	Required	Notes
created_at	timestamp	yes	—
resolved_at	timestamp	no	—
resolved_by	text	no	—

newsletter_subscribers

Newsletter sign-ups with double opt-in (feature switched off).

Column	Type	Required	Notes
id	uuid	yes	primary key
email	text	yes	—
status	text	yes	—
consent_version	text	yes	—
token_hash	text	yes	—
created_at	timestamp	yes	—
confirmed_at	timestamp	no	—
unsubscribed_at	timestamp	no	—

activity_events

Anonymous usage events for analytics.

Column	Type	Required	Notes
id	serial	yes	primary key
user_id	text	no	links to another table
anonymous_id	text	no	—
session_id	text	no	—
name	text	yes	—
props	jsonb	no	—
path	text	no	—
created_at	timestamp	yes	—

inventory

Stock on hand per SKU, low-stock level, and whether backorders are allowed.

Column	Type	Required	Notes
id	serial	yes	primary key
product_id	text	yes	—
size	text	yes	—
variant_id	text	no	links to another table
quantity	integer	yes	—

Column	Type	Required	Notes
low_stock_threshold	integer	yes	—
allow_backorder	boolean	yes	—
updated_at	timestamp	yes	—

restock_requests

Requests for more stock, raised by the manager or the morning autopilot, closed by the owner.

Column	Type	Required	Notes
id	text	no	—
product_id	text	yes	—
size	text	yes	—
variant_id	text	no	links to another table
requested_quantity	integer	yes	—
quantity_at_request	integer	yes	—
note	text	no	—
status	text	yes	—
requested_by	text	yes	—
resolved_at	timestamp	no	—
created_at	timestamp	yes	—

product_pricing

Owner-set prices and offers per SKU, versioned so two edits cannot overwrite each other.

Column	Type	Required	Notes
id	text	no	—
product_id	text	yes	—
size	text	yes	—
variant_id	text	no	links to another table
price	integer	yes	—
sale_price	integer	no	—
offer_label	text	no	—
offer_starts_at	timestamp	no	—
offer_ends_at	timestamp	no	—
updated_by	text	no	—
version	integer	yes	—
updated_at	timestamp	yes	—

rate_limits

Durable counters behind every rate limit and the AI chat spending budget.

Column	Type	Required	Notes
key	text	yes	primary key
count	integer	yes	—
window_start	timestamp	yes	—
window_seconds	integer	no	—

domain_events

The event log (outbox): order placed, paid, shipped, and so on, written in the same transaction.

Column	Type	Required	Notes
id	serial	yes	primary key
name	text	yes	—
subject	text	no	—
payload	jsonb	yes	—
request_id	text	no	—
created_at	timestamp	yes	—

event_deliveries

Which consumer has handled which event, with retries and failures.

Column	Type	Required	Notes
event_id	integer	no	—
consumer	text	yes	—
status	text	yes	—
attempts	integer	yes	—
last_error	text	no	—
next_attempt_at	timestamp	no	—
updated_at	timestamp	yes	—

consumer_registrations

Where each event consumer started reading the log.

Column	Type	Required	Notes
consumer	text	yes	primary key
start_event_id	integer	yes	—
registered_at	timestamp	yes	—

environmental_cache

Cached UV, humidity and air-quality readings per region.

Column	Type	Required	Notes
region	text	yes	primary key
uv_index	integer	no	—
humidity	integer	no	—
pm25	integer	no	—
fetched_at	timestamp	yes	—

ingredients

The canonical ingredient dictionary (INCI names, classes, caution flags).

Column	Type	Required	Notes
id	text	yes	primary key
inci_name	text	yes	—
common_name	text	yes	—
synonyms	jsonb	no	—
prescription_only	boolean	yes	—
pregnancy_caution	boolean	yes	—
photosensitising	boolean	yes	—
class	text	yes	—

ingredient_aliases

Alternative names for ingredients, including deliberately ambiguous ones.

Column	Type	Required	Notes
alias	text	yes	primary key
ingredient_id	text	no	links to another table
ambiguous_candidates	jsonb	no	—

evidence_sources

Sources cited by reviewed knowledge records.

Column	Type	Required	Notes
id	text	yes	primary key
title	text	yes	—
url	text	no	—
source_type	text	yes	—
retrieved_at	text	yes	—
limitations	text	yes	—

formulations

Versioned full ingredient lists per product (none recorded yet).

Column	Type	Required	Notes
id	text	yes	primary key
product_id	text	no	—
version	integer	yes	—
coverage	text	yes	—
full_inci	text	no	—
source_id	text	no	—
reviewed_by	text	yes	—
reviewed_at	text	yes	—

formulation_ingredients

Each ingredient position in a formulation, with concentration if known.

Column	Type	Required	Notes
formulation_id	text	no	—
position	smallint	yes	—
inci_label	text	yes	—
ingredient_id	text	no	links to another table
concentration_known	boolean	yes	—
concentration	numeric	no	—
unit	text	no	—

usage_profiles

Approved directions per formulation version (none approved yet).

Column	Type	Required	Notes
formulation_id	text	no	—
session	text	yes	—
frequency	text	yes	—
directions	text	yes	—
max_weekly_uses	smallint	no	—
evidence_ids	jsonb	no	—
reviewed_by	text	yes	—
reviewed_at	text	yes	—

ingredient_interactions

Rules about ingredients that should not be combined.

Column	Type	Required	Notes
id	text	no	—
ingredient_a	text	no	—
ingredient_b	text	no	—
tier	integer	yes	—
summary	text	yes	—
advice	text	yes	—
citation	text	no	—

routine_items

Legacy routine line items.

Column	Type	Required	Notes
id	text	no	—
user_id	text	no	—
label	text	yes	—
ingredient_ids	jsonb	no	—
prescribed	boolean	yes	—
created_at	timestamp	yes	—

idempotency_keys

Makes retried requests safe: the same key returns the same result.

Column	Type	Required	Notes
id	bigserial	yes	primary key
scope	text	yes	—
key	text	yes	—
request_hash	text	yes	—
status	text	yes	—
response_code	integer	no	—
response_body	jsonb	no	—
resource_type	text	no	—
resource_id	text	no	—
request_id	text	no	—
created_at	timestamp	yes	—
expires_at	timestamp	no	—

audit_logs

Who changed what, and when: prices, stock, order status, publishing, courier bookings.

Column	Type	Required	Notes
id	bigserial	yes	primary key
actor	text	yes	—
actor_role	text	no	—
action	text	yes	—
entity_type	text	yes	—
entity_id	text	yes	—
old_value	jsonb	no	—
new_value	jsonb	no	—
reason	text	no	—
request_id	text	no	—
created_at	timestamp	yes	—

payment_events

Every payment provider event, recorded once, with what it changed.

Column	Type	Required	Notes
id	bigserial	yes	primary key
provider	text	yes	—
provider_event_id	text	yes	—
event_type	text	yes	—
order_id	text	no	—
provider_payment_id	text	no	—
amount	integer	no	—
payload	jsonb	yes	—
outcome	text	no	—
request_id	text	no	—
received_at	timestamp	yes	—

wishlist_items

Saved products for signed-in customers.

Column	Type	Required	Notes
user_id	text	no	—
product_id	text	yes	—
added_at	timestamp	yes	—

cms_documents

Site content managed in the CMS: product copy, articles, pages.

Column	Type	Required	Notes
id	text	no	—
type	text	yes	—
slug	text	yes	—
status	contentStatus	yes	—
draft	jsonb	yes	—
published	jsonb	no	—
version	integer	yes	—
published_version	integer	no	—
published_at	timestamp	no	—
created_at	timestamp	yes	—
updated_at	timestamp	yes	—
updated_by	text	no	—

cms_revisions

Every saved version of a CMS document, for restore.

Column	Type	Required	Notes
id	bigserial	yes	primary key
document_id	text	no	—
version	integer	yes	—
body	jsonb	yes	—
action	text	yes	—
actor	text	yes	—
request_id	text	no	—
created_at	timestamp	yes	—

media_assets

Images uploaded through the CMS.

Column	Type	Required	Notes
id	text	no	—
storage_key	text	yes	—
content_type	text	yes	—
bytes	integer	yes	—
sha256	text	yes	—
alt	text	yes	—

Column	Type	Required	Notes
uploaded_by	text	yes	—
created_at	timestamp	yes	—

jobs

Background job queue with leases, retries and dead letters.

Column	Type	Required	Notes
id	bigserial	yes	primary key
type	text	yes	—
payload	jsonb	yes	—
status	jobStatus	yes	—
attempts	integer	yes	—
max_attempts	integer	yes	—
run_at	timestamp	yes	—
locked_until	timestamp	no	—
locked_by	text	no	—
last_error	text	no	—
dedupe_key	text	no	—
request_id	text	no	—
created_at	timestamp	yes	—
updated_at	timestamp	yes	—
completed_at	timestamp	no	—

export_watermarks

Progress markers for exporting events to analytics storage.

Column	Type	Required	Notes
name	text	yes	primary key
last_event_id	integer	yes	—
updated_at	timestamp	yes	—

kb_releases

Immutable, checksummed releases of the skincare knowledge base.

Column	Type	Required	Notes
id	text	yes	primary key
schema_version	integer	yes	—
is_fixture	boolean	yes	—
status	text	yes	—

Column	Type	Required	Notes
manifest	jsonb	yes	—
artifacts	jsonb	yes	—
checksum	text	yes	—
stored_by	text	yes	—
stored_at	timestamp	yes	—
published_at	timestamp	no	—
revoked_at	timestamp	no	—
revoked_reason	text	no	—

kb_active_release

Which knowledge release is live, and the previous one for rollback.

Column	Type	Required	Notes
singleton	boolean	yes	primary key
release_id	text	no	—
previous_release_id	text	no	links to another table
activated_by	text	yes	—
activated_at	timestamp	yes	—

scan_admissions

Quota records for photo scans, kept after scans are deleted.

Column	Type	Required	Notes
id	uuid	yes	primary key
scan_session_id	uuid	no	links to another table
user_id	text	no	links to another table
anonymous_owner_hash	text	no	—
ip_key	text	no	—
admitted_at	timestamp	yes	—

scan_attempts

Paid model calls per scan (at most two).

Column	Type	Required	Notes
scan_session_id	uuid	no	—
attempt	smallint	yes	—
started_at	timestamp	yes	—

Appendix B. Pages and API addresses

The site has **50 pages** and **34 API addresses**. Square brackets mark a variable part, for example `/products/[slug]`.

Pages

Address	What it is
/	Home page
/account	Customer account area
/account/addresses	Customer account area
/account/addresses/[id]/edit	Customer account area
/account/addresses/new	Customer account area
/account/data	Customer account area
/account/orders	Customer account area
/account/routines	Customer account area
/admin	Owner console overview
/admin-login	Staff sign-in (owner and manager)
/admin/analytics	Owner: sales, units, days of cover
/admin/catalogue	Owner: catalogue record preview
/admin/content	Owner: CMS
/admin/content/[type]/[slug]	Owner: CMS
/admin/content/media	Owner: CMS
/admin/inventory	Owner: stock levels
/admin/knowledge	Owner: knowledge releases
/admin/orders	Owner: all orders
/admin/orders/[orderNumber]	Owner: one order, with risk and attention panels
/admin/pricing	Owner: prices and offers
/admin/requests	Owner: restock requests, support messages, reviews to moderate
/admin/system	Owner: queues, dead jobs, scan backlog
/assistant	Ask Avyora: on-device answers, no AI
/checkout	Checkout
/checkout/return	Return page after the Cashfree payment screen
/collections	Shop: all products with filters
/compare	Compare products
/contact	Policy and contact page
/design-system	Internal design review page (not linked)
/journal	Journal articles from the CMS
/journal/[slug]	Journal articles from the CMS
/login	Customer sign-in and sign-up

Address	What it is
/manager	Stockroom: dispatch queue
/manager/help	Stockroom: step-by-step help
/manager/requests	Stockroom: restock request history
/manager/stock	Stockroom: stock counts and requests
/newsletter	Newsletter confirm and unsubscribe
/orders/[orderNumber]	Order confirmation and tracking (owner or signed link only)
/orders/[orderNumber]/invoice	Printable invoice
/privacy	Policy and contact page
/products/[slug]	Product page
/refund-policy	Policy and contact page
/routine-finder	Routine finder questionnaire and weekly plan
/scan	Optional face photo (switched off)
/shipping-policy	Policy and contact page
/signup	Customer sign-in and sign-up
/skin-chat	AI skin chat (signed-in customers)
/terms	Policy and contact page
/track-order	Track an order by number and email
/wishlist	Wishlist

API addresses

Address	Methods	Purpose (from the code)
/api/account/deliver-to	GET	What the header's "Deliver to" line shows: a first name, a city and a PIN
/api/activity	POST	Records a behavioural event
/api/admin/login	POST	Two-step sign-in for the owner: after the password, a six-digit code sent to OWNER_EMAIL
/api/admin/logout	POST	Clears the admin session cookie.
/api/auth/[...nextauth]	GET, POST	—
/api/cart	GET, POST	Anonymous visitors get a stable id so their cart can be stored and later merged.
/api/catalog/availability	GET	Current display prices and stock for the SKUs in a bag
/api/catalog/release	GET	The knowledge the browser's routine engine runs on (spec section 20): the active release's manifest and artifacts
/api/chat	POST	The AI skin chat
/api/consent	POST, DELETE	Routine saving (guests too), and photo processing (signed-in accounts only, and only while hosted scans are switched on)
/api/cron/events	GET	Node runtime: the Postgres driver cannot open a socket at the edge.
/api/cron/scan-retention	GET	Photo retention only, for a frequent schedule (re-audit A06)
/api/cron/sweep	GET	Node runtime: the Postgres driver cannot open a socket at the edge.

Address	Methods	Purpose (from the code)
/api/health	GET	Liveness and readiness for uptime monitors
/api/newsletter	POST	Explicit, unticked-by-default agreement to receive the newsletter.
/api/payments/cashfree/cancel	POST	Cashfree states after which no payment can ever succeed.
/api/payments/cashfree/create	POST	Creates our order, then the matching Cashfree order (Cashfree's order id is our order number)
/api/payments/cashfree/verify	POST	Called by the return page after Cashfree checkout
/api/payments/razorpay/create	POST	What a payment session returns, stored and replayed by the idempotency claim.
/api/payments/razorpay/verify	POST	Verifies the handshake Razorpay Checkout returns to the browser
/api/reviews	POST	A review from an account with a delivered order of the product
/api/routines	GET, POST	Saves a routine
/api/routines/[id]	GET, DELETE	An owned routine and whether its knowledge is current, outdated or revoked
/api/routines/[id]/feedback	POST	Weekly feedback on an owned routine: bounded enums only, accounts only, once per week.
/api/scans	POST	Starts a hosted scan session: signed-in, live photo consent, within the durable quotas.
/api/scans/[id]	GET, DELETE	Status only, never the image
/api/scans/[id]/upload	POST	The raw image bytes (JPEG, PNG or WebP, at most 4 MB)
/api/support	POST	The visitor agreed to be contacted about this request only.
/api/sync	GET	Opaque key of the signed-in account, or null for a guest.
/api/sync/merge	POST	Folds the browser's guest bag and wishlist into the signed-in account
/api/webhooks/cashfree	POST	Cashfree payment webhook: the authoritative record of what was paid
/api/webhooks/courier	POST	Constant-time, length-independent comparison (both sides hashed first).
/api/webhooks/razorpay	POST	Razorpay webhook — the authoritative record of what was actually paid
/api/wishlist	GET, POST	The signed-in customer's wishlist

Appendix C. Settings (environment variables)

There are **67 settings**, documented in `.env.example`. They are set in Vercel under Project → Settings → Environment Variables, and locally in `.env.local`. **Never commit real values.** Run `npm run check:env -- --production` to check a deployment's settings without printing any secret.

Setting	What it is for
DATABASE_URL	Any standard Postgres connection string. Neon today, RDS/Aurora after the AWS move — nothing else in the codebase changes. Neon: https://neon.tech -> project -> connection string (pooled)
AUTH_SECRET	Generate the secret with: openssl rand -base64 32

Setting	What it is for
AUTH_GOOGLE_ID	Google Cloud Console -> APIs & Services -> Credentials -> OAuth client ID (type: Web application). Authorised redirect URI must be exactly: http://localhost:9002/api/auth/callback/google (local) https://YOUR-DOMAIN/api/auth/callback/google (production)
AUTH_GOOGLE_SECRET	—
ADMIN_USERNAME	Generate with: node scripts/hash-password.mjs 'your-admin-password'
ADMIN_PASSWORD_HASH	PBKDF2 hash, as pbkdf2:<iterations>:<salt>:<hash>. The password is never stored.
SESSION_SECRET	Signs admin session cookies. At least 32 characters.
OWNER_TWO_STEP	Owner two-step sign-in: when OWNER_EMAIL is set and email delivery works, the owner must also enter a 6-digit code emailed to OWNER_EMAIL. Set to "off" only if email is broken and you are locked out.
GMAIL_USER	Gmail with an app password (Google account -> Security -> 2-Step Verification -> App passwords). Mail is always sent from GMAIL_USER. About 500 messages a day; move to Resend before real volume. Gmail wins if both are set.
GMAIL_APP_PASSWORD	—
RESEND_API_KEY	resend.com -> API Keys. EMAIL_FROM must be an address on a domain you have verified with Resend, or mail is rejected or lands in spam. Without these, the "email me a code" option is not offered at all — better than offering it and sending nothing.
EMAIL_FROM	—
TWILIO_ACCOUNT_SID	First configured wins: Twilio Verify, then Fast2SMS, then MSG91. With any of them set, guests must verify their mobile number at checkout. Twilio Verify: Twilio creates, sends and checks the code; no Twilio number or DLT registration needed. Use an API key (Console -> API keys), not the auth token, and a Verify service (Console -> Verify -> Service
TWILIO_API_KEY_SID	—
TWILIO_API_KEY_SECRET	—
TWILIO_VERIFY_SERVICE_SID	—
FAST2SMS_API_KEY	Fast2SMS OTP route: needs website verification in the Fast2SMS panel first.
MSG91_AUTH_KEY	MSG91: Transactional SMS to Indian numbers requires DLT registration with TRAI: your business registered with an operator, and the message template approved in advance. This is a legal requirement, not a provider one — messages from an unregistered sender are dropped by the carriers. msg91.com -> Auth Key, and the approved OTP template id. Without
MSG91_OTP_TEMPLATE_ID	—
DEMO_IDENTIFIERS	Email codes cannot reach anyone but the Resend account owner until a sending domain is verified, and SMS cannot be delivered at all until DLT registration completes. Both take time; the features are finished now. List your own addresses and numbers here and those identifiers may use those methods anyway — the code is shown on screen instead of bein
OWNER_EMAIL	Where the shop's own copy of each order goes. The customer's confirmation is sent regardless (it needs GMAIL_* or RESEND_* above); this is the alert to you.

Setting	What it is for
WHATSAPP_TOKEN	WhatsApp alerts via Meta's Cloud API — developers.facebook.com/apps WHATSAPP_TOKEN permanent access token for the app WHATSAPP_PHONE_NUMBER_ID the sending number's id, not the number itself WHATSAPP_TO where alerts go, with country code and no +, e.g. 91XXXXXXXXXX A business-initiated message outside a 24-hour window must use an approved template o
WHATSAPP_PHONE_NUMBER_ID	—
WHATSAPP_TO	—
WHATSAPP_TEMPLATE	—
WHATSAPP_TEMPLATE_LANG	—
NEXT_PUBLIC_SITE_URL	Public site URL, used to build the order link inside the confirmation email.
CRON_SECRET	Stock is reserved when an order is created, before payment. Cancellation and payment failure return it — but a customer who simply closes the tab fires neither, so those units would sit reserved forever. Vercel Cron calls /api/cron/sweep every 15 minutes (see vercel.json) and the endpoint requires this secret: left open, it would be a way for anyon
SELLER_NAME	Your registration. Without SELLER_GSTIN the invoice page says so plainly rather than printing an invalid tax invoice. SELLER_STATE decides the tax split: a buyer in the same state is billed CGST + SGST, anyone else IGST. Same total either way — different heads at filing time.
SELLER_GSTIN	—
SELLER_STATE	—
CASHFREE_APP_ID	Cashfree Merchant Dashboard -> Developers -> API Keys. Start with sandbox keys (CASHFREE_ENV=sandbox); production keys need completed KYC and CASHFREE_ENV=production. Without these, checkout still works but offers cash on delivery only. Webhook: Dashboard -> Developers -> Webhooks -> Add URL: https://YOUR-DOMAIN/api/webhooks/cashfree (payment event
CASHFREE_SECRET_KEY	—
CASHFREE_ENV	—
RAZORPAY_KEY_ID	Razorpay Dashboard -> Settings -> API Keys -> Generate Key. Start in Test mode; live keys need completed KYC. The secret is shown once at creation and never again. Without these, checkout still works but offers cash on delivery only.
RAZORPAY_KEY_SECRET	—
RAZORPAY_WEBHOOK_SECRET	Razorpay Dashboard -> Settings -> Webhooks -> Add New Webhook URL: https://YOUR-DOMAIN/api/webhooks/razorpay Events: payment.captured, payment.failed The webhook is the authoritative record of payment: the browser callback can be lost (closed tab, dead connection), so money must not depend on it.
NEXT_PUBLIC_SENTRY_DSN	sentry.io -> create a project -> Client Keys (DSN). Without a DSN nothing is sent anywhere and the SDK never initialises; structured JSON error logs still go to stdout, which Vercel collects today and CloudWatch will collect after the AWS move. NEXT_PUBLIC_ is required for the browser half. Session replay is disabled in code: it records what custom
SENTRY_DSN	—
SHIPROCKET_EMAIL	With these set, the manager's dispatch screen shows "Book courier" on packed orders, and Shiprocket's tracking updates move orders along by themselves. Create an API user in Shiprocket: Settings > API > Add New API User (its email must differ from your main Shiprocket login). Pickup location is the nickname of the pickup address exactly as saved in

Setting	What it is for
SHIPROCKET_PASSWORD	Courier: Shiprocket (optional)
SHIPROCKET_PICKUP_LOCATION	Courier: Shiprocket (optional)
SHIPROCKET_WEBHOOK_TOKEN	Tracking webhook: Shiprocket > Settings > API > Webhooks. URL: https://<your-site>/api/webhooks/courier and a long random security token, the same value here. Without it the webhook refuses every request. Generate one: openssl rand -hex 32
SHIPROCKET_PARCEL_WEIGHT_KG	Parcel size used for every order (kg / cm). Defaults: 0.5 kg, 15 x 12 x 8.
SHIPROCKET_PARCEL_LENGTH_CM	Courier: Shiprocket (optional)
SHIPROCKET_PARCEL_BREADTH_CM	Courier: Shiprocket (optional)
SHIPROCKET_PARCEL_HEIGHT_CM	Courier: Shiprocket (optional)
REDIS_REST_URL	Redis over HTTPS (Upstash REST API). Unset: each instance uses only its own small in-memory cache and reads Postgres on a miss. Postgres stays the record either way; losing Redis costs speed, never correctness.
REDIS_REST_TOKEN	Shared cache (optional)
CACHE_L2	Development/tests only: CACHE_L2=memory uses an in-process stand-in for Redis.
STORAGE_S3_ENDPOINT	Any S3-compatible bucket (Cloudflare R2, AWS S3, Backblaze B2, MinIO). Unset in development: files go to public/media-local/. Unset in production: uploads are switched off, with a message saying so.
STORAGE_S3_BUCKET	Object storage for CMS media (optional)
STORAGE_S3_REGION	Object storage for CMS media (optional)
STORAGE_S3_ACCESS_KEY_ID	Object storage for CMS media (optional)
STORAGE_S3_SECRET_ACCESS_KEY	Object storage for CMS media (optional)
STORAGE_PUBLIC_BASE_URL	The public URL prefix objects are served from (bucket domain or CDN).
STORAGE_ANALYTICS_BUCKET	Analytics landing zone: a separate, private bucket on the same endpoint. Never the media bucket, which is public. Unset: no export (development writes to .data/lake instead).
KAFKA_REST_URL	Kafka via the Confluent REST Proxy v3 API. Unset: nothing is relayed and checkout is unaffected either way; the stream is downstream of the outbox.
KAFKA_CLUSTER_ID	Event stream (optional)
KAFKA_API_KEY	Event stream (optional)
KAFKA_API_SECRET	Event stream (optional)
KAFKA_TOPIC_PREFIX	Event stream (optional)
DATABASE_READ_URL	Heavy reads that tolerate seconds of lag (sales reports) go here. Unset: everything reads the primary. Checkout, carts, payments and cache fills always read the primary regardless.
DATABASE_READ_MAX_LAG_SECONDS	Beyond this many seconds behind, reads go back to the primary.
RATE_LIMIT_KEY_SECRET	Secret for hashing limiter keys (IPs, emails). Falls back to AUTH_SECRET. Changing it restarts every rate-limit window.
RATE_LIMIT_OVERRIDES	Only on a proxy that OVERWRITES X-Forwarded-For (ALB, CloudFront). Vercel is detected automatically. Leave unset elsewhere: the header is spoofable. TRUSTED_PROXY=x-forwarded-for Optional JSON overrides of the starting limits, e.g. RATE_LIMIT_OVERRIDES={"login":{"limit":8,"ipLimit":40}}

Setting	What it is for
ANTHROPIC_API_KEY	AI skin chat (/skin-chat), Claude Haiku 5.5. Without a key the chat says it is not switched on yet. Create a key at console.anthropic.com and set a monthly spend limit there too. Limits: chatMinute, chatSpendDay (₹0.50 per person per day, in paise) and chatGlobalDay in RATE_LIMIT_OVERRIDES.

Appendix D. Database migrations

Each migration is a SQL file applied in order by `npm run db:migrate`. Production migrations are applied by hand, after the owner approves, against the production connection string.

#	File
0000	0000_new_bloodscream.sql
0001	0001_grey_bug.sql
0002	0002_great_iron_patriot.sql
0003	0003_mysterious_tiger_shark.sql
0004	0004_sleepy_shiver_man.sql
0005	0005_true_spot.sql
0006	0006_pink_stature.sql
0007	0007_new_stephen_strange.sql
0008	0008_heavy_sentinel.sql
0009	0009_awesome_gabe_jones.sql
0010	0010_petite_iron_lad.sql
0011	0011_redundant_omega_red.sql
0012	0012_late_karen_page.sql
0013	0013_nasty_slipstream.sql
0014	0014_catalog_variants.sql
0015	0015_knowledge_records.sql
0016	0016_personal_records.sql
0017	0017_knowledge_releases.sql
0018	0018_durable_limits.sql
0019	0019_routine_api.sql
0020	0020_support_requests.sql
0021	0021_newsletter_subscribers.sql
0022	0022_order_shipments.sql

Appendix E. Automated test files

100 test files, run with `npm test`. Files ending `.integration.test.ts` run against a real Postgres engine (PGlite) built from the same migrations as production.

Test file
src/app/(auth)/login/__tests__/auth-errors.test.ts
src/app/api/admin/__tests__/login-two-step.integration.test.ts
src/app/api/catalog/availability/__tests__/route.test.ts
src/app/api/health/__tests__/health.test.ts
src/app/api/payments/cashfree/__tests__/backout.integration.test.ts
src/app/api/support/__tests__/support.integration.test.ts
src/app/routine-finder/__tests__/quiz.test.ts
src/app/routine-finder/__tests__/routine-session.integration.test.ts
src/app/routine-finder/__tests__/routine-unlock.test.ts
src/components/admin/__tests__/promotion-model.test.ts
src/components/nv/home/__tests__/photos.test.ts
src/db/__tests__/read-router.test.ts
src/infrastructure/__tests__/event-retry.integration.test.ts
src/infrastructure/__tests__/idempotency.integration.test.ts
src/infrastructure/__tests__/jobs.integration.test.ts
src/infrastructure/__tests__/request-id.test.ts
src/infrastructure/__tests__/sigv4.test.ts
src/infrastructure/__tests__/tiered-cache.test.ts
src/lib/__tests__/account-sync.integration.test.ts
src/lib/__tests__/account-sync.test.ts
src/lib/__tests__/addresses-ownership.integration.test.ts
src/lib/__tests__/analytics.integration.test.ts
src/lib/__tests__/analytics.test.ts
src/lib/__tests__/auth.test.ts
src/lib/__tests__/cart-and-wishlist.integration.test.ts
src/lib/__tests__/cart.test.ts
src/lib/__tests__/cashfree.test.ts
src/lib/__tests__/checkout-phone.test.ts
src/lib/__tests__/checkout-quote.integration.test.ts
src/lib/__tests__/client-ip-and-body.test.ts
src/lib/__tests__/cn.test.ts
src/lib/__tests__/cod-risk.test.ts
src/lib/__tests__/content-claims.test.ts

Test file
src/lib/__tests__/demo-access.test.ts
src/lib/__tests__/edge-rate-limit.test.ts
src/lib/__tests__/google-linking.integration.test.ts
src/lib/__tests__/guest-phone-check.integration.test.ts
src/lib/__tests__/interactions.integration.test.ts
src/lib/__tests__/inventory.integration.test.ts
src/lib/__tests__/inventory.test.ts
src/lib/__tests__/invoice.test.ts
src/lib/__tests__/money.test.ts
src/lib/__tests__/notify-providers.test.ts
src/lib/__tests__/observability.test.ts
src/lib/__tests__/order-access.test.ts
src/lib/__tests__/order-idempotency.integration.test.ts
src/lib/__tests__/order-progress.test.ts
src/lib/__tests__/order-status-flow.test.ts
src/lib/__tests__/otp.integration.test.ts
src/lib/__tests__/price-source-of-truth.test.ts
src/lib/__tests__/pricing.test.ts
src/lib/__tests__/quote-client.test.ts
src/lib/__tests__/rate-limit.integration.test.ts
src/lib/__tests__/rate-limit.multi-instance.test.ts
src/lib/__tests__/razorpay.test.ts
src/lib/__tests__/regions.test.ts
src/lib/__tests__/rendering-policy.test.ts
src/lib/__tests__/routine-engine.test.ts
src/lib/__tests__/routine-guidance.test.ts
src/lib/__tests__/security.test.ts
src/lib/__tests__/shop-paths.test.ts
src/lib/__tests__/sql-injection.test.ts
src/lib/__tests__/stock-restoration.integration.test.ts
src/lib/__tests__/twilio.test.ts
src/lib/__tests__/unit-price.test.ts
src/modules/ai/__tests__/scan-inference.integration.test.ts
src/modules/assistant/__tests__/assistant.test.ts
src/modules/assistant/__tests__/order-lookup.integration.test.ts
src/modules/catalog/__tests__/catalog-migration.integration.test.ts
src/modules/catalog/__tests__/display-matches-charge.integration.test.ts

Test file
src/modules/catalog/__tests__/onboarding.test.ts
src/modules/catalog/__tests__/storefront-outage.test.ts
src/modules/cms/__tests__/cms.integration.test.ts
src/modules/ingredients/__tests__/import.integration.test.ts
src/modules/ingredients/__tests__/ingredients.test.ts
src/modules/inventory/__tests__/autopilot.integration.test.ts
src/modules/knowledge/__tests__/admin.integration.test.ts
src/modules/knowledge/__tests__/compile.test.ts
src/modules/knowledge/__tests__/releases.integration.test.ts
src/modules/newsletter/__tests__/newsletter.integration.test.ts
src/modules/payments/__tests__/payments.integration.test.ts
src/modules/payments/__tests__/state-machine.test.ts
src/modules/personal/__tests__/personal-records.integration.test.ts
src/modules/personal/__tests__/routines-api.integration.test.ts
src/modules/personalization/core/__tests__/audit-regressions.test.ts
src/modules/personalization/core/__tests__/bayes.test.ts
src/modules/personalization/core/__tests__/planner.test.ts
src/modules/personalization/core/__tests__/selection.test.ts
src/modules/recommendations/__tests__/recommendations.integration.test.ts
src/modules/reviews/__tests__/reviews.integration.test.ts
src/modules/scans/__tests__/admission.integration.test.ts
src/modules/scans/__tests__/image-pipeline.test.ts
src/modules/scans/__tests__/sessions.integration.test.ts
src/modules/search/__tests__/catalog-search.test.ts
src/modules/shipping/__tests__/shipping-flow.integration.test.ts
src/modules/shipping/__tests__/shiprocket.test.ts
src/modules/skin-chat/__tests__/skin-chat.test.ts
src/modules/__tests__/admin-concurrency.integration.test.ts
src/modules/__tests__/background-jobs.integration.test.ts
src/test/migrated-db.test.ts